HEXWARDEN

Security Audit Report

Transient Labs Stacks v2.8.0

AUDITOR
Justin Kalland

VERSION
v1

PUBLICATION DATE
2026-10-20

Introduction

This report presents the results of a time-bounded security assessment of Transient Labs Stacks v2.8.0, conducted by Justin Kalland using HΞXWARDΞN from October 3 through October 19, 2026.

Summary

Security risk is concentrated in two areas: lifecycle isolation and settlement finality. Administrative calls and authorizations intended for an earlier listing or mint round are not always bound to that generation. If executed late, they can change or cancel a successor sale, alter its price or access rules, waive fees, or redirect proceeds. No Critical or High-severity findings were identified, but the repeated nature of this pattern makes it the clearest systemic concern.

Auction settlement also depends on mutable fees, royalty resolution, transfer validation, sanctions checks, token behavior, and recipient compatibility after bids or assets have been committed. Changes or failures in those dependencies can reduce seller proceeds, strand funds or NFTs, or delay finalization. Remediation should prioritize binding operations to explicit generation identifiers, committing settlement-critical terms before escrow, validating payout and asset recipients, and providing reliable recovery paths when external dependencies fail.

Public issue totals

- Findings: 35
- Observations: 27

Contents

Introduction

Summary

Public issue totals

Scope

Measured source inventory

Methodology

HΞXWARDΞN statistics

Proofs of concept

Disclaimer

Findings

TITLE	SEVERITY	STATUS
HEX-001: Royalty-resolution failures can be finalized as zero roy…	Medium	New
HEX-002: Round-blind seller-fee waivers can exempt successor mint…	Medium	New
HEX-003: Stale protocol-fee removal can waive a successor listing	Medium	New
HEX-004: Unversioned listing updates can apply obsolete terms to …	Medium	New
HEX-005: Stale payout authorization can redirect successor-listin…	Medium	New
HEX-006: Stale price updates can underprice successor mint rounds	Medium	New
HEX-007: Unversioned drop lifecycle calls can replace successor s…	Medium	New
HEX-008: Round-blind payout updates can redirect successor-drop p…	Medium	New
HEX-009: Unversioned NFT recovery claims can withdraw later escrow	Medium	New
HEX-010: Failed payouts can strand credits owned by non-calling c…	Medium	New
HEX-011: Post-bid fee and royalty changes can reduce locked selle…	Medium	New
HEX-012: Royalty-gated transfer validation can strand escrowed NF…	Medium	New
HEX-013: Post-escrow validator replacement can defeat auction fin…	Medium	New
HEX-014: Directional transfer policies can strand NFTs in auction…	Medium	New
HEX-015: Sender-blocked ERC-20 payouts can finalize NFT sales aga…	Medium	New
HEX-016: Royalty-engine cache poisoning can suppress secondary-sa…	Medium	New
HEX-017: Pending purchases can consume allowance in a different s…	Low	New
HEX-018: Unsafe percentage arithmetic can block high-unit auction…	Low	New
HEX-019: Unversioned cancellation calls can terminate successor l…	Low	New
HEX-020: Final-window top-ups can suppress higher competing bids	Low	New
HEX-021: Future-ID ERC-1155 drops can survive collection-authorit…	Low	New

TITLE	SEVERITY	STATUS
HEX-022: Velocity purchases can persist unrepresentable phase end…	Low	New
HEX-023: Post-bid collection ownership changes can suppress confi…	Low	New
HEX-024: Unmetered NFT lookups can exhaust auction-settlement gas	Low	New
HEX-025: Sanctions-oracle errors can bypass failed-payout recovery	Low	New
HEX-026: Recipient-restricted bidders can force the seven-day set…	Low	New
HEX-027: Privileged token transfers can break bid-backed auction …	Low	New
HEX-028: Malformed dependency responses can block auction settlem…	Low	New
HEX-029: Mint settlement can pay a fee receiver after it becomes …	Low	New
HEX-030: Selectable drops are incompatible with Editions metadata…	Low	New
HEX-031: Round-blind access and timing updates can alter successo…	Low	New
HEX-032: Round-blind velocity and supply updates can prematurely …	Low	New
HEX-033: Stale listing authorizations can recreate canceled sales…	Low	New
HEX-034: Seller-fee multiplication can reject representable mint …	Low	New
HEX-035: Highest bidders can extend auctions beyond the configure…	Low	New

Observations

TITLE	STATUS
OBS-001: Mint-target compatibility depends on a Transient Labs access-control in…	New
OBS-002: Paid ERC-721 mint recipients may lack a usable recovery path	New
OBS-003: Unvalidated static deadlines can make drop phases unreadable	New
OBS-004: Pausing does not preserve active auction bidding time	New
OBS-005: Ownership renunciation can remove recovery from later oracle failure	New
OBS-006: Direct ERC-20 transfers to the auction house cannot be recovered as sur…	New
OBS-007: Receiver-checking ERC-721 collections cannot enter auction escrow	New
OBS-008: Uncapped native payout callbacks can starve subsequent mint execution	New
OBS-009: Winning contracts may lack a usable path for auction awards	New
OBS-010: Fixed-price NFT recipients are not checked for acceptance capability	New
OBS-011: Expired listings can be renewed in removed currencies	New
OBS-012: Unbounded selectable supply can make aggregate bitmap reads unavailable	New
OBS-013: Presale documentation omits the required recipient pre-hash	New
OBS-014: External callbacks can make listing events report stale receivers	New
OBS-015: Drop lifecycle events omit round identity	New
OBS-016: Deprecated sanctions sources can silently stop receiving updates	New
OBS-017: Decay-rate documentation can lead administrators to disable velocity sa…	New
OBS-018: Recipient-retention checks can reject valid hook-enabled ERC-20 payouts	New
OBS-019: Ownership renunciation while paused can permanently disable sales	New

TITLE	STATUS
OBS-020: Terminal authority changes can eliminate every remaining mint-issuance …	New
OBS-021: The minimum signed decay rate can disable velocity purchases	New
OBS-022: Extreme velocity rates can reject otherwise permitted purchase batches	New
OBS-023: Positive velocity adjustments can overflow the stored duration	New
OBS-024: Auction self-destinations can leave assets in protocol custody	New
OBS-025: Self-address payout receivers can place mint proceeds in protocol custo…	New
OBS-026: Global seller-fee changes can reprice active mint drops	New
OBS-027: Late selected-index conflicts amplify reverted purchase gas	New

Scope

The assessment covered the 16 source files listed below for Transient Labs Stacks v2.8.0. These scoped files match the Transient Labs `tl-stacks` repository at commit `3a52148d850b354b6307c819cb01e8e2805e937a`. Tests, dependencies, and other compiled-support sources were available to support analysis but were not part of the reviewed production scope.

Files in scope

- `src/TLAuctionHouse.sol`
- `src/TLStacks1155.sol`
- `src/TLStacks721.sol`
- `src/helpers/CreatorLookup.sol`
- `src/helpers/ICreatorLookup.sol`
- `src/helpers/IRoyaltyEngineV1.sol`
- `src/helpers/IRoyaltyLookup.sol`
- `src/helpers/RoyaltyLookup.sol`
- `src/lib/CurrencyAllowlist.sol`
- `src/lib/IChainalysisSanctionsOracle.sol`
- `src/lib/SanctionsCompliance.sol`
- `src/lib/TransferHelper.sol`
- `src/utils/CommonUtils.sol`
- `src/utils/TLAuctionHouseUtils.sol`
- `src/utils/TLStacks1155Utils.sol`
- `src/utils/TLStacks721Utils.sol`

Measured source inventory

tl-stacks

- Tree SHA-256: `9dc7edb5eca4ec237a451c9c7b21a1f9c4217a8e811582fa8ec2131572651d4d`
- Scoped sources: 16 files, 1,731 nSLOC
- Compiled support: 85 files, 22,398 nSLOC
- Tests: 26 files, 14,124 nSLOC
- Scoped declarations: 6 contracts, 2 abstract contracts, 8 interfaces, 0 libraries

Methodology

The assessment combined manual review with agentic investigation through HΞXWARDΞN, Justin Kalland's private EVM audit framework. Justin directed the investigation and questioned system assumptions; agents pursued lines of inquiry and tested suspected vulnerabilities. Their evidence informed further review, testing, and changes in direction. Every reported finding and observation was validated through at least one independent proof of concept (PoC), with Justin reviewing the supporting evidence before inclusion.

HΞXWARDΞN Stats

- LLM tokens consumed: 3,516,085,217

- Cumulative agentic time: 640:21:39
- Agentic runs: 2,309

Proofs of concept

This assessment includes 119 executable proofs of concept covering all 62 reported findings, observations, and known issues. Full executable proofs of concept are omitted from this standalone report and are available only in package-bearing report formats.

Findings

HEX-001: Royalty-resolution failures can be finalized as zero royalties

- ID: HEX-001
- Status: New
- Class: Finding
- Severity: Medium
- Location: RoyaltyLookup.getRoyalty() and getRoyaltyView(); RoyaltyLookup._getERC2981RoyaltyInfo(); TLAuctionHouse._quoteRoyalties()

Summary

Royalty-resolution failures are converted into empty schedules, allowing a secondary sale to finalize without paying an otherwise available royalty. The omitted amount becomes seller proceeds and leaves no claimable royalty debt, while the per-sale loss remains bounded by the Auction House's 10% royalty cap.

Description

Expected behavior

Secondary-sale settlement should distinguish a valid empty royalty schedule from a failed resolution. The package states that the Auction House respects royalties configured on secondary-sale tokens (`README.md:23-34`); when a configured lookup, engine, or direct ERC-2981 callback cannot return a valid schedule, settlement should apply an explicit failure policy instead of treating the failure as an authoritative zero. A healthy engine's intentionally empty response may remain authoritative.

Observed behavior

RoyaltyLookup.getRoyalty() and getRoyaltyView() return empty arrays when a configured engine has no code, reverts, or returns ABI-decodable recipient and amount arrays of unequal length. Because a nonzero engine takes precedence, these failures do not fall back to the token's ERC-2981 quote. In direct mode, _getERC2981RoyaltyInfo() catches any `royaltyInfo()` failure and returns the same empty arrays.

The Auction House repeats this behavior at its own boundary: getRoyalty() and _quoteRoyalties() together with _capRoyalties() convert a codeless or reverting top-level lookup, or ABI-decodable unequal arrays, into an empty schedule. _completeSale() treats that result as final, transfers the NFT, skips royalty payment, and sends the residual proceeds to the seller-selected payout receiver. Because no payee was resolved, the pay-or-credit path records no royalty debt.

The resolver- and engine-failure proofs use 10 ETH secondary auctions and healthy controls that pay a 1 ETH royalty. When each dependency is made to revert, permissionless settlement pays the artist zero, pays the seller 10 ETH under the zero-fee proof configuration, transfers the NFT, and records no artist credit; restoring the top-level resolver later does not reverse the finalized sale or create debt.

The arithmetic variant uses a 10,000-bps ERC-2981 fraction at `floor((2^256 - 1) / 10,000) + 1`. The packaged implementation's checked multiplication before division (`dependencies/@openzeppelin-contracts-5.6.1/token/common/ERC2981.sol:57–71`) panics although the exact result and the Auction House's 1,000-bps capped amount remain representable; settlement then consumes the caught failure as zero royalty.

The gas variant uses the same 1 ETH quote on two 10 ETH auctions with a deliberately expensive callback. Supplying 15,000,000 gas at the `settleAuction()` call entry pays the royalty, while supplying 10,000,000 gas at that call entry exhausts the callback's forwarded gas but leaves enough gas for settlement to complete without the royalty. These are proof-specific call-entry budgets, not universal transaction limits.

Root cause

`RoyaltyLookup` and `TLAuctionHouse._quoteRoyalties()` erase royalty-resolution failures by encoding them as the same empty arrays used for a legitimate zero-royalty schedule.

The helper loses status in both the configured-engine and direct ERC-2981 branches, while the Auction House independently loses status for its configured top-level lookup and malformed schedules. `_completeSale()` therefore cannot distinguish "no royalty" from "royalty lookup failed" and distributes the unresolved share as seller residual. Its credit mechanism only preserves a failed transfer to a known recipient, so it cannot record an obligation when resolution produced no recipient.

For the gas manifestation, nested EIP-150 retention lets the callback exhaust its forwarded allocation while the callers retain enough gas to catch the failure and continue. For the arithmetic manifestation, multiply-before-divide overflow supplies the callback failure, but the package-owned defect is the decision to suppress that failure rather than preserve its status.

Conditions and assumptions

The sale must be secondary and have a nonzero royalty that remains intended at settlement. A configured top-level lookup or engine must be codeless, revert, or return ABI-decodable arrays of unequal lengths; arbitrary invalid ABI is not established. A successful intentionally empty engine schedule is outside this finding.

For an auction, the sale must be ended and otherwise settleable. Any account can settle during the failure window, although the evidence does not show that the settler can induce an ordinary resolver or engine outage; dependency selection, pausing, and repair remain owner controls.

The direct variants require ERC-2981 resolution without an engine override. The overflow manifestation additionally requires a fraction above 1,000 bps, a price between the provider's multiplication boundary and the Auction House cap boundary, and a sufficiently funded owner-allowlisted standard ERC-20. The gas manifestation requires an unusually expensive or input-sensitive callback and a compatible chain-specific gas window.

Impact

An affected settlement finalizes NFT delivery and distributes the proceeds while the royalty beneficiary receives neither payment nor claimable credit. The omitted amount is added to the seller-selected proceeds; in the resolver proof, restoring the dependency afterward does not reverse the sale or create an obligation. Direct harm is bounded to 10% of the sale value per affected sale.

The final severity is Medium. A configured royalty of up to 10% can be permanently omitted from a finalized sale, and permissionless settlement makes a credible lookup or engine failure actionable and potentially repeatable; the gas variant also lets the caller create the failure window through its call-entry budget. The cap and settlement-time prerequisites

bound each sale, while the arithmetic and gas variants require exceptional amounts or callback costs. The evidence establishes individual sale outcomes rather than aggregate loss.

Recommendation

Return a status-bearing royalty result—such as `Resolved(schedule)`, `NoRoyalty`, or `LookupFailed`—from `RoyaltyLookup` and preserve it through `_quoteRoyalties()`. Treat codeless targets, caught reverts or callback exhaustion, and decoded length mismatches as failures, and prevent `_completeSale()` from consuming a failure status as an empty schedule.

Choose an explicit handling policy: revert or defer settlement so a later resolution can be attempted, commit the royalty schedule before finalization, or preserve a recoverable obligation. If ERC-2981 is used after confirmed engine failure, retain engine precedence for every successful engine response, including an intentionally empty schedule. Failing closed can delay NFT and fund finality when a dependency remains broken, while precommitted schedules introduce storage and staleness tradeoffs.

Code validation addresses only initially codeless dependencies, a minimum gas threshold cannot guarantee arbitrary callback completion, and events alone would expose rather than prevent the incorrect allocation. Full-precision arithmetic can remove the demonstrated overflow trigger in controlled providers but cannot harden the marketplace against arbitrary external failures.

Validation objective

Regression coverage should distinguish a legitimate empty schedule from a codeless lookup or engine, an explicit revert, decoded unequal arrays, an ERC-2981 arithmetic failure, and callback gas exhaustion. For each failure, settlement should either revert without transferring the NFT or proceeds, or preserve an explicit recoverable royalty obligation; healthy nonzero quotes should still be paid or credited, and intentional valid empty responses should still settle. Vary gas around a working callback's threshold and exercise direct and engine resolution across every auction, fixed-price, native-currency, and ERC-20 path intended to share the corrected logic.

HEX-002: Round-blind seller-fee waivers can exempt successor mint drops

- ID: HEX-002
- Status: New
- Class: Finding
- Severity: Medium
- Location: TLStacks721.waiveSellerFee(); TLStacks1155.waiveSellerFee()

Summary

An authorized collection administrator can advance an ERC-721 or ERC-1155 drop before a genuine pending protocol-owner fee waiver executes, causing the waiver to bind to the successor round instead. Successor sales then skip the configured percentage seller fee and reduce protocol revenue for that round.

Description

Expected behavior

A protocol-owner seller-fee waiver prepared for drop round `r` should remain bound to that round or revert if a collection administrator advances the lifecycle before the call executes. The documented lifecycle ties a waiver to one drop round and resets it when the drop is closed or reconfigured (README.md (`README.md:14–21`)). Intentional preconfiguration should remain available when the owner explicitly identifies the current unconfigured round.

Observed behavior

Seller-fee waiver authority belongs to the protocol contract owner, while the NFT owner or an `ADMIN_ROLE` holder independently controls the drop lifecycle. In both implementations, `closeDrop` deletes the current drop and increments its round (ERC-721, ERC-1155); `configureDrop` can then install successor terms in the unconfigured slot without another increment (ERC-721, ERC-1155). The ERC-721 proof executes the pending waiver before configuring the successor, while the ERC-1155 proof configures the successor first. In each ordering, the unchanged waiver resolves the current counter after the close and marks the successor round.

The ERC-721 paired control charges a 0.5 ETH percentage fee on a 10 ETH successor sale under serial `waive → close → configure` ordering, whereas the reordered path charges zero. The ERC-1155 paired control charges 0.05 ETH on a 1 ETH successor sale, whereas the reordered path sends the full 1 ETH to the payout receiver. These purchases identify the correct successor round, but settlement reads the retargeted waiver flag and omits the percentage fee (TLStacks721._settleUp(), TLStacks1155._settleUp()).

Root cause

`TLStacks721.waiveSellerFee()` and `TLStacks1155.waiveSellerFee()` resolve a mutable current round at execution instead of binding the protocol owner's authorization to an expected round.

The ERC-721 calldata identifies only the stable NFT address, while the ERC-1155 calldata identifies only the stable NFT address and token ID. Neither function includes an `expectedRound` or equivalent generation identifier. Because a separate authorized principal controls the round transition, transaction ordering can change the round that receives a genuine pending authorization without changing its calldata.

Conditions and assumptions

The ordering requires a genuine pending protocol-owner waiver that was contextually intended for the predecessor round and an NFT owner or `ADMIN_ROLE` holder able to close that round before the waiver executes. The call must be observable or otherwise orderable; private submission, cancellation, or atomic owner-administrator coordination prevents the demonstrated race.

Economic loss additionally requires a nonzero percentage seller-fee rate and at least one valid, completed sale in the successor round. The finding assumes that a waiver prepared for an observed round is not open-ended authorization for whichever round is current at execution; if protocol policy intentionally grants that authority, the observed behavior follows that policy.

Impact

The ERC-721 proof demonstrates 0.5 ETH of protocol revenue omitted from a 10 ETH successor sale at a 500-basis-point rate, with the seller receiving 10 ETH instead of 9.5 ETH. The ERC-1155 proof demonstrates a 0.05 ETH omission from a 1 ETH successor sale, with the payout receiver receiving 1 ETH instead of 0.95 ETH.

Because the waiver flag remains active for the round and settlement consults it on each purchase, later applicable sales in the affected successor round are expected to omit the percentage fee as well. The configured rate is capped at 5%, while aggregate loss depends on sale volume and value.

Medium severity reflects potentially repeated protocol-revenue loss under plausible but role- and ordering-constrained conditions. For ERC-1155, the collection administrator can close the affected round to prevent future waived sales, but the contract cannot recover fees already omitted. The supported loss is limited to percentage seller fees from the affected successor round.

Recommendation

Require both `waiveSellerFee` functions to accept an `expectedRound` and revert before mutation when it differs from the current round. This binds the protocol owner's authorization to one lifecycle generation while preserving deliberate preconfiguration when the owner identifies the current unconfigured round.

This change requires an ABI migration or replacement entry point and requires callers to obtain the intended round before submission; queued calls should intentionally become invalid after a round transition. Retaining an unrestricted legacy path would preserve the weakness. An owner-controlled revocation path may help address an accidentally applied waiver, but it does not replace round binding.

Validation objective

For both implementations, verify that a waiver naming round `r` fails after an administrator advances the drop to round `r+1`, leaves the successor unwaived, and causes a completed successor purchase to charge the configured percentage fee. Also verify that deliberate preconfiguration naming the current unconfigured round still succeeds, that a waiver applied to an active predecessor resets after closing and configuring its successor, and that separate fixed-fee accounting remains unchanged.

HEX-003: Stale protocol-fee removal can waive a successor listing

- ID: HEX-003
- Status: New
- Class: Finding
- Severity: Medium
- Location: TLAuctionHouse.removeProtocolFee()

Summary

A pending owner-authorized fee waiver for one listing can attach to a fresh successor listing at the same NFT key. By ordering a delist and relist before that transaction executes, the predecessor seller can cause the successor sale to skip up to the configured 5% protocol fee.

Description

Expected behavior

An owner fee-removal transaction prepared for one listing should either waive that listing or revert if a delist and relist has replaced it. A fresh successor listing should remain unwaived, while an already applied waiver may continue through ordinary listing updates if that is the intended policy.

Observed behavior

`TLAuctionHouse` stores the current listing in one mutable slot keyed by NFT address and token ID. Fresh `_list` and `_buildListing` operations reset `zeroProtocolFee` to `false` and assign a new listing ID, but `removeProtocolFee` checks only that a listing currently occupies the key before setting that occupant's waiver flag.

In the reproduced sequence, the owner call is encoded while listing 1 is current. The predecessor seller uses `delist`, and a distinct successor seller creates listing 2 before the unchanged call executes. Listing 2 then completes a 10 ETH sale with no protocol fee, whereas a matched unwaived sale pays 0.5 ETH through the `_completeSale` fee branch.

Root cause

`TLAuctionHouse.removeProtocolFee()` binds the owner's authorization only to the mutable `(nftAddress, tokenId)` slot and does not require the listing generation that the owner intended to waive.

Because the same key is reused after a fresh delist and relist, authentic owner calldata can resolve to a different listing at execution. The calldata cannot express or validate the intended generation, and the emitted event likewise omits the affected listing ID. Buyer operations avoid this boundary crossing by validating an expected listing ID.

Conditions and assumptions

A genuine owner-authorized fee-removal transaction must be pending, observable or otherwise orderable, and intended for the predecessor listing. The predecessor seller must be able to delist the unbid listing and control the returned NFT; a distinct successor seller requires cooperation or common control. The replacement must finish before the owner call, and

the successor must be live when it lands because a call during the empty-slot interval reverts.

Revenue loss requires a nonzero configured fee and a completed successor sale. If product policy intentionally authorizes whichever listing occupies the NFT key at execution, the behavior is intended; however, the listing-specific description, fresh-list reset, and new-listing invariant support lifecycle-specific authorization.

The condition is limited to fresh delist/relist replacement. `updateListing` intentionally assigns a new ID while preserving an already applied waiver.

Impact

The reproduced 10 ETH successor sale paid no protocol fee at a 500-bps setting, leaving the protocol without 0.5 ETH and the successor seller with the full price. More generally, the loss is bounded to the configured protocol-fee percentage—at most 5%—of one completed successor-listing sale per redirected waiver.

Medium severity reflects meaningful but bounded protocol-revenue loss through credible transaction ordering. The need for a genuine owner transaction and an ordering race constrains likelihood. The evidence does not show buyer asset loss, forged owner authority, or a repeated protocol-wide drain, and actual harm depends on the successor sale's value.

Recommendation

Bind fee removal to the listing lifecycle the owner observed. Add an owner-supplied expected listing ID to `removeProtocolFee`, compare it with the current listing before mutation, revert on mismatch, and include the validated ID in the removal event.

A direct ID check also invalidates a pending waiver after an ordinary `updateListing`, because updates assign new IDs. If pending waivers must survive those updates, use a separate lineage generation that changes only on fresh listing creation while preserving an already applied waiver through permitted updates. Consider an owner-only revocation path for accidental exemptions.

Validation objective

Verify that a fee-removal call prepared for listing A reverts after A is delisted and listing B is created, that B remains unwaived and charges the configured fee when sold, and that a call carrying the current authorized generation succeeds. If intended, confirm that an applied waiver persists through ordinary updates without allowing pending authorization to cross a fresh-list boundary. Exercise the property across relevant buy-now and auction settlement paths using both native and allowed ERC-20 currencies.

HEX-004: Unversioned listing updates can apply obsolete terms to replacement listings

- ID: HEX-004
- Status: New
- Class: Finding
- Severity: Medium
- Location: TLAuctionHouse.updateListing()

Summary

A delayed seller-authorized listing update can overwrite a compatible replacement listing with obsolete terms, allowing a permissionless buyer to purchase the NFT at the stale price. This can complete a sale below the seller's intended replacement price when authenticated seller operations execute out of order.

Description

Expected behavior

An update authorized while listing A is current should either apply to A or revert once A has been delisted and replaced by listing B. The contract should compare an expected listing ID before writing replacement terms, as `bid` and `buyNow` do before acting on the current record.

Observed behavior

`updateListing` loads whichever listing currently occupies the `(nftAddress, tokenId)` key. It verifies the current seller, bid state, listing type, and currency, but receives no expected input listing ID.

The reproduction creates listing A at 10 ETH and obtains genuine seller authorization for an update to 1 ETH. An independent nonce lane then delists A and creates same-seller listing B at 100 ETH. A permissionless relay submits the unchanged authorization for A; the auction house accepts it against B, creates listing C at 1 ETH, and permits the relay to obtain C's current ID and buy the NFT atomically. No seller signature or protocol state is forged.

Root cause

`TLAuctionHouse.updateListing()` authenticates the seller against the current NFT-keyed record without binding the authorization to the listing generation that the seller intended to change.

`ListingConfig` has no expected listing ID, while `TLAuctionHouse` storage maintains one current record per NFT key. `_buildListing` assigns a fresh output ID only after the current successor has been selected, so that increment protects later buyer operations but does not bind the seller's input authorization. By contrast, `topUpBid` checks an expected listing ID for out-of-order smart-account transactions.

Conditions and assumptions

The path requires a seller account or relay workflow that can execute genuine authenticated operations out of order, such as through independent nonce lanes. The scalar nonce ordering of a conventional externally owned account normally prevents this sequence.

The predecessor update must remain executable and available to the triggering relayer or buyer after the successor is created. Private submission, expiry, or effective cancellation can prevent the trigger.

The successor must have the same seller, listing type, and currency, remain unbid, and be valid while the auction house is unpaused and the seller is unsanctioned. Existing validation rejects an incompatible or invalid successor. This finding treats the seller's update as generation-specific intent; if the protocol deliberately defines it as authorization for whichever compatible listing is current at execution, the observed selection matches that policy.

Impact

The stale update can complete an NFT sale under obsolete terms, transferring the asset for less than the seller specified in the replacement listing. In the reproduction, a 100 ETH successor is replaced by a new 1 ETH listing and sold for 1 ETH. This can cause material seller loss for a valuable NFT, although the 100 ETH asking price does not establish the NFT's realizable value.

The Medium rating reflects an irreversible, financially meaningful sale under obsolete terms. The effect is bounded to one compatible successor per stale authorization, and reachability depends on the constrained unordered workflow and authorization availability described above.

Recommendation

Add an `expectedListingId` parameter to `updateListing()` and compare it with the loaded listing ID before validating or writing replacement terms. Revert with `ListingChanged` on a mismatch, and enforce the same generation check in every callable update route.

This changes the public ABI and requires callers and relayers to include the current listing ID. Retaining an unrestricted legacy entry point would preserve the stale-generation path, so any compatibility transition should prevent that entry point from mutating a later generation.

Validation objective

Exercise independently sequenced seller authorizations in a regression test. An update carrying A's ID should revert after A is delisted and B is listed, leaving B's ID and terms unchanged; an update carrying B's ID should succeed and create the next ID. Retain coverage showing that mismatched seller, type, currency, bid state, pause state, sanctions, and invalid configuration still revert.

HEX-005: Stale payout authorization can redirect successor-listing proceeds

- ID: HEX-005
- Status: New
- Class: Finding
- Severity: Medium
- Location: `TLAuctionHouse.updatePayoutReceiver()`

Summary

A seller-authorized payout update can outlive its intended listing generation and overwrite a same-seller replacement listing's receiver. If it executes after a bid, settlement can send the successor listing's seller proceeds to the stale receiver instead of the intended recipient.

Description

Expected behavior

A payout-receiver authorization should apply only to the listing generation the seller reviewed when authorizing it. A matching update should remain usable before or after a bid while preserving the active listing ID, bidder, and bid amount.

Observed behavior

The listing state uses a stable NFT-address and token-ID storage slot, while `_buildListing` assigns each replacement listing a fresh ID. A same-seller replacement can therefore occupy the original slot with a different generation ID.

`bid` enforces that generation boundary by checking an expected listing ID. In the reproduced sequence, a bid carrying generation A's ID was rejected, while a bid carrying generation B's ID succeeded.

The unchanged generation-A payout authorization then executed, preserving generation B's ID, highest bidder, and bid amount while replacing its receiver. During settlement, `_completeSale` used the overwritten receiver: the bidder received the NFT, the obsolete receiver received the fixture's full 1 ETH seller remainder, and the intended generation-B receiver received nothing.

Root cause

`TLAuctionHouse.updatePayoutReceiver()` omits a listing-generation check before mutating the current slot's payout receiver.

`updatePayoutReceiver` resolves the current listing from `(nftAddress, tokenId)` and authenticates its current seller, but accepts no `expectedListingId`. A valid delayed call therefore cannot distinguish the generation under which it was authorized from a same-seller replacement. Preserving the listing ID after a valid update is appropriate; the missing control is an input-generation comparison before mutation.

Conditions and assumptions

The same seller address must control both listing generations. A smart account, relayer queue, or equivalent mechanism must allow valid operations to execute outside the seller's intended order; ordinary single-EOA nonce ordering normally prevents this schedule. The contract accounts for independently ordered smart-account operations in `topUpBid`, but no deployment-specific prevalence is established.

The stale call must remain a genuine seller authorization; this condition does not enable signature forgery or arbitrary receiver selection. The contract must remain unpaused, and the seller and old-intent receiver must remain unsanctioned and otherwise valid. The seller must neither cancel the authorization nor restore generation B's receiver before settlement.

The old-intent receiver need not have been configured on generation A, but it must have been encoded in an authorization created while generation A existed.

Impact

The concrete harm is misdirection of one successor listing's seller remainder. In the focused reproduction, the obsolete receiver receives the full 1 ETH bid and the intended generation-B receiver receives zero. When protocol fees or royalties are configured, they are paid first, so the amount at risk is the remaining seller proceeds.

The condition is rated Medium because the complete seller remainder can be redirected after a bid, balanced by the required same-seller replacement and authenticated out-of-order execution. It does not let an outsider invent a receiver or affect another seller's listing, and the seller can correct the receiver before settlement. Loss becomes permanent only if the old-intent receiver is unavailable or unwilling to return the funds.

Recommendation

Require `updatePayoutReceiver` to accept an `expectedListingId` and compare it with the current listing ID before writing `payoutReceiver`. Revert with the existing `ListingChanged` error on a mismatch, preserve the current listing ID after a successful update, and retire or deprecate any unversioned entrypoint. This change requires an ABI and integration migration so wallets and relayers obtain and submit the intended listing ID.

Validation objective

Regression tests should verify that:

- An authorization carrying generation A's ID reverts after relisting as generation B and does not change B's receiver.
- An authorization carrying B's ID succeeds both before and after a bid while preserving B's ID, bidder, and bid amount.
- Settlement pays the receiver selected by the valid B-scoped authorization.
- Reserve-auction and buy-now flows, using native and allowlisted ERC-20 currencies, enforce the same generation-binding property.

HEX-006: Stale price updates can underprice successor mint rounds

- ID: HEX-006
- Status: New
- Class: Finding
- Severity: Medium
- Location: `TLStacks721.updateDropPrices()`; `TLStacks1155.updateDropPrices()`

Summary

An authorized price update prepared for an earlier mint round can execute after that round is closed and replaced, overwriting the successor's intended price in both `TLStacks721` and `TLStacks1155`. Buyers can then purchase successor-round inventory at the obsolete lower price, reducing proceeds for the replacement payout receiver.

Description

Expected behavior

An administrative price update should remain bound to the mint round for which it was prepared. Once a collection or token key advances from round `r` to round `r+1`, predecessor-round calldata should revert without modifying the successor's terms, while a current-round update should continue to succeed. This safety property follows from the explicit round lifecycle and buyer-side round binding; the supplied documentation does not expressly promise optimistic-concurrency protection for administrative instructions.

Observed behavior

In each variant, authorized price-update calldata is prepared while round 0 is current. Before it executes, independently ordered administration closes that round and configures round 1 at 100 ETH. The unchanged call then succeeds, leaves the current round at 1, and lowers its public price to 1 ETH.

A fresh permissionless buyer supplies the correct round-1 identifier and exact 1 ETH payment, receives one successor-round token, and transfers 1 ETH to the replacement payout receiver. The buyer-facing round checks (ERC-721; ERC-1155) correctly recognize round 1, while maximum-price protection acts only as a ceiling and therefore accepts the lower current price.

Root cause

`TLStacks721.updateDropPrices()` and `TLStacks1155.updateDropPrices()` overwrite the active drop slot without accepting or validating the round for which the update was prepared.

Each contract stores the active drop separately from its round counter (ERC-721 state; ERC-1155 state). The closure paths advance that counter (ERC-721; ERC-1155), but subsequent configuration reuses the stable collection key or collection-and-token key. Because previously prepared calldata identifies only that stable key, current role authorization proves that the caller may update the active drop but cannot establish that the instruction was intended for its current generation.

Conditions and assumptions

The price update must be prepared while an older round is current, and the caller must remain the NFT collection owner or an `ADMIN_ROLE` holder. Another independently ordered authorized actor must close and reconfigure the round before that update executes; a single conventional EOA's nonce ordering ordinarily prevents its own later rollover calls from overtaking its earlier update.

At execution, Stacks must remain unpaused, the caller unsanctioned, the selected currency allowed, and the replacement drop active. The demonstrated scope is a regular public native-currency ERC-721 or ERC-1155 drop using close-and-reconfigure rollover, followed by a fresh exact-value purchase after repricing and before correction.

Impact

The demonstrated result is one successor-round token minted and settled for 1 ETH after that round had been configured at 100 ETH. The replacement payout receiver receives only the stale 1 ETH price for the completed purchase, which is irreversible within the demonstrated flow.

Additional inventory could sell at the lower price until correction, but aggregate revenue loss depends on remaining supply, mint allowances, detection time, buyer demand, and whether buyers would have paid the configured price. The effect is bounded to the affected replacement drop and does not forge administrative authority, redirect the payout receiver, or remove existing balances.

The severity is Medium because the condition can cause material, irreversible sale-term and settlement harm, while the privileged stale trigger, independent ordering requirement, and drop-bounded scope constrain likelihood and blast radius.

Recommendation

Add an `expectedRound` parameter to both `updateDropPrices()` implementations and reject a mismatch before loading or mutating the active drop. Include the affected round in update events so administrative systems can reconcile each instruction with the intended generation.

This changes the external ABI and requires administrative clients and queued operations to fetch and submit the current round. If compatibility requires a new selector, remove or deprecate the unguarded path so callers cannot bypass the generation check; a mismatch should require an explicit refresh and renewed instruction rather than an automatic retry against the successor.

Validation objective

Regression tests should verify that current-round updates succeed for both token variants; identical predecessor-round calldata reverts after close and reconfiguration without changing the successor's round or configured prices; independently ordered retained role holders cannot apply stale updates; and existing authorization, pause, sanctions, currency, and buyer-side validations continue to operate as intended.

HEX-007: Unversioned drop lifecycle calls can replace successor sales with obsolete terms

- ID: HEX-007
- Status: New
- Class: Finding
- Severity: Medium
- Location: `TLStacks721._drops` and `_rounds`; `TLStacks1155._drops` and `_rounds`

Summary

Delayed but still-authorized drop lifecycle calls can cross a replacement or completed sale because ERC-721 and ERC-1155 administration is not bound to the lifecycle state for which it was prepared. A stale close can erase a healthy successor, while stale configuration can reopen obsolete access, price, payout, and metadata terms; the ERC-721 sequence allowed a buyer excluded from the healthy sale to complete a mint and payment under the superseded terms.

Description

Expected behavior

A lifecycle operation prepared for a particular drop state should either affect only that state or fail after another authorized operation changes it. A fresh, state-matched close or configuration should remain available, while buyer-side round checks should continue to protect purchases.

Observed behavior

Both `TLStacks721.closeDrop` and `TLStacks1155.closeDrop` identify a drop only by its stable collection key, plus the token ID for ERC-1155. In separate close-only variants, a call prepared during round 0 remained valid after another authorized administrator closed that round and configured round 1. The delayed call deleted the round-1 configuration and advanced the counter to round 2; in the ERC-1155 variant, a purchase naming round 1 then reverted.

Configuration behaves similarly after a natural sale end. In both configuration-only variants, exact 0.1 ETH configuration calldata prepared against an empty slot remained pending while an independently configured one-token round sold out at 1 ETH, then succeeded as round 1. The ERC-721 variant additionally confirmed that the same calldata was rejected while the intervening round was live. The `TLStacks721.configureDrop` and `TLStacks1155.configureDrop` implementations installed the earlier receiver, price, and ten-token supply; each variant completed one purchase for 0.1 ETH to that receiver.

The combined ERC-721 variant shows the two manifestations interacting. A healthy replacement excluded the obsolete-sale buyer and set a 1 ETH price, but a stale close erased it. A diagnostic call with the obsolete configuration had been rejected while the healthy round was live; after the close, the queued configuration restored its 0.1 ETH price, allowlist, payout receiver, and `Drop` metadata field, allowing the excluded buyer to mint with payment routed under those terms.

Root cause

`TLStacks721` and `TLStacks1155` authorize drop closure and configuration against mutable per-drop state without requiring the caller to commit to the lifecycle version it observed.

Each `closeDrop` implementation rechecks current authority but accepts no expected version before deleting whichever drop occupies the stable key and incrementing its round. Each `configureDrop` implementation decides only from the execution-time phase and proposed terms, then overwrites the slot when no live drop blocks it; it carries no commitment to the prior lifecycle state.

The sale round is not a complete freshness token for configuration. Initial configuration does not advance it, and sellout leaves the first sale ended at round 0, so the initially empty state and the ended first round expose the same round value. The ERC-721 and ERC-1155 buyer checks validate the round that is current at purchase time; once stale administrative terms become current, those checks cannot identify their obsolete intent.

Conditions and assumptions

The caller must remain the NFT collection owner or an `ADMIN_ROLE` holder. A separate authorized principal, queue, or smart-account lane must change the lifecycle before the delayed call executes; one EOA's nonce ordering prevents its own later transaction from overtaking the earlier call. This is not a privilege escalation.

Stale configuration additionally requires an ended intervening drop or an unconfigured slot created by a stale close. Stacks must be unpaused; the stale timing window and sanctions, currency, receiver, supply, type, and approved-mint validations must pass. Sale harm requires conflicting terms and buyer activity before correction. The claim assumes delayed calls target their observed state; the API neither encodes nor expressly promises that binding.

The package permits multiple role holders, but no evidence establishes independently ordered administration or queued execution in a current deployment. Current exposure is unestablished; the condition applies wherever deployment topology permits this ordering.

Impact

The strongest variant establishes one completed ERC-721 mint and payment under superseded terms. A buyer excluded by the healthy allowlist acquired an NFT for 0.1 ETH while the healthy round specified 1 ETH; the token used obsolete metadata and 0.1 ETH went to the superseded receiver. Closing the remaining sale does not roll back that completed allocation or payment.

The standalone ERC-721 and ERC-1155 configuration variants likewise complete one purchase after obsolete price and payout terms reopen. Because configuration replaces the complete sale terms, stale eligibility, supply, or timing may broaden the effect, but harm beyond the demonstrated purchases depends on the term differences and buyer activity. The close-only variants establish deletion of an active replacement; because an administrator can configure a later round, their direct result is recoverable sale interruption and reconfiguration work, while opportunity loss and user gas costs depend on activity during the interruption. They do not show theft, permanent denial, or effects on existing mints, escrowed assets, or settled funds.

The grouped finding is Medium severity because its strongest verified manifestation causes direct, irreversible sale-integrity harm, but the scope is one collection and drop and the ordering requires a delayed, still-authorized call plus an independently ordered privileged transition. Full-round consumption, seller revenue loss, protocol-wide loss, and current deployment prevalence are not established.

Recommendation

Introduce a monotonically increasing lifecycle version for each ERC-721 collection and each ERC-1155 collection/token-ID pair. Require both `closeDrop` and `configureDrop` to accept the version observed by the caller, compare it immediately before mutation, and advance the version on every accepted close and configuration, including the first configuration. A

mismatch should revert without changing the drop, sale round, or lifecycle version.

Keep this version separate from the buyer-visible sale round unless those semantics are deliberately redesigned, because the current round cannot distinguish the initial empty slot from the ended first round. An atomic, version-bound replacement operation is a viable alternative for close-and-reconfigure workflows. Any legacy unversioned path should receive an equivalent freshness guard; callers and relayers will need to read the current version and handle safe concurrency reverts.

Validation objective

Across both token standards, prepare stale close and configuration operations, perform an independent lifecycle transition, and verify that the stale operations revert without changing the successor's terms, counters, or availability. Specifically verify that a configuration prepared for an empty slot fails after an intervening first round ends, and that an old close-and-configure pair cannot replace a healthy sale. Fresh version-matched operations should succeed, each accepted mutation should advance the version exactly once, repeated old operations should fail, and existing buyer round checks and access controls should remain effective.

HEX-008: Round-blind payout updates can redirect successor-drop proceeds

- ID: HEX-008
- Status: New
- Class: Finding
- Severity: Medium
- Location: `TLStacks721.updateDropPayoutReceiver()`; `TLStacks1155.updateDropPayoutReceiver()`

Summary

Delayed payout-update calldata prepared for an ERC-721 or ERC-1155 drop can execute after that drop is closed and replaced, overwrite the successor round's receiver, and send its seller proceeds to an obsolete address. This requires independently sequenced authorized operations and a successor purchase before the receiver is corrected.

Description

Expected behavior

Administrative payout calldata prepared for round `N` should either update that round before rollover or fail without changing round `N+1`. A replacement drop should retain the payout receiver chosen during its configuration, preserving the generation boundary that buyer operations enforce through `expectedRound`.

Observed behavior

An administrator can prepare a complete payout update while round 0 is active. An independently sequenced owner or administrator can then close round 0 and configure round 1 with a different receiver through the ERC-721 `closeDrop()` path or the ERC-1155 `closeDrop()` and `configureDrop()` paths. Because the delayed update contains no round, it remains executable after replacement if the caller stays authorized and overwrites round 1's receiver without changing the round counter.

A buyer can still name the current round: the ERC-721 `expectedRound` check and ERC-1155 `expectedRound` validation confirm the purchase's round, not the provenance of an earlier administrative mutation. In each proof, one 1 ether round-1 native-currency purchase completed, the buyer received the token, the stale receiver received the payment, and the configured successor receiver received zero. The serial-order controls applied the update before closure and confirmed that replacement discarded it, isolating the adverse result to the reordered update.

Root cause

The `TLStacks721.updateDropPayoutReceiver()` and `TLStacks1155.updateDropPayoutReceiver()` functions mutate the currently keyed drop without validating the round for which the call was prepared.

Both contracts store active drop data separately from their generation counters: `TLStacks721` uses `_drops` and `_rounds`, and `TLStacks1155` follows the same layout. Closing a drop deletes the current slot and advances its round, while successor configuration repopulates the same collection key for ERC-721 or NFT-address/token-ID key for ERC-1155.

The setter resolves only that key and the new receiver, so unchanged delayed calldata acquires a new meaning when the slot holds a successor round.

Purchase validation cannot detect this stale administrative intent because it only verifies the current round. Settlement then consumes the overwritten receiver through `TLStacks721._settleUp()` or `TLStacks1155._settleUp()`.

Conditions and assumptions

The delayed caller must remain the NFT contract or collection owner, or an `ADMIN_ROLE` holder. The ordering requires at least two independently sequenced authorized principals or an equivalent queue, retry, smart-account module, or relayer; one conventional EOA's nonce sequence prevents its own later rollover transaction from overtaking an earlier update. This is stale authorized intent, not an outsider privilege escalation.

The successor drop must be active and the protocol unpaused, and the caller and receiver must satisfy the applicable sanctions and validity checks. The update must execute after successor configuration, and a purchase must occur before a corrective update, pause, or closure. Loss becomes unrecoverable only if the obsolete receiver does not return the funds.

The claim is limited to the payout-receiver setters for ERC-721 and ERC-1155 drops; it does not establish the same behavior for other administrative entry points.

Impact

The demonstrated harm is incorrect settlement: in each variant, a 1 ether successor-round native-currency purchase completes and delivers the token, while the stale receiver receives the seller proceeds and the configured successor receiver receives zero. Because the proofs configure zero fees, this equals the entire payment; with nonzero fees, only net seller proceeds are exposed, and protocol fees remain on their configured path.

Subsequent purchases would use the same overwritten receiver until correction or the end of the round, so the diversion could repeat, but each proof executes only one purchase. Permanent loss is not demonstrated and depends on the stale receiver not returning the funds. The scope is bounded to the affected ERC-721 collection round or ERC-1155 NFT-and-token drop.

Severity is Medium because a completed sale can materially misdirect seller proceeds. Continuing privilege, independent transaction ordering, per-drop scope, available administrative correction, and conditional recoverability constrain the result below High.

Recommendation

Add a caller-supplied `expectedRound` to both `updateDropPayoutReceiver()` functions and compare it with the stored current round before changing the drop. A mismatch should revert without modifying the receiver. Remove or explicitly deprecate the permissive signatures so administrative clients cannot silently retain execution-time current-round semantics.

This interface change requires an ABI and client migration. Preserve the existing role, pause, sanctions, phase, and receiver checks; if compatibility precludes a direct round argument, bind each update to a generation-specific authorization or configuration hash instead. Consider including the round in update events so operators can identify the affected generation.

Validation objective

For each implementation, verify that a same-round update succeeds; a round-0 update executed after closure and round-1 configuration reverts without changing the successor receiver; a fresh round-1 update succeeds; and a round-1 purchase pays only the receiver validly selected for that round. Retain the serial-order control showing that an update applied before closure is discarded. For ERC-1155, also verify that native-currency and supported ERC-20 settlement preserve their intended fee splits after the change.

HEX-009: Unversioned NFT recovery claims can withdraw later escrow

- ID: HEX-009
- Status: New
- Class: Finding
- Severity: Medium
- Location: `TLAuctionHouse.claimableNft`

Summary

A former seller can withdraw a successor's escrowed NFT, and a stale smart-account authorization can redirect an NFT from a later recovery cycle, leaving a listing unbacked or consuming the newer recovery credit. These failures occur because recovery authority survives the auction or credit for which it was created.

Description

Expected behavior

Cancellation should create an executable NFT recovery authorization only for escrow held for that auction. The authorization should not overlap a later live listing or become valid again against a later recovery credit, while bidder unwind and legitimate seller recovery remain available.

Observed behavior

After a bid-backed auction remains unsettled through its duration and seven-day grace period, `cancelUnsettledAuction` deletes the listing, credits the bidder's funds, and records the seller as the NFT claimant under the collection and token ID. Three independent manifestations show that this recovery state can reach a later asset.

In the custody-blind manifestation, the packaged TRACE recovery function (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/trace/TRACE.sol:221–225`) moves the token out of a live auction before cancellation. Cancellation still creates a claim for the former seller. If a successor later relists the same token and receives a bid, the former seller can claim the successor's NFT; the new listing and bid remain recorded, and settlement reverts.

In the custody-backed manifestation, cancellation occurs while the auction house still holds the NFT, but a later TRACE recovery or burn/reissue transition makes the unconsumed claim stale. After a successor creates a bid-backed listing for the same key, the prior claimant can withdraw its NFT and leave the listing unbacked. The TRACE administrator and former claimant can be distinct actors.

Separately, a smart-account seller can authorize a claim against recovery A, consume A through another current claim, and leave the earlier operation pending. That operation correctly reverts while no credit exists, but it becomes valid again when a later canceled auction creates recovery B for the same seller and token. Execution then transfers B's NFT to the earlier recipient and clears B.

Root cause

`TLAuctionHouse.claimableNft` stores one claimant per collection and token ID without binding the authorization to an escrow, listing, or recovery generation.

The cancellation path writes a bare claimant address without retaining the canceled listing ID or a recovery nonce, and it can do so without confirming current custody. The listing path ignores an outstanding claim, while the claim path checks the current claimant but not an expected generation or an active successor listing. The same key can therefore become both listed and claimable, contrary to the project's custody invariant, or can make an old authorization valid when the same claimant value recurs.

Conditions and assumptions

All manifestations require a bid-backed auction to remain unsettled through its duration and seven-day grace period before cancellation, plus reuse of the same collection and token ID. Settlement before cancellation prevents the recovery state from being created.

The successor-escrow paths also require an external custody or token-generation transition and a later bid-backed relisting under the same key. The packaged TRACE collection supplies such a transition through owner/admin recovery; ordinary immutable custody blocks these paths. The former claimant and collection administrator may be distinct.

The later-credit path instead requires the same smart-account seller in two recovery cycles, normal consumption of the first credit, and an earlier unrevoked operation that remains independently executable out of order. Ordered EOA transactions do not satisfy this condition, and the earlier destination must remain valid and unsanctioned.

Impact

The successor-escrow manifestations cause the later seller to lose custody of the escrowed NFT without receiving sale proceeds. The listing and highest bid remain recorded but unbacked, so settlement fails.

The bidder's principal is not transferred to the stale claimant. It remains in the auction house until the successor auction can be canceled after its grace period and credited for withdrawal; the custody-backed variant verifies that recovery, while the custody-blind variant does not execute the second cancellation. The supported bidder harm is therefore delay, failed settlement, and additional transaction cost rather than principal theft.

The later-credit manifestation transfers one recovered NFT to a recipient authorized for an earlier credit and deletes the current credit. That auction is already canceled and its bidder is credited, so this path does not drain an active listing or demonstrate bidder loss. Permanent seller loss depends on the earlier destination no longer being intended and not returning the NFT.

The final severity is Medium because the successor-escrow paths demonstrate unauthorized loss of custody over a later seller's NFT, but exposure is bounded to a prepared token identity and requires multiple lifecycle and timing prerequisites. NFT value and aggregate exposure are not established, and the smart-account manifestation has the narrower execution-model requirement.

Recommendation

Bind every NFT recovery authorization to the canceled listing ID or a monotonically increasing per-token recovery nonce. Store that identity with the claimant, require the caller to provide the expected identity, and match it before deleting the credit or transferring the NFT. The claim path should also reject an NFT that currently backs a live listing.

Separate bidder unwind from seller claim activation. Cancellation should still remove the failed listing and credit the bidder when NFT interactions fail, but it should create an executable seller claim only after establishing custody of the corresponding escrow. Isolate the custody check from the unwind, or use a later activation step, so a reverting, ineffective, or gas-consuming NFT cannot block bidder recovery.

The listing path should reject an outstanding claim or perform an explicit, authorized, generation-aware reconciliation. Blindly clearing claims can erase legitimate recovery rights, while permanently rejecting the token key can make externally recovered identities unlistable. Merely blocking listing while a claim exists does not prevent an old smart-account operation from reaching a later credit after the first claim has been cleared.

Versioning changes storage and the claim interface, and potentially its events, so integrations and any transition for existing credits require consideration. Preserve the atomic behavior under which a failed NFT transfer leaves the recovery credit intact.

Validation objective

Regression tests should verify that pre-cancellation custody loss still permits bidder unwind without creating an executable claim; ordinary cancellation still lets the seller recover escrow; and failing NFT interactions cannot block bidder crediting. Post-cancellation recovery or burn/reissue must not let an older claimant withdraw a successor listing, and rejection must leave that listing, bid, and custody unchanged. Finally, an authorization bound to recovery A must remain invalid after recovery B is created, while a B-bound claim succeeds and a failed transfer leaves B claimable.

HEX-010: Failed payouts can strand credits owned by non-calling contracts

- ID: HEX-010
- Status: New
- Class: Finding
- Severity: Medium
- Location: `TLAuctionHouse.claimFunds()`

Summary

A failed payout can leave sale proceeds or refunds fully backed in `TLAuctionHouse` but permanently inaccessible when the credited contract cannot originate `claimFunds()`. Restoring native-currency receipt or ERC-20 transfers does not recover the historical credit, even though settlement may already have delivered the NFT.

Description

Expected behavior

Deferred payout accounting should preserve both the beneficiary's entitlement and an exercisable recovery path. After a temporary transfer failure clears, the protocol should either retry payment to the credited beneficiary, let an authority established before credit creation redirect the funds, or revert atomically before NFT ownership changes.

Observed behavior

On payout failure, `TLAuctionHouse._credit()` and `_payout()` store the amount in `claimableFunds[to][currencyAddress]` and allow settlement to continue. `claimFunds()` looks up only `claimableFunds[msg.sender][currencyAddress]`; its `to` argument chooses the withdrawal destination but cannot identify a different creditor. During a completed purchase, the listing is deleted before payment distribution, and `_completeSale()` transfers the NFT before attempting payout.

In the native-currency variant, a fixed-interface seller treasury rejects a 1 ETH payout while the purchase still completes. The buyer receives the NFT, the listing is deleted, and the Auction House records a fully backed credit for the treasury; seller, buyer, and auction-house owner calls reach only their own empty balances. After the treasury resumes accepting ETH, it can receive a fresh payment and its controller can withdraw that payment, but the controller cannot reach the historical credit; a forwarding-capable control treasury can redirect its own credit.

In the ERC-20 variant, `_escrowFunds()` pulls payment from the buyer, so a token restriction affecting only the Auction House as sender does not stop ingress. While that restriction is active, a 100-token fixed-price sale delivers the NFT but credits the entire seller payout to a sweep-only receiver. After the issuer unblocks the Auction House, a later payout reaches the receiver and its controller sweeps the tokens, yet the earlier credit remains fully backed and inaccessible.

Root cause

`TLAuctionHouse.claimFunds()` binds withdrawal authority to the credited address as transaction sender and provides no separately authorized retry or delegation path.

The failed-payment path uses the payout destination as the credit owner, while the recipient check establishes only that the address is nonzero and unsanctioned. It neither requires a contract recipient to originate an Auction House call nor records a recovery controller. Because `claimFunds()` can change only the delivery destination and the completed purchase has already removed the listing, no remaining function can repair the authority binding.

Conditions and assumptions

Both manifestations require a contract payout or refund recipient with no outbound execution, claim-forwarding, or upgrade capability for as long as recovery is needed. For native currency, a failed transfer—or a path such as canceled-auction recovery that creates credit directly—must create the balance. Exceeding the native-transfer gas allowance can cause that failure, but does not strand funds if the recipient can later call; EOAs and callable or upgradeable contracts can claim normally.

The ERC-20 manifestation additionally requires an allowlisted token whose issuer or blocklist administrator can block transfers from the Auction House while buyer ingress remains usable, with the restriction active during the purchase and later removed. The fixed receiver must lack a generic executor, claim method, upgrade path, and token-level recovery. If the token also blocks ingress, the purchase reverts; pausing or changing the payout receiver before finalization avoids the transition.

Impact

The proofs demonstrate inaccessible seller proceeds after NFT delivery: 1 ETH in the native-currency flow and the full 100-token payout in the ERC-20 flow. Each credit remains accounted for and backed in the Auction House, but the beneficiary has no route to exercise the ledger entitlement.

The same authority boundary governs previous-bidder payouts, direct canceled-auction credits, and seller, royalty, and protocol distributions. A seller payout or refund can represent the full transaction value; royalty and protocol-fee exposure is limited to the configured share. These additional roles follow from the shared source path rather than separate proof executions.

This finding is rated Medium because the entire affected payout can become permanently inaccessible after completed settlement. The fixed-receiver and payout-failure prerequisites—and, for the ERC-20 manifestation, privileged issuer action—limit likelihood. Harm is bounded to affected recipients and credits: no attacker receives the funds, the demonstrated accounting remains fully backed, and no protocol-wide insolvency is established.

Recommendation

Separate credit ownership from exclusive transaction-sender authority. Record a recovery controller or refund address when each payout or refund recipient is configured, before any credit can arise, or add a permissionless retry that can send funds only to the credited beneficiary after a transient failure clears. Bind any delegation to the beneficiary, currency, and amount; authenticate delegated claims and prevent replay; preserve the credit if delivery fails; and retain sanctions checks.

Alternatively, when no preauthorized recovery authority exists, make a payout failure revert finalization atomically so NFT ownership and payment do not diverge. This restores atomicity at the cost of allowing a malfunctioning token or payee to block settlement. Do not give sellers, marketplace administrators, or unrelated callers unrestricted power to redirect another beneficiary’s credit.

Validation objective

Verify that a fixed recipient can recover after native-payment acceptance or ERC-20 egress resumes, either through retry to itself or a controller authorized before credit creation. Confirm that unrelated actors cannot redirect the credit, repeated claims cannot spend it twice, and failed retry or delegated delivery preserves the full ledger balance. Cover seller payouts, bidder and canceled-auction refunds, royalties, protocol fees, native currency, ERC-20 failure paths, pause behavior, and sanctions checks; for a source-selective token block, require either full transaction reversion without transferring the NFT or retaining the buyer’s payment, or recovery after unblocking. Also confirm that ordinary direct payouts remain sweepable.

HEX-011: Post-bid fee and royalty changes can reduce locked seller proceeds

- ID: HEX-011
- Status: New
- Class: Finding
- Severity: Medium
- Location: `TLAuctionHouse._completeSale()`

Summary

After a bid prevents a seller from updating or delisting an auction, settlement can apply later changes to protocol fees, creator classification, or royalty configuration. Authorized administrators can therefore reduce the seller's payout by up to 5% for protocol fees or 10% for royalties while the bidder's escrowed payment and NFT delivery remain unchanged.

Description

Expected behavior

Once the first accepted bid disables the seller's ordinary update and delist paths, settlement should preserve a seller-approved bound on deductions or minimum net proceeds. Administrative fee and royalty changes may apply prospectively to auctions that have not received a bid; applying them to an already-bound auction should require seller consent or an effective exit before settlement.

Observed behavior

The `Listing` record stores the bid and whether the protocol fee is waived, but it does not commit the fee rate, sale classification, royalty policy, maximum deduction, or minimum seller payout. `TLAuctionHouse.bid()` escrows the bid, after which `updateListing()` and `delist()` reject the seller.

The auction-house owner can still change the protocol fee, creator lookup, or outer royalty lookup. Separately, an ERC721TL collection authority can change a token royalty (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:300–311`), and the configured royalty-helper owner can replace its nested engine. At settlement, the auction uses the current classification and royalty quote, caps aggregate royalties at 10%, and deducts the current fee and royalties before paying the seller.

Separate reproductions show the resulting payout changes on 10 ETH bids. Raising the protocol fee from zero to 500 basis points leaves the seller with 9.5 ETH. Replacing a 1% outer royalty quote with a capped 10% quote pays the original recipient nothing, the replacement recipient 1 ETH, and the seller 9 ETH; isolated token-royalty, inner-engine, and creator-classification changes also produce a 1 ETH royalty and 9 ETH seller payout under their respective prerequisites. In the coordinated variant, classification-only and engine-only controls each leave the seller with 10 ETH, while applying both changes pays 1 ETH to the engine-selected recipient and 9 ETH to the seller.

Root cause

`TLAuctionHouse._completeSale()` partitions escrowed proceeds using mutable settlement-time fee, classification, and royalty state that the listing never commits when the first bid binds the seller.

The settlement path evaluates `_isPrimarySale()` and `_quoteRoyalties()` against current configuration, while the packaged royalty helper delegates to its current engine or the token's current ERC-2981 response. The royalty cap limits the size of a change but does not prevent retroactive repricing. The delayed `cancelUnsettledAuction()` path does not restore seller control because permissionless settlement is available after expiry before the additional seven-day cancellation window opens.

Conditions and assumptions

An affected auction must have a binding bid, remain unsettled, and otherwise be able to settle. An authorized role must change payout-affecting state after that bid and before settlement, and the resulting fee or royalty recipient must be economically distinct from the seller; no access-control bypass is involved.

The fee manifestation requires a non-waived listing. Royalty manifestations require settlement to consult the changed policy layer; their specific secondary-sale, fallback, classification, and coordination prerequisites are described with the corresponding proof variants.

For auction-house and royalty-helper owner paths, applicability assumes compromise or an authorized update whose retroactive effect falls outside the seller's accepted terms. Collection authorities can be economically distinct from secondary sellers. Sellers can price against the published 5% and 10% caps, but they cannot commit a lower observed deduction after a bid binds them.

Impact

The demonstrated harm is a direct reduction in locked seller proceeds. On a 10 ETH winning bid, the fee variant moves 0.5 ETH to the fee receiver, while the royalty variants can move 1 ETH to a royalty recipient; outer-provider replacement can also displace the recipient identified by the earlier quote. The absolute loss scales with sale value even though the deductions are proportionally capped.

This is rated Medium because settlement can materially reallocate an already-escrowed payment after the seller loses its ordinary exit. The 5% protocol-fee cap, 10% aggregate royalty cap, privileged or configuration-specific prerequisites, and per-sale demonstrated scope keep the issue below High. The bidder is not charged above the winning bid and receives the NFT.

Some affected settings are global and could apply to multiple qualifying auctions that settle afterward, but the reproductions establish only individual sales. They do not quantify aggregate or deployment-wide exposure.

Recommendation

Commit an enforceable seller-side settlement bound no later than the first accepted bid. This can snapshot the applicable fee and an immutable royalty-policy version covering every mutable layer, or record a seller-selected maximum aggregate deduction or minimum net payout that settlement must enforce. Committing only the outer royalty-helper or engine address is insufficient if nested engine state or the token's royalty response can still change.

Alternatively, make the auction apply fee, lookup, classification, token-royalty, and engine changes only to auctions whose first bid occurs after the new policy takes effect. If dynamic settlement-time policy must apply to funded auctions, give the seller an effective consent or atomic unwind path that refunds the bid and returns the NFT before changed terms can settle.

Snapshotting adds state and can delay legitimate policy updates; versioning or economic ceilings preserve more flexibility but require explicit higher-bid, top-up, and recovery semantics. Engine-change events can improve monitoring, but notice alone does not restore the commitment.

Validation objective

Verify that each post-bid mutation—protocol-fee increase, outer-lookup replacement, token-royalty change, inner-engine replacement, creator reclassification, and the coordinated classification-and-engine case—either preserves the existing seller’s committed payout boundary or enters the selected atomic recovery path. Confirm that changes made before the first bid govern new auctions, fee waivers remain effective, higher bids and top-ups use the committed policy correctly, and access controls, caps, permissionless settlement, NFT delivery, and gross-bid accounting remain intact across supported auction types and native and allowlisted ERC-20 payments.

HEX-012: Royalty-gated transfer validation can strand escrowed NFTs

- ID: HEX-012
- Status: New
- Class: Finding
- Severity: Medium
- Location: TLAuctionHouse._list()

Summary

A seller can lose access to an NFT held in `TLAuctionHouse` custody when a zero-to-positive royalty change activates a configured transfer validator that rejects auction-house-operated transfers. The token can enter escrow while validation is skipped, but delisting, settlement, and later claiming then revert until an authorized party restores transferability.

Description

Expected behavior

`TLAuctionHouse` should accept a policy-sensitive NFT into escrow only when its supported delisting, settlement, and claim paths will not depend on a mutable collection setting continuing to suppress an otherwise rejecting transfer policy. It should reject an incompatible listing before taking custody or retain an exit that does not rely on that same condition.

Observed behavior

`TLAuctionHouse._list()` escrows the NFT through an ordinary `transferFrom`. For `ERC721TL` assets, `ERC721TL._update()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:429–439`) invokes the configured validator only for a non-mint, non-burn transfer with a positive effective royalty; at zero royalty, it skips validation. An NFT can therefore enter escrow even when the installed validator would reject the auction house if called.

The collection owner or an administrator can later make the applicable royalty positive through the royalty setters (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:301–311`). The unchanged validator then runs when the house transfers the NFT out through `TLAuctionHouse._transferNft()`, causing delisting, settlement, and claiming to revert if it rejects the auction house. Each failed transfer leaves the NFT in custody.

For a bid auction, cancellation after the seven-day grace period still credits the bidder's full bid and records an NFT claim for the seller. The bidder can recover the principal, but the seller's claim repeats the rejected transfer and cannot recover the NFT.

Root cause

`TLAuctionHouse._list()` accepts a policy-sensitive NFT into custody without establishing that the auction house can later transfer it out when mutable token policy becomes active.

For `ERC721TL`, validator invocation is coupled to the effective royalty, so zero royalty can conceal a policy that would reject auction-house-operated transfers once the royalty becomes positive. Because the affected exit paths use the same policy-sensitive transfer behavior, the auction house assumes that successful ingress under the token's current state implies

continued transferability, without an independent compatibility guard or protected recovery path.

Conditions and assumptions

This requires a configured validator that rejects auction-house-operated transfers, zero effective royalty when the seller approves and lists the NFT, and a later authorized change to a positive applicable royalty. A positive token-specific override, which supersedes the default (`dependencies/tl-creator-contracts-4.1.1/src/lib/ERC2981TLUpgradeable.sol:158–169`), prevents the zero-royalty ingress condition.

The royalty change requires the collection owner or an `ADMIN_ROLE` holder, while the affected seller need not hold collection privileges. The positive royalty and incompatible validator policy must remain in place for the NFT to remain inaccessible.

The accepted package does not establish that a current deployment uses an incompatible validator. Its initializer policy list is intended to allow Transient protocol operations, so standard configurations may not satisfy this prerequisite; the condition applies to current or future configurations only if they install such a validator.

Impact

The seller can lose access to the escrowed NFT for a potentially unbounded period. An unbid listing cannot be delisted; a bid auction cannot settle, and the seller's recorded NFT claim cannot execute after cancellation.

For a bid auction, cancellation after the seven-day grace period still lets the bidder recover the full principal. Fixed-price failures revert atomically, the NFT is not transferred to an attacker, and collection authority can restore recovery by returning the effective royalty to zero or otherwise restoring compatibility.

The Medium rating reflects the prolonged loss of seller access under plausible but specific conditions. The privileged trigger, configuration dependency, listing-local scope, reversibility, and recovery limits keep the consequence below High.

Recommendation

Add a listing-time escrow safeguard in `TLAuctionHouse._list()`: reject a policy-sensitive NFT before taking custody unless compatibility with auction-house-operated exits is established independently of its current effective royalty, or provide a protected recovery path that does not rely on the same policy-sensitive `transferFrom` behavior. An additional rescue function that uses the same ordinary transfer path would encounter the same validator rejection.

As a secondary integration change, `ERC721TL._update()` could invoke any configured validator for every non-mint, non-burn transfer, including at zero royalty. This would expose an already-incompatible policy during listing, but it changes existing zero-royalty behavior and adds the external call, gas cost, and policy restrictions for collections that currently leave those transfers unrestricted.

Validation objective

Regression tests should show that a zero-royalty NFT with an already-rejecting configured validator is rejected before custody, leaving seller ownership and listing state unchanged. They should also cover compatible listing, delisting, settlement, and claiming at zero and positive royalties, and confirm that an allowed royalty change cannot strand an accepted NFT. Treat independent post-custody validator replacement or internal-policy changes as separate cases.

HEX-013: Post-escrow validator replacement can defeat auction finality

- ID: HEX-013
- Status: New
- Class: Finding
- Severity: Medium
- Location: TLAuctionHouse._list()

Summary

A seller aligned with ERC721TL collection authority can replace an enabled transfer validator after custody and bidding, block delivery of the NFT, and force the auction into delayed cancellation. The bidder eventually recovers the full principal, but auction finality fails and the bidder bears extended illiquidity and associated costs while the seller can recover the NFT after restoring transfers.

Description

Expected behavior

Once the auction house has taken custody of an NFT and accepted a bid, seller-aligned collection authority should not be able to invalidate the winning transfer. The marketplace should either preserve an executable NFT-egress path or constrain and compensate for collections whose transfer policy remains mutable during escrow.

Observed behavior

`TLAuctionHouse._list()` takes custody without committing to the transfer policy that will govern the NFT's eventual exit. For positive-royalty ERC721TL tokens, `ERC721TL._update()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:429–439`) invokes the collection's current transfer validator before changing ownership.

After custody and bidding, the collection owner or an `ADMIN_ROLE` member can replace that validator through `ERC721TL.setTransferValidator()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:410–415`). A rejecting replacement prevents `TLAuctionHouse.delist()` from returning an unbid NFT, makes `TLAuctionHouse.settleAuction()` revert before delivery or payout, and blocks redemption of the seller's NFT claim through `TLAuctionHouse.claimNft()`.

After the auction and its seven-day grace period, `TLAuctionHouse.cancelUnsettledAuction()` can cancel without transferring the NFT, credit the winning bid to the bidder, and record an NFT claim for the seller. The documented recovery flow holds the NFT for the seller after such a cancellation (README.md (`README.md:46–49`)); restoring the permissive validator then allows the seller to recover the NFT and the bidder to withdraw the full principal.

Root cause

`TLAuctionHouse._list()` accepts ERC721TL custody without binding or constraining the mutable transfer validator that governs later NFT egress.

ERC721TL evaluates the live validator pointer at transfer time, while the auction listing records no enforceable validator commitment. Delisting, settlement, and claiming all ultimately depend on the same `TLAuctionHouse._transferNft()` path, so the auction house has no validator-independent route for returning or delivering the NFT.

Cancellation separates bidder-fund recovery from the failing NFT call, but it only records an NFT claim. Redeeming that claim still depends on the collection's current transfer policy, allowing seller-aligned collection authority to turn the broken-token recovery flow into a post-bid cancellation option.

Conditions and assumptions

The path requires an ERC721TL collection initialized with a nonzero validator and a positive effective royalty at each blocked transfer; a zero-initialized validator cannot later be enabled through this setter. The collection owner or an `ADMIN_ROLE` member must replace the pointer after custody and bidding but before the first successful permissionless settlement, then maintain rejection through cancellation.

For the cancellation option to benefit the seller, the seller must control or coordinate with collection authority. These are supported native configurations, but they limit the finding to auctions with the stated collection policy, authority, and timing.

Impact

A seller aligned with collection authority can make a winning auction non-binding after observing the bid. The bidder's funds remain locked from bidding through the auction and until strictly more than seven days after its end, after which cancellation and withdrawal recover the full principal. The bidder may nevertheless bear failed-settlement gas, cancellation costs, financing costs, and opportunity loss.

The seller's NFT remains inaccessible while the rejecting validator persists. After cancellation and policy restoration, the seller can retain the NFT instead of honoring the winning bid; any economic benefit depends on the NFT's later value exceeding the seller's forgone net proceeds and costs.

The severity is Medium because the condition causes a consequential loss of auction finality, prolonged bidder illiquidity, and conditional NFT unavailability under plausible but privileged conditions. Atomic settlement prevents seller payout before delivery, full bidder-principal recovery remains available, and restoring the validator recovers the NFT. The evidence does not establish principal theft, irreversible NFT loss, cross-listing corruption, or protocol-wide denial of service.

Recommendation

Make bid acceptance conditional on transfer behavior that seller-aligned collection authority cannot change until escrow ends. For ERC721TL integrations, consider a token-level delay or lock on validator-pointer changes affecting escrowed tokens, or a narrowly scoped auction-egress authorization committed when custody begins. Recording the old pointer in auction metadata is insufficient unless the token enforces that commitment.

If unrestricted ERC-721 compatibility is retained, identify mutable-policy collections before bidding and consider seller collateral designed to compensate bidders when a post-bid transfer failure forces cancellation. Eligibility restrictions reduce composability, while collateral does not restore auction finality or NFT availability. Any protected egress route should avoid bypassing unrelated transfer-safety controls, and pointer locking addresses this condition only if the validator's own behavior cannot change independently.

Validation objective

List positive-royalty tokens under an enabled permissive validator, accept a bid, and attempt pointer replacement through both collection-owner and `ADMIN_ROLE` authority. Verify that the correction rejects or defers the change, prevents bid acceptance, or preserves delisting, settlement, and claiming despite the change.

Retain the recovery behavior for genuinely broken NFTs: cancellation remains available after the grace period, bidder principal remains recoverable, and failed NFT claims remain recorded. Cover zero-initialized validators, zero royalties, unbid delisting, permissionless settlement, and post-cancellation claiming as boundary cases.

HEX-014: Directional transfer policies can strand NFTs in auction escrow

- ID: HEX-014
- Status: New
- Class: Finding
- Severity: Medium
- Location: `TLAuctionHouse._list()`

Summary

A directional transfer policy can allow `TLAuctionHouse` to escrow a seller's NFT while rejecting every later transfer from the auction house. The NFT can remain inaccessible until collection authority restores a compatible validator or royalty configuration; fixed-price payments revert, and auction bidders retain a delayed full-refund path.

Description

Expected behavior

For a recognized policy-gated collection, an NFT accepted into auction-house custody should retain a policy-compatible delivery or return path. If the auction house cannot support the collection's outbound policy, listing should fail before ownership leaves the seller.

Observed behavior

For a positive effective royalty, `ERC721TL._update()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:428–439`) validates a transfer using the caller, current owner, recipient, and token ID. During `TLAuctionHouse._list()`, those values are the auction house, seller, and auction house, so a directional validator can admit the NFT.

After custody changes, `delist()`, delivery through `_completeSale()`, and `claimNft()` initiate transfers from auction-house ownership. The unchanged validator can therefore reject delisting, fixed-price delivery, auction settlement, and the seller's eventual NFT claim.

After failed settlement, `cancelUnsettledAuction()` credits the bidder and records the seller's NFT claim without transferring the NFT. The bidder can withdraw the credited bid, but claiming the NFT repeats the blocked transfer; transaction rollback preserves both the seller's claim and auction-house ownership.

Root cause

`TLAuctionHouse._list()` accepts a policy-gated NFT after validating only seller-to-house ingress, without establishing that the NFT can later leave auction-house custody.

Once custody changes, the policy sees a different transfer direction. Every delivery and recovery route reuses `_transferNft()`, which performs an ordinary `transferFrom` followed by `ownerOf`. `_list()` has no pre-admission egress compatibility check, and the helper provides no protected validator-aware egress path.

Conditions and assumptions

The NFT must have a positive effective royalty and a nonzero, interface-compatible validator before listing. The unchanged policy must allow `(caller = auction house, from = seller, to = auction house)` while rejecting transfers whose `from` is the auction house, and the seller must own, approve, and list the NFT. The prerequisite validator configuration is controlled by an `ERC721TL` owner or administrator (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:301–315`).

The NFT remains blocked only while the restrictive validator and positive royalty remain applicable. Collection authority can alter royalties or replace or disable the validator (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:409–415`); indefinite inaccessibility therefore assumes that authority is unavailable, renounced, or unwilling to restore compatibility.

A compatible static validator follows the `ITransferValidator` (`dependencies/tl-creator-contracts-4.1.1/src/interfaces/ITransferValidator.sol:4–27`) interface; whether a particular validator enforces the directional rule depends on its configured implementation.

Impact

An affected seller can lose access to one listed NFT while the restrictive policy remains. An unbid listing cannot be delisted. A fixed-price purchase reverts atomically, so the buyer retains payment; a bid auction cannot settle, but after the seven-day grace period either party can cancel and the bidder can recover the full bid while the seller's NFT claim remains blocked.

Medium severity reflects the NFT's potentially unbounded inaccessibility. The specific privileged collection configuration, possible repair by collection authority, listing-local scope, and complete bidder recovery constrain the rating. The demonstrated outcome is prolonged or potentially indefinite inaccessibility, not proven irreversible asset loss or a protocol-wide effect.

Recommendation

Reject recognized validator-enabled NFTs before custody unless the auction house can support policy-compatible egress. Possible approaches include an authoritative validator simulation for a house-to-seller return, restricting accepted validators and rulesets to known-compatible combinations, or consistently using the validator's protected authorized-transfer workflow across delisting, delivery, and claiming.

Validator-specific integration adds gas and coupling, while configuration restrictions narrow ERC-721 compatibility. A recovery path that merely repeats `transferFrom` cannot override a policy that rejects the transfer.

Validation objective

Add regression coverage showing that a positive-royalty NFT with an unchanged directional policy is either rejected before ownership leaves the seller or can exit successfully through delisting, fixed-price delivery, auction settlement, and post-cancellation claiming. Verify that failed admission leaves no listing and that bidder refund accounting remains intact.

HEX-015: Sender-blocked ERC-20 payouts can finalize NFT sales against inaccessible consideration

- ID: HEX-015
- Status: New
- Class: Finding
- Severity: Medium
- Location: `TLAuctionHouse._completeSale()`

Summary

`TLAuctionHouse` can transfer an NFT to the buyer and remove its listing even when a sender restriction prevents the ERC-20 sale proceeds from leaving the auction house. Failed payouts become fully backed credits, but redirecting a claim still sends from the restricted auction house, leaving the seller without usable consideration until the token issuer removes the restriction.

Description

Expected behavior

An ERC-20 sale should not irreversibly deliver the NFT unless the seller's settlement-critical proceeds can leave escrow. If outbound payment is impossible, completion should revert atomically so the NFT and listing remain available for settlement or safe unwind.

A deferred credit presented as redirectable should also provide a recovery path that addresses the reason the original payment failed.

Observed behavior

Incoming ERC-20 custody uses a strict balance-checked transfer. For an auction, this allows the bid to enter custody before the token administrator restricts the auction house. A fixed-price purchase can accept payment under a preexisting source-only restriction because the buyer, rather than the auction house, is the incoming transfer's sender.

Both auction settlement and a fixed-price purchase reach the common completion path. `_completeSale` transfers the NFT before processing payment, while `_payout` uses a non-reverting ERC-20 transfer and records a credit for each failed outgoing transfer. Every outbound transfer can therefore fail while sale finalization still succeeds.

In both reproduced paths, the buyer received the NFT, the listing no longer existed, the full ERC-20 price remained in the auction house, and the seller and fee recipient held correctly backed credits that they could not redirect. A later `claimFunds` call changes only the destination while retrying the transfer from the same restricted sender; its revert preserves the credit but does not make the tokens accessible. In the auction branch, the seller also cannot use post-settlement cancellation to recover the NFT.

Root cause

`TLAuctionHouse._completeSale()` transfers the NFT before establishing that the auction house can send the seller's ERC-20 proceeds.

The package documents NFT-first payout ordering and claim redirection for failed ERC-20 blocklist payouts in README.md:46-49 (`README.md:46–49`). The fail-soft abstraction treats recipient-specific and sender-wide transfer failures as equally recoverable through that destination change. It neither verifies that the auction house remains capable of sending the currency before committing NFT delivery nor provides a recovery route that uses a different sender.

Conditions and assumptions

This condition requires an owner-allowlisted ERC-20 whose administrator can reject transfers from the auction house. The scheduled-auction variant requires successful bid escrow followed by the sender restriction before settlement; the fixed-price variant requires a source-only restriction that still permits the buyer-to-auction-house transfer. The marketplace must remain unpaused and finalization must occur while the restriction is active. Once the restriction exists, a purchase is ordinary and auction settlement is permissionless.

A two-sided restriction active before a fixed-price purchase blocks the incoming payment, whereas an auction remains affected if that restriction is imposed after custody. The inaccessibility lasts only until the restriction is removed, and issuer unblocking restores withdrawal of the fully backed claims. Native-currency sales are outside this sender-freeze mechanism.

Impact

The buyer or bidder can receive the NFT and the listing can be removed while the complete ERC-20 price remains in the auction house. The seller and fee recipient hold credits they cannot use while the sender restriction persists, and the seller can no longer cancel the finalized sale to recover the NFT.

A prolonged restriction can therefore leave the seller without either the NFT or usable consideration. Because sales share the auction-house sender, the restriction may affect multiple active sales using the same currency.

This is not theft or insolvency: the credits remain fully collateralized, and issuer unblocking restores withdrawal. The condition is rated Medium because it causes consequential incorrect settlement, but the required privileged token restriction, low likelihood, full backing, and issuer-dependent recovery keep it below High. The evidence does not establish restriction duration, exposure in any deployed instance, or value at risk.

Recommendation

Make ERC-20 sale finalization conditional on a successful strict transfer of the seller's net proceeds. If that settlement-critical transfer fails, revert the entire transaction so the payment, NFT ownership, and listing state remain unchanged; ancillary protocol or royalty payments may remain fail-soft if that availability tradeoff is intentional.

A strict seller transfer may delay settlement when only the recipient is blocked, although the seller can update the payout receiver before finalization. For auctions, retaining the listing also keeps the grace-period unwind available. Any alternative recovery design should not depend exclusively on transfers from the restricted address and should preserve authorization and solvency invariants.

Validation objective

Regression tests should verify that a post-custody sender restriction makes auction settlement revert without deleting the listing or transferring the NFT, and that a preexisting source-only restriction makes `buyNow` revert without retaining buyer funds. They should also confirm that recipient-only failures retain the intended redirection behavior if fail-soft payouts remain supported, grace-period cancellation preserves the seller's NFT when settlement cannot pay, and removing and later restoring a restriction neither corrupts credits nor permits duplicate withdrawal.

HEX-016: Royalty-engine cache poisoning can suppress secondary-sale royalties

- ID: HEX-016
- Status: New
- Class: Finding
- Severity: Medium
- Location: RoyaltyLookup.getRoyalty()

Summary

An untrusted caller can make an affected Manifold V1 engine persist `NONE` for an entire collection by querying a nonexistent token ID. When Stacks is configured with that engine, later valid secondary sales accept the empty quote and leave the omitted royalty—up to 10% of the sale value—with the seller.

Description

Expected behavior

A failed royalty probe for one caller-selected token ID should not change royalty detection for other valid tokens. If a valid token quotes a royalty, later secondary-sale settlement should continue paying that royalty unless an authorized configuration change removes it.

Observed behavior

Manifold V1 caches a royalty specification by registry-resolved royalty address rather than token ID. A caller can publicly invalidate an existing correct cache and invoke the mutating adapter path with a nonexistent token whose royalty probe reverts. If every other specification probe and fallback also fails, the engine caches `NONE`.

While `NONE` remains cached, valid-token previews and settlement return empty arrays without re-detecting EIP-2981. The auction's royalty-quote path accepts that empty response, and sale completion transfers the NFT, pays no creator royalty, and leaves the omitted share with the seller's payout receiver.

The reproduction establishes a baseline 10% valid quote, permissionless invalidation, invalid-ID poisoning, an empty valid-token preview, and a completed purchase in which the creator receives zero and the seller receives the full test price.

Root cause

`RoyaltyLookup.getRoyalty()` forwards caller-selected token IDs into a state-mutating royalty-engine lookup without isolating collection-wide cache changes from token-specific lookup failures.

Manifold V1 derives a negative classification from token-ID-specific probes but persists it in a collection-scoped cache; the packaged royalty-engine interface explicitly describes specification caching. Public invalidation and mutation let an untrusted caller choose the probe that determines this shared state.

Stacks then treats an empty engine response as authoritative during settlement and has no independently distinguishishable “unknown specification” state. Restricting only `RoyaltyLookup.getRoyalty()` would be insufficient because the engine’s invalidation and mutating lookup are themselves public.

Conditions and assumptions

Applicability requires an affected Manifold V1 or behaviorally equivalent engine selected through the owner-controlled configuration. Current deployment exposure is not established by the supplied evidence.

Registry resolution must reach the affected collection or another royalty address with the same token-ID-sensitive behavior, without a successful alternative specification or nonempty fallback. A valid token must return an EIP-2981 royalty while an unknown token ID makes every detection probe fail. The packaged Transient Labs and OpenZeppelin royalty implementations return default data for unknown IDs and do not meet this condition.

The effect applies to secondary sales and requires a mutating engine call. Read-only previews cannot themselves poison the cache, although they consume an already-poisoned result.

Impact

The reproduction directly demonstrates one completed sale where a quoted 10% creator royalty becomes zero and the seller retains the omitted amount. The auction caps royalty payouts at 10% of sale value, bounding direct creator loss per affected sale.

Repeated losses are inferred rather than directly measured: they require qualifying secondary sales while the cache remains poisoned. Anyone can restore detection by invalidating the cache and issuing a valid lookup, but an attacker can repeat the same public poisoning sequence.

Medium severity reflects materially incorrect settlement with a permissionless, repeatable trigger under the stated configuration. The specific engine and collection prerequisites, together with unestablished current deployment exposure, limit the practical scope; no NFT theft, escrow loss, settlement denial, or protocol-wide harm was demonstrated.

Recommendation

Configure and pin a royalty-engine version that preserves the invariant that an untrusted token-specific lookup cannot persist a collection-wide negative classification. At the engine layer, cache only positive specification detections, validate the token used for negative detection, or represent “not detected for this token” separately from a collection-wide absence of royalties.

Any access-control change must cover direct engine invalidation and mutation rather than only the Stacks adapter. Avoiding negative caching may increase gas for collections without royalties, while restricting cache refresh can complicate legitimate royalty-standard migrations; account for those tradeoffs in the chosen correction.

Validation objective

After a valid royalty has been detected, invalidate the cache and query multiple nonexistent token IDs through both the adapter and direct engine entrypoints. A subsequent valid-token preview and secondary-sale settlement must still return and pay the original creator royalty. Confirm that legitimate royalty-standard changes remain refreshable and genuinely royalty-free collections remain usable.

HEX-017: Pending purchases can consume allowance in a different sale phase

- ID: HEX-017
- Status: New
- Class: Finding
- Severity: Low
- Location: TLStacks721._validatePurchase(); TLStacks1155.purchase()

Summary

Presale-valid purchase calldata can execute after a regular drop enters public sale because the buyer cannot bind the transaction to the expected phase. The call is then validated and recorded against the public phase, consuming the recipient's separate public allowance and potentially causing a later public purchase to fail.

Description

Expected behavior

The project documentation (`README.md:21`) states that purchase parameters protect a buyer when a drop's phase changes while the transaction is pending. Regular drops maintain separate presale and public-sale allowances for ERC-721 and ERC-1155, so a presale-bound purchase should not silently consume public allowance after the transition.

The ERC-721 purchase NatSpec describes price and allowance in terms of the current phase. The condition is therefore a gap between the broad pending-transaction protection and the explicit execution-time semantics, rather than proof that the existing ABI unambiguously promises phase pinning.

Observed behavior

TLStacks721._validatePurchase() derives the live phase before interpreting proof and allowance inputs, and TLStacks721._updateDropState() records the purchase under that phase. TLStacks1155.purchase() performs the equivalent selection inline, with TLStacks1155._updateDropState() applying the selected accounting bucket. Once public sale is active, both paths ignore the supplied presale proof and presale allowance.

The proofs encoded calldata that was valid during presale, restored the pre-purchase state, advanced only the timestamp across the scheduled boundary without changing the drop round, and replayed the same inputs and ETH value. The ERC-721-only and cross-standard variants recorded a public mint rather than a presale mint; the follow-up public purchases then reverted because the recipient's separate public allowance had been exhausted.

Root cause

`TLStacks721._validatePurchase()` and `TLStacks1155.purchase()` select the execution-time phase without checking a buyer-supplied expected phase.

The entry points bind the drop round, currency, and maximum unit price, but not the phase. A scheduled phase boundary does not change the round, while compatible currency and pricing leave the other commitments valid. The live phase therefore controls proof interpretation, cost selection, allowance checks, event attribution, and counter updates.

Conditions and assumptions

This affects regular drops with a presale followed by a public sale. A presale-valid transaction must remain pending until public sale while the round and currency stay unchanged, supply and recipient public allowance remain available, the public price does not exceed `maxCostPerToken`, and ordinary sanctions, mint-permission, receiver, payout, and settlement checks pass. Velocity drops reject presales and are not affected.

For native currency, `msg.value` must exactly match the live sale amount plus the protocol fee, so equal all-in phase pricing with an unchanged fee is the demonstrated case. For ERC-20 payment, zero ETH is required; source inspection indicates that the public cost may differ if it remains within the submitted maximum and the payer has sufficient balance and approval.

A natural scheduled transition requires no privileged action. A collection owner or NFT `ADMIN_ROLE` holder can also accelerate the boundary through TLStacks721.updateDropDuration() or TLStacks1155.updateDropDuration().

Impact

A delayed presale-valid purchase can consume the recipient's public-sale allowance instead of the presale allowance, causing a later public purchase to revert. The economic value of that lost opportunity depends on the sale's demand and token value.

The severity is Low because the demonstrated harm is a narrow loss of phase-specific control. The call preserves the requested quantity and recipient as well as the expected round and currency; the live price cannot exceed the submitted maximum, and supply and per-phase caps remain enforced. The evidence does not establish theft, overcharging, an allowlist bypass, or excess minting.

The proofs establish which allowance the call consumes, not loss of the minted token or recovery of the expired presale opportunity. Phase binding would reject the stale call and leave the public allowance available, but it would not complete the expired presale purchase.

Recommendation

Add a buyer-supplied expected phase to every affected regular-drop purchase entry point, including `TLStacks721.purchase()`, `TLStacks721.purchaseSelected()`, and `TLStacks1155.purchase()`. Reject a mismatch before interpreting phase-specific proof fields, checking allowance, taking payment, or updating state, while preserving the existing round, currency, and maximum-price checks.

If changing the current ABI is too disruptive, expose a versioned or phase-specific entry point and clearly distinguish the legacy execution-time semantics. A deadline can supplement phase binding, but it cannot prevent an administrator from ending presale early. Strict binding also introduces expected boundary-time reverts and does not restore an expired presale opportunity.

Validation objective

Verify that presale-bound calls revert without transferring payment or changing balances, supply, ownership, or either phase counter after both a natural transition and an administrator-induced early transition. Cover ERC-721 and ERC-1155 regular purchases, the ERC-721 selectable path, equal-price native payment with an unchanged fee, and ERC-20 public prices at or below the submitted maximum; correctly phase-bound presale and public calls should continue to succeed.

HEX-018: Unsafe percentage arithmetic can block high-unit auction settlement

- ID: HEX-018
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLAuctionHouse._completeSale()`; `TLAuctionHouse._capRoyalties()`

Summary

TLAuctionHouse can accept and escrow high-raw-unit ERC-20 auction bids that later cannot settle because its protocol-fee and royalty-budget calculations multiply before dividing. A triggering bid leaves the winner's funds and seller's NFT unavailable until owner intervention or cancellation after the strict seven-day grace period, forcing the auction to unwind.

Description

Expected behavior

An accepted and fully escrowed auction bid should remain settleable when the NFT, currency, parties, and mathematically intended fee and royalty payouts are otherwise valid. Percentage calculations should cover the admitted value domain whenever their final results fit in `uint256`, or the bid should be rejected before escrow.

Observed behavior

`TLAuctionHouse.bid()` accepts and escrows amounts without checking whether the downstream percentage products fit in `uint256`. With a 250-basis-point protocol fee, a bid of `floor(type(uint256).max / 250) + 1` is accepted, but settlement after the auction ends panics. The revert preserves the listing, escrowed NFT, and full bid; removing only that listing's protocol fee allows the unchanged auction to settle.

With the protocol fee disabled, a secondary-auction bid of `floor(type(uint256).max / 1,000) + 1` is likewise accepted and escrowed, but royalty-budget calculation panics before NFT delivery. The production ERC-2981 fallback can return equal empty arrays for a plain ERC-721, so this manifestation does not require a positive royalty. The listing, NFT, and bid remain escrowed; cancellation is rejected at exactly the end time plus seven days, succeeds one second later, and enables both parties to reclaim their full assets.

The royalty-budget expression is also reached by secondary fixed-price completion and public royalty quotation. At qualifying values those calls panic, but the fixed-price failure is atomic and does not create the auction's custody delay.

Root cause

In `TLAuctionHouse`, `_completeSale()` and `_capRoyalties()` use checked multiply-before-divide percentage arithmetic, allowing an intermediate product to overflow even when the final fee or royalty budget is representable.

The protocol-fee branch evaluates `value * protocolFeeBps / BASIS`. For any fee `p >= 2`, a value above `floor(type(uint256).max / p)` overflows during multiplication, although the fee cap keeps the intended result representable. Bid admission imposes no corresponding ceiling, and because the global fee is read at settlement rather

than snapshotted with the bid, a limit based only on the admission-time fee would be incomplete.

The royalty-cap calculation similarly evaluates `value * MAX_ROYALTY_BPS / BASIS`. With `MAX_ROYALTY_BPS` equal to 1,000, the intermediate product overflows above `floor(type(uint256).max / 1,000)` even though the intended budget is only `floor(value / 10)`. The panic occurs after a successful external lookup returns, so the surrounding lookup `try/catch` does not handle it.

Conditions and assumptions

Both manifestations require an owner-allowlisted, standard exact-transfer ERC-20 with sufficient raw supply, a seller who selects that currency, and a bidder who funds a winning bid above the relevant threshold. The bidder needs no privilege. High decimal precision can make the displayed quantity moderate, but the 72- and 77-decimal proof tokens are unusual and decimal metadata alone does not establish economic value.

The protocol-fee case requires an unwaived listing fee and a live settlement-time fee of at least two basis points. The royalty case requires secondary-sale classification and a code-bearing royalty lookup that successfully returns equal-length arrays; empty arrays suffice. Primary sales, missing or reverting lookups, and unequal arrays avoid that royalty path.

No qualifying deployed currency, allowlist, fee setting, or lookup configuration was supplied. The evidence therefore establishes source-level behavior, not exposure of a current deployment.

Impact

A permissionless bidder can prevent one qualifying auction from completing, leaving the seller's NFT and the winning bid unavailable. The seller cannot delist or rewrite an auction after a bid; under unchanged configuration, the parties must wait until the strict post-auction seven-day boundary, cancel, and separately claim their assets unless the protocol owner first corrects the fee or royalty-lookup condition.

The resulting denial and forced unwind can cost the seller the intended sale opportunity, although price movement and other economic loss were not measured. The royalty-budget manifestation also disrupts high-unit secondary fixed-price purchases and royalty quotations, but those calls do not create the demonstrated auction escrow delay.

Severity is Low because the trigger requires exceptional raw-unit values, prior owner allowlisting, seller currency selection, and committed bidder funds; the blast radius is one qualifying auction, owner mitigation is available, and eventual recovery is complete. No theft, insolvency, partial payout, or permanent loss was demonstrated.

Recommendation

Replace both checked multiply-before-divide expressions with full-precision multiplication and division, such as `Math.mulDiv(value, protocolFeeBps, BASIS)` and `Math.mulDiv(value, MAX_ROYALTY_BPS, BASIS)`. Preserve the existing floor rounding and the 10% aggregate royalty cap.

An admission-time ceiling is less direct because it would reject otherwise representable values, must account for every downstream percentage calculation, and cannot rely only on a protocol fee that remains mutable until settlement. Skipping empty royalty schedules alone would leave nonempty schedules exposed.

Validation objective

Verify that auction settlement, secondary `buyNow()`, and public royalty quotation handle values at each overflow boundary, one unit above it, and near `uint256` maximum without arithmetic panic while producing exact floor-rounded fees, capped royalties, conserved payouts, and correct NFT delivery. Cover protocol fees of 0, 1, 2, 250, and 500 basis

points; waived fees; empty and nonempty equal-length royalty schedules; fee changes after bidding; and the deliberate primary-sale, absent-lookup, reverting-lookup, and mismatched-array bypasses. Confirm that unrelated-failure rollback and post-grace cancellation behavior remain unchanged.

HEX-019: Unversioned cancellation calls can terminate successor listings

- ID: HEX-019
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLAuctionHouse.delist()`, `TLAuctionHouse.cancelUnsettledAuction()`

Summary

A delayed seller-authorized cancellation can delete a successor listing because `delist` and `cancelUnsettledAuction` do not bind the operation to the listing generation for which it was prepared. An unbid successor can be removed and its NFT returned to the seller; after the grace period, the same defect can terminate an otherwise-settleable auction, leaving the bidder and seller to recover their assets.

Description

Expected behavior

Seller authorization to cancel a listing should apply only to the intended listing generation. If the NFT is reclaimed and relisted under a fresh ID, an operation prepared for the prior ID should revert before changing the successor listing, escrowed assets, bidder credit, or NFT-recovery state.

Observed behavior

The unbid-listing variant authorizes `delist` while generation A is current, removes A and creates generation B through an independent smart-account nonce lane, and confirms that B has a fresh `Listing.id` and holds the NFT in escrow. Executing the A-era call then deletes B and returns the NFT to the same seller. In a separate case, a bid arriving before the delayed call causes `AuctionStarted` to reject it and preserves the current listing, NFT escrow, and bid escrow.

The auction variant queues the exact cancellation calldata during generation A. After A is canceled by its bidder, the seller reclaims and relists the NFT, and generation B receives a generation-bound `bid`. Once B's auction and seven-day grace period have elapsed, the proof confirms that B can settle successfully, restores the pre-settlement state, and executes the A-era call; B is deleted, the second bidder's principal becomes claimable, and the NFT remains escrowed and claimable by the seller.

Root cause

In `TLAuctionHouse`, `delist` and `cancelUnsettledAuction` act on the current NFT-keyed listing without requiring the generation identifier for which the caller authorized the operation.

The current-listing storage uses one NFT-address and token-ID key, while `_buildListing` assigns a fresh `Listing.id` to each replacement. Both cancellation functions load the current record and evaluate their caller and state checks against it, but neither compares its ID with an expected ID. When the same seller controls both generations, stale seller authorization can therefore pass against and mutate the successor.

By contrast, generation-sensitive bidder operations such as `bid` and `topUpBid` accept a listing ID and reject a mismatched generation.

Conditions and assumptions

The demonstrated path requires genuine seller authorization; it does not let an outsider forge or bypass that authority. The seller must use a smart-account, queued, or relayed transaction system that permits independently authorized operations to remain live and execute out of order, and the same account must remain the successor's seller. Ordinary EOA scalar nonce ordering prevents this sequence; wallet-side cancellation, expiry, or private sequencing can also stop it.

The `delist` manifestation requires the successor to remain active and unbid and the seller to remain unsanctioned; a bid blocks the call. The auction manifestation requires a replacement bid, expiry of that auction and its seven-day grace period, no prior settlement, and the stale caller still qualifying as seller. At that point the seller could intentionally cancel the current auction; the defect is stale intent crossing generations, not an authorization bypass.

Impact

Removing an unbid successor interrupts the sale and returns the escrowed NFT to the authorized seller. The seller can relist, so the supported harm is recoverable availability disruption, with possible lost sale opportunity, relisting costs, and failed pending-buyer transaction costs that were not quantified.

The post-grace variant prevents an otherwise successful auction settlement: the winning bidder does not receive the NFT, and the expected sale does not complete. The bidder's principal is credited for recovery, while the NFT remains escrowed and claimable by the seller, requiring the seller to reclaim and relist.

The final severity is Low because the demonstrated harm is bounded to the successor sale and the assets remain recoverable, while reachability depends on specialized authorization ordering and same-seller relisting. The evidence does not demonstrate NFT theft, permanent asset loss, payout diversion, or attacker profit.

Recommendation

Require both `delist` and `cancelUnsettledAuction` to accept an `expectedListingId` and compare it with the current `Listing.id` before deleting the listing, returning the NFT, or crediting recovery balances. Revert with the existing `ListingChanged` error on a mismatch.

This interface change is ABI-breaking, so existing sellers, bidders, and account integrations may need a versioned migration and must fetch and submit the intended listing ID. Do not retain an externally usable unversioned overload, which would preserve the stale-generation path. Preserve current-generation behavior, including the bid-state guard, while-paused delisting, and cancellation and recovery under the existing pause, sanctions, and settlement-failure semantics.

Validation objective

Regression tests should verify that generation-A calls to both functions revert after generation B becomes current and leave B's listing, NFT escrow, bid escrow, and recovery state unchanged. A delist carrying B's ID should still succeed while B is eligible and unbid, while an intervening bid should continue to trigger `AuctionStarted`. A cancellation carrying B's ID should remain available to an eligible seller or highest bidder after the grace period, including under the intended pause, sanctions, and failed-settlement recovery conditions.

HEX-020: Final-window top-ups can suppress higher competing bids

- ID: HEX-020
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLAuctionHouse.topUpBid()`

Summary

A current highest bidder can raise the competing-bid floor in the auction's final window without extending the deadline, causing a pending higher bid to revert under favorable ordering. The incumbent can then win the NFT at lower gross consideration than under challenger-first ordering, while the challenger loses the acquisition opportunity.

Description

Expected behavior

Closing-price discovery should not allow the incumbent to invalidate an already-sufficient competing amount without leaving meaningful response time. Because highest-bidder top-ups are intentionally non-extending, a threshold-changing top-up should either be unavailable during the final window or preserve a response opportunity through another bounded mechanism.

Observed behavior

With an incumbent bid of 100 ETH, `TLAuctionHouse.getNextBid()` returns 105 ETH. At the deadline, a one-wei top-up changes the floor to 105 ETH plus two wei without changing the deadline. A pending 105 ETH bid then reverts, and `TLAuctionHouse.settleAuction()` awards the NFT one second later for 100 ETH plus one wei.

A buffered case behaves similarly: topping up from 100 ETH to 191 ETH raises the floor to 200.55 ETH, suppresses a 200 ETH bid, and permits settlement at 191 ETH. Under challenger-first ordering, the 105 ETH bid succeeds and extends the auction by 300 seconds.

Root cause

`TLAuctionHouse.topUpBid()` changes `highestBid` through the deadline without applying the response-window extension used for an ordinary late bid.

The contrasting `TLAuctionHouse.bid()` path recalculates the minimum from live state and applies the extension only after the incoming bid passes that check. `topUpBid()` accepts any positive incumbent increment and changes the value used by `TLAuctionHouse._calcNextBid()`, but performs no corresponding timing update. A top-up ordered first can therefore invalidate a pending quote before the ordinary bid reaches the extension logic.

Non-extension prevents an incumbent from holding an auction open through repeated small top-ups. The weakness arises from allowing that non-extending action to change the competing-bid threshold during the closing window.

Conditions and assumptions

The actor must already be the current highest bidder, although that role requires no owner or administrator authority and can be acquired through ordinary bidding. The actor must fund a suppressing total, observe or anticipate the competing amount, obtain earlier ordering in the final effective bid opportunity, and leave the challenger without another eligible inclusion opportunity.

Private order flow can prevent observation. Challenger-first inclusion, undisclosed bid slack, insufficient incumbent funding, or another eligible inclusion opportunity can prevent the terminal result.

Impact

When the required ordering conditions hold, the incumbent can win despite a funded higher competing transaction, and the auction can settle at lower gross consideration than the challenger-first control. The maximum difference approaches 4.7619% of the suppressed amount and may be material in absolute terms.

The challenger spends transaction gas and loses the acquisition opportunity but retains the bid principal when the call reverts. The percentage bound applies to gross consideration; net seller proceeds, protocol fees, and royalties depend on configuration, and royalty changes are not necessarily proportional.

Severity is Low because the incumbent must commit approximately 95.238% or more of the suppressed offer, accept the risk of winning and purchasing the NFT, secure favorable final-window ordering, and face the possibility of a retry. The effect is auction-bounded and can suppress a higher bid, but the trigger is costly, timing-dependent, and not broadly repeatable at low cost.

Recommendation

Reject `topUpBid()` during the final `EXTENSION_TIME` window or treat a final-window top-up as an ordinary bid increase that opens an equivalent response window. If bounded auction duration is required, pair the response-window rule with a hard maximum end and define cap behavior so a threshold-changing top-up cannot be accepted without response time.

Extending every one-unit top-up without an additional bound would reintroduce the documented risk that an incumbent can prolong the auction through repeated small top-ups.

Validation objective

Verify that no incumbent action can both raise the next-bid floor during the closing window and leave the response deadline unchanged. Cover exact-deadline behavior, buffered competing amounts, repeated minimum top-ups, any maximum-end rule, challenger-first extension, and legitimate top-ups outside the closing window.

HEX-021: Future-ID ERC-1155 drops can survive collection-authority replacement

- ID: HEX-021
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLStacks1155.configureDrop()`

Summary

`TLStacks1155` can preserve a drop configured for an ERC-1155 token ID that does not yet exist and later apply it after collection authority changes. If a successor creates that ID and reapproves Stacks, a buyer can mint the successor's token under the predecessor's supply and payout terms, diluting the successor's supply and displacing its primary-sale opportunity.

Description

Expected behavior

Collection authority changes should prevent a predecessor's pending instructions from becoming effective against a token created by the successor. A drop targeting a token that does not yet exist should either be rejected or require explicit authorization from the current collection authority before it can execute.

Observed behavior

`TLStacks1155` accepts a drop for a nonexistent token ID after checking that the configuring caller is the collection's current owner or administrator and that Stacks currently has the approved-minter role. The `_drops` mapping records only the collection address and token ID, while the accepted collection implementation assigns IDs sequentially in `ERC1155TL._createToken()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-1155/ERC1155TL.sol:465–480`), so an authorized predecessor can target the next predictable ID.

After an ownership transfer or administrator revocation, the predecessor loses authority; `revokeAllRoles()` (`dependencies/tl-creator-contracts-4.1.1/src/lib/OwnableAccessControlUpgradeable.sol:99–105`) also invalidates Stacks' role, but the historical drop remains. Once the successor creates the ID and reapproves the same Stacks contract, `purchase()` consumes the stored supply, allowance, price, duration, and payout receiver, and `externalMint()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-1155/ERC1155TL.sol:240–246`) recognizes Stacks' newly current role. The proof reproduced a revoked-administrator branch in which a free drop minted after only Stacks was reapproved, and an ownership-and-role-replacement branch in which a buyer received one token and the predecessor-selected receiver received the 1 ETH test price under zero-fee settings; the predecessor could no longer close either drop.

Root cause

`TLStacks1155.configureDrop()` validates authorization only when a drop is written, and permits the write to target a nonexistent token without binding it to a collection-authority generation.

The resulting record remains keyed only by collection address and token ID after authority changes. At purchase, `TLStacks1155` trusts that historical record, while the collection checks only Stacks' current aggregate minting role at the final mint call. Regranting that service role therefore reconnects predecessor-authored instructions to a token first created by the successor.

Conditions and assumptions

The planting actor must be the ERC1155TL collection owner or an `ADMIN_ROLE` administrator when configuring the drop; no permissionless initial write is demonstrated. The target must be a predictable future sequential ID, and the proof covers first creation of the next ID rather than ID reuse. That exact ID must later be created and remain unlocked, the drop must remain active with available supply and buyer allowance, and the same Stacks deployment must retain or regain `APPROVED_MINT_CONTRACT`.

If Stacks remains revoked or the token is locked, minting and settlement revert atomically. The current authority can remove a known record through `closeDrop()`; the indexed `DropConfigured` event and per-ID queries support detection, but there is no collection-wide on-chain active-drop enumeration. The scope is the accepted `TLStacks1155` and ERC1155TL integration; generic `externalMint` integrations were not tested.

Impact

A stale predecessor-authored drop can create unexpected units of a token first created by the successor and route sale proceeds to the predecessor-selected receiver. This can dilute the successor's intended supply, interfere with its sale control, and cause a lost primary-sale opportunity. In the zero-fee paid proof, the receiver obtained the full 1 ETH test price; with nonzero fees, `_settleUp()` sends the calculated net seller proceeds instead of necessarily the gross price.

The final severity is Low because the effect is confined to a collection and depends on an unusual chain of prerequisites: privileged future-ID configuration, authority replacement, later creation of the exact ID, reapproval of the same Stacks deployment, and an active, undiscovered drop that an eligible buyer can consume. Keeping Stacks revoked, locking the token, or closing the exact known drop prevents execution.

The evidence establishes the behavior in the accepted v2.8.0 code but does not establish that any current deployment contains a planted drop. It also does not establish affected deployment counts, token market value, aggregate loss, or reuse of previously created IDs.

Recommendation

Require an ERC1155TL token to exist before accepting its drop by checking `getTokenDetails(tokenId).created` (`dependencies/tl-creator-contracts-4.1.1/src/erc-1155/IERC1155TL.sol:14–26`). This prevents pre-creation drop state from attaching to a token created under successor authority.

If pre-creation setup or compatibility with collections that lack this query must remain, bind each pending drop to a collection-controlled authorization epoch and require explicit authorization from the current owner after ownership or role-generation replacement. Collection-wide enumeration or bulk invalidation would also help successors discover and retire inherited state. The existence check narrows pre-creation workflows and generic compatibility, while an epoch design preserves them at the cost of additional storage and lifecycle coordination.

Validation objective

Verify that drops for nonexistent IDs are either rejected or cannot execute after administrator revocation or ownership-and-role replacement, exact-ID creation, and Stacks reapproval until the successor authorizes that exact drop. Confirm that a blocked paid purchase neither mints tokens nor transfers seller proceeds, that explicit successor authorization restores intended operation, and that supported existing-token drops retain their expected lifecycle.

HEX-022: Velocity purchases can persist unrepresentable phase endpoints

- ID: HEX-022
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLStacks721._updateDropState()`; `TLStacks1155._updateDropState()`

Summary

An accepted duration update or successful positive-velocity purchase can leave an ERC-721 or ERC-1155 drop with a phase endpoint that overflows even though its stored duration remains representable. Subsequent phase queries, purchases, and phase-gated administrative updates then panic until an authorized NFT owner or administrator closes the drop, deleting its configuration and advancing the round.

Description

Expected behavior

Every successful drop configuration, duration update, and purchase should leave `startTime + presaleDuration + publicDuration` representable, so phase evaluation and administrative controls remain usable. A write that would cross this boundary should follow an explicit policy before any purchase effects become persistent.

Observed behavior

The ERC-721 phase helper and ERC-1155 phase helper construct the complete endpoint with checked additions. The purchase paths validate the old phase, then update ERC-721 state or update ERC-1155 state without validating the endpoint produced by the new duration; the ERC-721 and ERC-1155 duration setters likewise store replacement durations after evaluating only the old phase.

Paid two-token purchases in both implementations can start with a one-day duration and use a positive rate of `floor((type(uint256).max - 1 day) / 2)`. Each purchase settles and mints, leaves supply remaining, and stores `type(uint256).max - 1`; the next phase lookup, purchase, and decay-rate update then panic.

Paid one-token variants begin with an endpoint exactly at `type(uint256).max` and add one second, confirming the same transition at an ordinary quantity. The ERC-1155 variant confirms that every phase-dependent update and direct reconfiguration fails while a separate token ID remains usable. A separate ERC-1155 route uses a two-second adjustment, maximum supply and allowance, a zero-priced allowlisted ERC-20, and `floor((type(uint256).max - 86,400) / 2)` tokens accepted as one mint amount; it also stores `type(uint256).max - 1`.

Direct updates of REGULAR drops to the maximum duration reproduce the invalid endpoint in both managers. In every manifestation, `TLStacks721.closeDrop()` or `TLStacks1155.closeDrop()` remains available because closure does not evaluate the phase; it deletes the affected configuration and advances its round.

Root cause

`TLStacks721._updateDropState()` and `TLStacks1155._updateDropState()` preserve field-local arithmetic but not the cross-field phase-endpoint invariant.

The positive branches require the rate multiplication and stored-duration addition to fit, but do not require the resulting duration to fit after adding `startTime` and `presaleDuration`. The direct duration setters have the same invariant gap. A value can therefore be valid in storage while being unusable by the phase helpers.

The purchase ordering makes this state persistent: the contracts validate the old phase, update duration and supply, then settle and mint without validating the new endpoint. Once the endpoint is invalid, in-place repairs and replacement configuration first evaluate the malformed phase and panic; only the phase-independent close path bypasses that boundary.

Conditions and assumptions

The privileged path requires an NFT owner or administrator to give a REGULAR drop a representable duration whose complete endpoint overflows. Buyer-triggered VELOCITY paths require a configured adjustment whose multiplication and stored-duration addition fit while the complete endpoint does not, and the purchase must leave supply remaining; sellout returns `ENDED` before the invalid calculation.

Buyer variants include paid normal-quantity purchases after an extreme rate or near-boundary endpoint, plus a packaged ERC-1155 route using a two-second rate, maximum-scale supply and allowance, effectively free settlement, and a very large quantity accepted as one mint amount. A future start or presale can provide a repair window before evaluation reaches the invalid endpoint.

The claim covers the packaged `TLStacks721` and `TLStacks1155` implementations. An authorized NFT owner or administrator can call `closeDrop()` during pause or sanctions; replacement configuration remains subject to the usual pause and sanctions gates and advances the round.

Impact

An affected sale cannot answer phase queries, accept purchases, or use its phase-gated administrative setters. Recovery requires an authorized close and a new configuration; closure deletes the prior sale terms, advances the drop round, and makes transactions prepared for the old round stale.

The scope is one ERC-721 collection drop or one ERC-1155 collection/token-ID drop. Severity is Low because the prerequisite values are exceptional and an authorized close provides bounded recovery, although an otherwise eligible buyer can trigger the outage once those values exist and service remains unavailable until an administrator responds.

The triggering purchase settles and mints successfully, while later failed calls revert atomically. The evidence establishes no asset loss, trapped payment, privilege gain, permanent loss, or effect on unrelated drops.

Recommendation

Enforce complete phase-endpoint representability before every initial configuration, direct duration update, and positive velocity adjustment in both mint managers. Use guarded subtraction to validate the prefix and require `newPublicDuration <= type(uint256).max - startTime - presaleDuration` before mutating drop state, settling payment, or minting. Centralizing this check will keep the write paths consistent.

Choose and document one boundary policy. Rejecting an unsafe adjustment preserves the configured arithmetic but makes that purchase revert; saturating at the largest representable endpoint preserves phase liveness but changes extension semantics. A tightly authorized repair path that validates replacement fields without first evaluating the old phase can preserve a legacy round when closure would otherwise be required.

Validation objective

Add regression coverage for both token standards, direct REGULAR duration changes, paid normal-quantity VELOCITY purchases, and the large-quantity ERC-1155 route. Test the exact representable endpoint and the first value beyond it, verifying that an unsafe write either reverts before payment, minting, supply, allowance, or duration changes or produces an explicitly documented saturated state. Every successful transition should leave phase queries, later purchases, and administrative updates callable; retain coverage for sellout, future-start and presale behavior, round advancement, and isolation between drops.

HEX-023: Post-bid collection ownership changes can suppress configured royalties

- ID: HEX-023
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLAuctionHouse._isPrimarySale()`

Summary

When the default current-owner creator lookup is configured, transferring collection ownership to the seller after a binding bid can make settlement reclassify the auction as primary and suppress an independently configured royalty. The skipped share—up to the 10% cap—remains in the seller-selected payout instead of reaching the royalty beneficiary.

Description

Expected behavior

Once a bid makes a secondary auction binding, changing collection administrative ownership should not retroactively suppress an independently controlled royalty. Settlement should instead use a classification committed at that boundary or stable token-level creator provenance. The project describes configured royalties as applying to secondary sales (`README.md:23–34`).

Observed behavior

The reproduction creates a 10 ETH reserve auction with an immutable 10% ERC-2981 royalty for a recipient distinct from the collection owner and seller. When the first bid becomes binding, the sale is secondary and quotes a 1 ETH royalty. The seller cannot delist the token after bidding, consistent with the post-bid listing restrictions.

The collection owner then transfers ownership to the seller without changing the royalty recipient or schedule. The default lookup begins reporting the seller as creator, and the packaged edge test likewise expects this ownership change to alter the classification. After the auction ends, an unrelated account settles it: the royalty recipient receives nothing, the seller receives the full 10 ETH bid, and the bidder receives the NFT.

Root cause

`TLAuctionHouse._isPrimarySale()` derives the primary-sale decision from mutable collection ownership at settlement instead of binding it when the auction becomes funded.

The stored `Listing` retains the seller, bidder, and bid amount but no committed creator identity or sale classification. The default `CreatorLookup.getCreator()` ignores `tokenId` and returns the collection's current owner. Once that address equals the stored seller, `TLAuctionHouse._completeSale()` skips royalty lookup and payment, leaving the otherwise quoted share in the seller-selected payout.

Conditions and assumptions

The path requires the default current-owner `CreatorLookup` to remain configured through settlement, a seller distinct from the collection owner when the bid binds, a nonzero royalty independent of collection ownership, and an ownable collection whose owner can transfer ownership to the seller before settlement. The transfer requires authorized or compromised collection-owner authority; no auction-house administrator privilege is required. Economic benefit ordinarily requires cooperation or common control with the seller.

The path is most relevant to immutable royalty arrangements on supported generic ownable collections, including collaborator beneficiaries with separate authority. In the packaged `ERC721TL` implementation (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:297–311`), the collection owner can already alter royalties directly, so this path adds less incremental authority.

Current-owner classification is documented. The issue is its effect on an already funded auction and an independent beneficiary, not an outsider ownership bypass, an accidental implementation discrepancy, or a violation of a published bid-time immutability guarantee.

Impact

A qualifying royalty beneficiary can lose up to the 10% royalty cap, with the skipped amount remaining in the seller-selected payout. The reproduction demonstrates a 1 ETH loss on a 10 ETH auction. The bidder still receives the NFT and pays no additional principal.

Severity is Low. The loss can be material within one sale, but the path requires current collection-owner authority, the default lookup, an independent royalty, and a post-bid transfer of collection ownership to the seller; the effect is bounded to qualifying sales.

The evidence directly covers a supported generic ownable integration, not the packaged mutable-royalty collection implementation. It does not establish a current production configuration or aggregate loss; broader exposure depends on deployment configuration and owner-seller alignment. The finding concerns configured on-chain royalty behavior and does not establish any legal or off-chain entitlement.

Recommendation

Bind a token-aware creator identity or a `royaltyApplicable` primary/secondary flag when the first bid makes an auction binding, then use that committed value during settlement. Prefer stable token-level creator provenance where the integration supports it.

This approach adds per-listing storage and gas cost and requires an explicit policy for auctions that are primary when the commitment is created. Snapshotting only royalty applicability would not freeze later changes to royalty amounts, recipients, or lookup providers; those are separate control decisions. If settlement-time classification remains intentional, document its effect on funded auctions and independent royalty beneficiaries.

Validation objective

Verify that a secondary auction with a nonzero independent royalty still pays that royalty after collection ownership moves to the seller, and that its committed classification cannot change between the first bid and settlement. Also verify that a sale committed as legitimately primary retains its intended treatment and that ordinary secondary settlement without an ownership transfer remains unchanged.

HEX-024: Unmetered NFT lookups can exhaust auction-settlement gas

- ID: HEX-024
- Status: New
- Class: Finding
- Severity: Low
- Location: TLAuctionHouse._completeSale()

Summary

An NFT can consume nearly all available gas during optional creator and royalty lookups, causing an otherwise valid auction settlement to revert. This can leave the winning bid and NFT in escrow and impose failed transaction costs; without marketplace-owner intervention, cancellation remains unavailable until strictly more than seven days after expiry.

Description

Expected behavior

Creator and royalty lookups are informational: a failed creator lookup should classify the sale as secondary, while a failed royalty lookup should leave the sale value with the seller. If the NFT's ordinary transfer functions remain usable, lookup failures should not consume the gas needed for delivery and settlement accounting.

Observed behavior

TLAuctionHouse._completeSale() resolves the creator and royalties before delivering the NFT. CreatorLookup.getCreator() forwards gas to the NFT's `owner()` without an explicit limit; if the call exhausts its gas, the adapter catches the failure and returns the zero address, classifying the sale as secondary and entering the royalty path. RoyaltyLookup._getERC2981RoyaltyInfo() then calls `royaltyInfo()` without an explicit limit, so a second caught failure can leave too little gas for the transfer.

The reproduction first settled a control auction through both adapters. After changing only the NFT's lookup behavior, it confirmed that both callbacks were reached and that settlement failed with a 30,000,000-gas budget. The revert preserved the listing, winning bid, NFT custody, and 1 ETH escrow; cancellation failed at exactly auction end plus seven days, then succeeded one second later, allowing the bidder and seller to recover their respective assets while the lookup gas burn remained active. This isolates the failure from the NFT transfer itself and is consistent with the documented seven-day hostile-asset fallback (`README.md:46–49`).

Root cause

`TLAuctionHouse._completeSale()` invokes the optional creator and royalty adapters without reserving gas for the mandatory NFT transfer and settlement accounting that follow.

Each NFT lookup crosses two EIP-150 call boundaries, allowing a nested gas-consuming callback to use approximately 97% of the caller's current gas even when the failure is caught. Repeating this pattern sequentially can leave roughly 0.1% before overhead; selecting a fallback value does not restore the gas already consumed.

The native-currency payout path uses an explicit gas stipend to protect the remaining settlement work, but no analogous reserve protects NFT delivery from the informational lookups.

Conditions and assumptions

Both optional lookup modules, which the constructor leaves unset, must be configured. The demonstrated path uses the supplied creator adapter and a royalty adapter with no configured royalty engine, causing its ERC-2981 fallback to call a seller-controlled or mutable NFT. No auction-house privilege is required: the seller controls an NFT whose `owner()` and `royaltyInfo()` burn gas while `transferFrom()` and `ownerOf()` remain functional, and a bidder participates in the scheduled auction.

Applicability also requires a usable or economically practical transaction-gas ceiling below the build-specific completion threshold and no earlier marketplace-owner intervention through the global lookup setters. The demonstrated persistent delay applies to a native-currency scheduled auction. Fixed-price purchases reach the same lookup sequence but revert atomically without the demonstrated persistent bid lock.

Impact

At a 30,000,000-gas budget, settlement reverts and callers incur the failed transaction cost. Without owner intervention, the demonstrated auction keeps a 1 ETH winning bid and the NFT unavailable until cancellation becomes possible strictly more than seven days after expiry, delaying the bidder's use of its capital and preventing auction finality during that period.

The substantial prerequisites and temporary, recoverable, auction-scoped disruption support a Low rating. No seller proceeds are released on failure; the bidder's full principal and the seller's NFT remain recoverable through delayed cancellation and claims. The evidence does not demonstrate theft, permanent lock, or realized monetary loss.

Recommendation

Enforce a deterministic settlement-gas reserve at the auction-house-to-adapter boundary for every optional creator and royalty lookup. Cap the gas forwarded and return data processed, then apply the documented creator or royalty fallback when the budget is exceeded. Limiting only the NFT-facing calls would not protect settlement from a hostile replacement adapter.

Choose budgets that preserve enough gas for NFT delivery, ownership verification, and settlement accounting under supported runtime configurations. Because a limit that is too low may skip legitimate complex royalty engines, validate representative integrations and consider a narrowly scoped per-collection or per-auction bypass for operational recovery. Retain the seven-day fallback because arbitrary NFTs can still fail their transfer semantics independently.

Validation objective

With `transferFrom()` and `ownerOf()` functioning normally, simultaneous gas exhaustion in `owner()` and `royaltyInfo()` must not prevent settlement at the supported transaction budget. Regression tests should cover individual and dual gas-consuming callbacks, malformed and reverting responses, valid high-cost adapters, configured and unset royalty engines, preserved fallback behavior, and sufficient residual gas for delivery and accounting.

HEX-025: Sanctions-oracle errors can bypass failed-payout recovery

- ID: HEX-025
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLAuctionHouse._payout()`

Summary

A payout-recipient sanctions-oracle error can make an otherwise valid Auction House settlement revert before fallback credit is recorded, leaving the winning bid and NFT escrowed. Settlement remains unavailable until the oracle condition or payout receiver changes, or the auction becomes eligible for delayed cancellation.

Description

Expected behavior

`TLAuctionHouse._payout()` is documented as never reverting. When sanctions screening rejects a recipient or a direct transfer fails, the function should record an internal credit so the enclosing settlement can complete without releasing funds to that recipient.

Observed behavior

After `TLAuctionHouse.settleAuction()` successfully checks the seller and winner, `TLAuctionHouse._completeSale()` routes protocol fees, royalties, and seller-selected proceeds through `_payout()`. Each call performs another sanctions-oracle query; if one of these later lookups reverts, the error escapes before the transfer attempt or `_credit` fallback.

The native-currency reserve-auction proof makes only the seller-selected payout receiver's query revert. A permissionless settlement call returns the exact oracle error, records no claimable credit, and atomically restores the active listing, escrowed NFT, and winning bid. Changing the response for the same receiver from an error to sanctioned (`true`) allows settlement to complete and credits the proceeds.

Source review also shows that `TLAuctionHouse.bid()` uses `_payout()` for previous-bidder refunds. The same uncaught lookup can therefore disrupt that branch as well as the fee and royalty payout legs, although those manifestations are source-inferred rather than directly exercised.

Root cause

`TLAuctionHouse._payout()` performs an unguarded sanctions lookup before its transfer-or-credit fallback.

`SanctionsCompliance._isSanctioned()` directly calls and ABI-decodes the configured oracle without handling a revert, malformed response, or unavailable contract. Because `_payout()` invokes it before the bounded transfer attempt or `_credit`, these failures escape and atomic rollback unwinds the settlement.

Conditions and assumptions

The configured oracle must allow the earlier seller and winning-bidder checks to succeed but fail during a later payout-recipient lookup. The failure may be recipient-specific, call-context-specific, or related to gas available later in execution; a uniform outage ordinarily fails an earlier entry-point check.

Once this oracle condition exists, any caller can trigger the failure through ordinary settlement without a specialized sequence or race, but the evidence does not establish a permissionless way to create the condition. Recovery depends on the relevant authorized or affected party: the protocol owner can replace or disable the oracle, a cooperative seller can change the payout receiver, and the seller or winner can cancel an unsettled auction after the seven-day grace period.

Impact

An affected completed auction cannot settle while the qualifying lookup failure persists. The winning bid and NFT remain escrowed, delaying the seller's and winner's access to assets of potentially arbitrary value and potentially causing the sale to be abandoned.

Complete rollback preserves custody and limits the supported harm to recoverable unavailability. The Low rating reflects the unusual selective dependency failure and the available repair or delayed-unwind paths that bound the disruption.

Recommendation

Give `_payout()` a bounded, fault-aware sanctions lookup that distinguishes an explicit unsanctioned response from a sanctioned response and lookup failure. Attempt a transfer only after an explicit unsanctioned response; for a sanctioned response, oracle revert, malformed response, missing code, or gas-bounded timeout, record the amount as claimable credit while retaining enough gas to update that credit.

This approach keeps release fail-closed during oracle uncertainty, but defers immediate payment into a claimable obligation that remains subject to the existing sanctions checks.

Validation objective

Verify that selective reverts, malformed responses, codeless endpoints, and gas-consuming lookups create the correct credit without reverting the enclosing payout operation. Cover sanctioned and unsanctioned controls, native and ERC-20 funds, previous-bidder refunds, protocol fees, royalty payments, and seller proceeds while preserving escrow-accounting invariants and sanctions checks on claims.

HEX-026: Recipient-restricted bidders can force the seven-day settlement-failure path

- ID: HEX-026
- Status: New
- Class: Finding
- Severity: Low
- Location: TLAuctionHouse.bid()

Summary

A fully funded bidder whom the NFT policy rejects as a recipient can become the winner, causing settlement to revert and keeping both the seller's NFT and the winning bid escrowed until cancellation opens just over seven days after auction end. Bid acceptance fixes the bidder as the delivery recipient without checking deliverability or providing an alternate-recipient resolution.

Description

Expected behavior

A bidder should not be able to choose a predictably undeliverable settlement recipient and force the seller and both escrowed assets through the full failed-settlement grace period. Recovery should preserve auction finality and escrow solvency while providing a bounded response to delivery failure.

Observed behavior

During listing, `TLAuctionHouse._list()` transfers the NFT to the auction house, which the recipient policy permits. `TLAuctionHouse.bid()` then accepts full payment and records the bidder as both `highestBidder` and `recipient` without consulting the NFT or its validator; after a bid exists, `TLAuctionHouse.delist()` prevents the seller from withdrawing the listing.

After the auction closes, replacement bids are rejected and `TLAuctionHouse.settleAuction()` directs delivery to the highest bidder. `TLAuctionHouse._completeSale()` attempts the NFT transfer before releasing funds, so a recipient-policy revert rolls back the complete settlement and leaves the listing, NFT, and bid in escrow. `TLAuctionHouse.cancelUnsettledAuction()` remains unavailable at auction end plus 604,800 seconds and opens one second later; under the demonstrated policy, the bidder and seller can then claim their original assets.

Root cause

`TLAuctionHouse.bid()` binds an accepted bidder as the mandatory NFT recipient without checking whether the escrowed NFT can be delivered to that address.

The settlement design provides neither an alternate-recipient mechanism nor a recorded delivery-failure state from which an earlier safe unwind can proceed. A policy can permit transfer into escrow yet reject transfer from escrow to the winner—for ERC721TL, `ERC721TL._update()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:428–`

439) invokes the configured validator for applicable transfers—and the atomic revert leaves the generic seven-day cancellation path as the only demonstrated resolution. This makes the fallback selectable by the bidder rather than limited to unexpected or transient settlement failures.

Conditions and assumptions

An affected ERC-721 must use a recipient-sensitive policy that permits transfer into auction-house escrow but rejects the bidder. For ERC721TL, this requires a configured validator and positive effective royalty. Callback-only rejection is outside this policy shape because `TLAuctionHouse._transferNft()` uses `transferFrom`; an operator-only restriction generally cannot distinguish the transfers because the auction house operates both.

The bidder must pass the sanctions check, fully fund a valid bid, and remain highest through closing. A valid pre-close outbid prevents the condition, while a collection owner or administrator can restore delivery through privileged validator or royalty changes (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:300–310`). Recovery also requires an allowed later destination for the seller's NFT claim.

The bidder needs no protocol privilege but locks the full winning bid and bears its opportunity cost. The seller-selected auction duration is ordinary auction time; only the following grace period is additional delay attributable to this condition.

Impact

The demonstrated harm is one failed sale and continued custody of the NFT and full winning bid for seven additional days after the auction ends. During that period, the seller cannot use or relist the NFT and the bidder cannot use the escrowed funds. Lost market opportunity and NFT price movement are credible consequences, but their monetary value is inferred and unquantified.

Once cancellation opens, the bid is credited to the bidder and the NFT to the seller; under the demonstrated policy, both parties claim their original assets. The evidence therefore does not establish theft, permanent principal loss, or protocol-wide insolvency.

Low severity reflects that the behavior remains within the documented settlement-and-recovery flow, affects one auction, requires the bidder to escrow the complete winning amount, and returns both principals after a bounded delay. The evidence does not establish cheap repetition or harm materially greater than that disclosed recovery path.

Recommendation

Add a settlement-recovery path that prevents a predictably undeliverable winner from forcing the full generic grace period while preserving auction finality and escrow solvency. One option is to let the winner designate or update an alternate nonzero, unsanctioned recipient after close, then record a failed-delivery state and permit an earlier or materially shorter safe unwind if delivery still fails.

A bid-time check for recognized validator interfaces can reduce predictably invalid bids, but it should not be the only control because arbitrary ERC-721 transfer behavior and mutable policies can change between bidding and settlement. Any early-unwind design should prevent a winner from escaping an otherwise valid bid or a seller from cancelling an otherwise settleable auction by manufacturing a failed attempt.

Validation objective

Verify that recipient-sensitive NFTs can still enter escrow; a restricted winning recipient cannot hold both principals through the full generic grace period once delivery failure is established; failed delivery never releases sale funds; retry to an allowed recipient and the selected unwind path behave correctly; neither party can manufacture failure to evade a valid

auction; and sanctions checks, cancellation boundaries, claims, and escrow solvency remain correct.

HEX-027: Privileged token transfers can break bid-backed auction custody

- ID: HEX-027
- Status: New
- Class: Finding
- Severity: Low
- Location: TLAuctionHouse._list()

Summary

A privileged TRACE collection actor can remove an NFT from auction escrow after a bid, causing settlement to revert. The bidder can recover the full principal only after the auction ends and the seven-day grace period expires, while a seller aligned with that actor can retain the NFT and avoid the sale.

Description

Expected behavior

After `TLAuctionHouse` accepts a bid, the auctioned NFT should remain under the auction house's exclusive control until permissionless settlement or a protocol-managed recovery path completes. Collection-level recovery authority should not be able to revoke custody and invalidate an already funded auction.

Observed behavior

Two equivalent TRACE auctions each accepted a 10 ETH bid. After the TRACE administrator transferred one token from the auction house back to the seller, the untouched auction settled normally, but the affected `settleAuction()` call reverted with `ERC721IncorrectOwner`; atomic rollback preserved the listing and bid.

After the auction ended and the seven-day grace period elapsed, the bidder used `cancelUnsettledAuction()` and withdrew the complete 10 ETH principal. The seller retained the NFT but received no proceeds from the canceled auction.

Root cause

`TLAuctionHouse._list()` accepts custody from any ERC-721 without ensuring that the auction house will retain exclusive transfer control after a bid.

The listing path checks ownership and transfers the NFT into escrow once, consistent with the package's stated support for any compliant ERC-721 (`README.md:23–24`), but it does not enforce compatibility with collection-level transfer powers. TRACE allows its owner or an `ADMIN_ROLE` member to call `transferToken()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/trace/TRACE.sol:218–225`), which invokes an internal `_transfer()` implementation (`dependencies/@openzeppelin-contracts-upgradeable-5.6.1/token/ERC721/ERC721Upgradeable.sol:337–357`) without a caller-authorization check. Supplying the auction house as the current owner therefore lets the privileged actor move the escrowed token without auction-house approval.

The cancellation path safely returns the bid principal, but it cannot restore the binding auction outcome. Because TRACE documents this authority as a chip-recovery facility (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/trace/ITRACE.sol:51–57`), the defect is an integration and trust-boundary mismatch rather than a TRACE access-control bypass.

Conditions and assumptions

The path requires a TRACE token with the tested transfer behavior, a live bid, and a collection owner or `ADMIN_ROLE` member willing to remove the NFT before settlement. For the seller to gain a strategic cancellation option, the seller must control or coordinate with that actor; an unrelated administrator can disrupt settlement without necessarily benefiting the seller.

The token must remain outside escrow until the cancellation time gate. NFTs without a comparable privileged transfer mechanism are unaffected.

Impact

The bidder loses the auction outcome and cannot receive the NFT. The bidder's principal remains unavailable until the auction ends and the additional seven-day grace period expires, creating gas, financing, opportunity, and potential lost-acquisition costs.

If the seller controls or coordinates with the privileged TRACE actor, the seller can conditionally allow favorable auctions to settle and remove the NFT from unfavorable ones. This non-binding option is limited to the affected auction and depends on token value and market movement.

Severity is Low because settlement reverts atomically, the bidder can recover the complete principal within the documented recovery flow, the canceled auction pays no proceeds to the seller, and the trigger requires privileged token authority aligned against the bidder. The proof does not demonstrate theft of bidder principal.

Recommendation

Enforce an explicit escrow-compatibility boundary before accepting bid funds. Restrict bid-bearing listings to approved collections whose privileged transfer paths cannot remove tokens from auction escrow, or make TRACE recovery escrow-aware so an active auction token cannot move without auction-house authorization.

A custody check at bidding or settlement can detect that the NFT has left, but detection alone does not restore auction finality. If broad collection support is retained, add an immediate lost-custody refund path as defense in depth. Preserve legitimate chip recovery through an escrow-aware exception or coordinated release process where required.

Validation objective

Verify before bidder funds are accepted that a TRACE owner or administrator either cannot remove a bid-backed token from auction escrow or cannot list that collection. Confirm that an untouched auction still settles normally and, if intended, that legitimate TRACE recovery remains available outside active escrow.

HEX-028: Malformed dependency responses can block auction settlement

- ID: HEX-028
- Status: New
- Class: Finding
- Severity: Low
- Location: TLAuctionHouse._isPrimarySale(), TLAuctionHouse._quoteRoyalties()

Summary

A directly configured creator lookup that successfully returns malformed ABI data can block every sale completion, while the same failure from a direct royalty lookup can block quotation and secondary-sale settlement. For an already-bid auction, the NFT and highest bid remain escrowed until the owner repairs the dependency or a participant cancels strictly more than seven days after the auction ends.

Description

Expected behavior

An unset or unusable creator lookup should make `TLAuctionHouse._isPrimarySale()` return `false`, conservatively classifying the transaction as a secondary sale so completion can continue. A royalty provider that cannot produce a usable schedule should similarly yield an empty royalty schedule through `TLAuctionHouse.getRoyalty()` and `TLAuctionHouse._quoteRoyalties()`, allowing quotation and secondary-sale settlement to continue.

Observed behavior

An ordinary creator-lookup revert enters the fallback, but a one-byte successful response fails caller-side address decoding: public classification and settlement both revert. Because every sale completion reaches classification through `TLAuctionHouse._completeSale()`, persistent malformed behavior can make completion unavailable across the affected auction-house instance.

On the royalty path, an ordinary provider revert produces an empty schedule, while a successful malformed response causes `getRoyalty` and secondary-auction settlement to revert during decoding. The supplied royalty adapter converts the tested malformed inner-engine response into an outer revert that the auction house catches, so this manifestation was established only for malformed success returned directly to the auction house.

In each auction manifestation, the failed settlement is atomic: the listing stays active, the NFT remains with the auction house, and the highest bid remains escrowed. The owner can restore settlement by repairing or clearing the lookup. Without owner repair, the unsettled-auction recovery path rejects cancellation at the exact seven-day grace boundary and permits it one second later, after which the bidder can claim the bid and the seller can claim the NFT.

Root cause

The typed external calls in `TLAuctionHouse._isPrimarySale()`, `TLAuctionHouse.getRoyalty()`, and `TLAuctionHouse._quoteRoyalties()` do not safely handle successful responses whose return data is malformed.

Each path relies on a `catch` clause to apply its fail-secondary or fail-open fallback. An ordinary provider revert reaches that handler, but a successful malformed response instead fails when Solidity decodes the return data in the caller before entering the `returns` body, bypassing the handler. The creator call expects an ABI-encoded address, while the royalty call expects two ABI-encoded dynamic arrays; the creator path's preceding code-length check establishes only that the target has code, not that its response is valid.

Conditions and assumptions

Only the auction-house owner can select lookups through `TLAuctionHouse.setCreatorLookup()` and `TLAuctionHouse.setRoyaltyLookup()`. The trigger requires a code-bearing provider that reports success with malformed ABI data. The creator manifestation can affect every sale type, whereas the royalty manifestation requires a secondary sale; persistent escrow requires an already-bid auction awaiting settlement.

The creator adapter and royalty adapter revert on the malformed inner responses addressed by the evidence, allowing the auction house's outer catch to apply. These manifestations therefore require malformed success to reach `TLAuctionHouse` directly.

The evidence does not establish a deployed provider, proxy arrangement, or unprivileged response-control path. A previously valid mutable provider could later return malformed data, but applicability depends on the deployment-specific provider and its control model.

Impact

An ended auction can remain unable to settle while the malformed response persists, temporarily denying the seller access to the NFT and the bidder access to the highest bid. If neither the owner nor the provider restores normal behavior, participant-directed recovery becomes available only strictly more than seven days after the auction ends and cancels the intended sale instead of completing it.

Because the lookups are global, the creator condition can disrupt every sale completion on an affected instance; the royalty condition can disrupt quotations and multiple secondary sales, including atomically reverting secondary fixed-price purchases. The proofs directly exercised one active auction for each manifestation; the broader availability scope follows from the shared call paths.

This issue is rated Low. Owner configuration is the only demonstrated trigger, no unprivileged response control is established, failed transactions roll back atomically, the owner can repair the dependency immediately, and auction participants can recover after the grace period. No theft, partial transfer, incorrect payout, asset destruction, or permanent loss was demonstrated.

Recommendation

Handle creator and royalty lookups through bounded low-level `staticcall` logic that checks call success and validates return-data structure before decoding. Treat an unusable creator response as `false` and an unusable royalty response as an empty schedule, matching the existing fallback policies. For creator responses, validate canonical address encoding; for royalty responses, safely validate the dynamic offsets, lengths, and two-array shape. An externally catchable decoding helper or restriction to a trusted immutable adapter are alternatives.

A setter-time probe alone does not address a lookup that changes after configuration. Define whether trailing creator return data is accepted to avoid an unintended compatibility change. Failing open on a malformed royalty response preserves settlement availability but may omit royalties while the dependency is unhealthy; if royalty preservation should take priority, document and implement that policy consistently.

Validation objective

Regression tests should cover unset and codeless targets, ordinary reverts, empty and truncated successful responses, noncanonical creator words, malformed royalty offsets and lengths, mismatched royalty arrays, valid responses, and the chosen trailing-data policy through both direct providers and the supplied adapters. Verify creator classification, both royalty quote methods, auction settlement, fixed-price purchases, primary-sale royalty bypass, owner replacement, and cancellation rejection at the exact grace boundary followed by availability immediately afterward. Malformed responses should follow the selected fallbacks without changing valid-response behavior or allowing partial state changes.

HEX-029: Mint settlement can pay a fee receiver after it becomes sanctioned

- ID: HEX-029
- Status: New
- Class: Finding
- Severity: Low
- Location: TLStacks721._settleUp() · TLStacks1155._settleUp()

Summary

TLStacks721 and TLStacks1155 can continue paying a stored protocol-fee receiver after the active sanctions oracle flags the address. An eligible buyer can therefore complete a mint and send configured fees to a receiver that the contracts would reject if configured again.

Description

Expected behavior

When sanctions checking is enabled, each mint contract should apply the active oracle's current decision before sending a nonzero settlement payment to `protocolFeeReceiver`. If the receiver is flagged, settlement should either revert or withhold the affected amount under a defined recovery policy until an eligible receiver is configured.

Observed behavior

The owner-only configuration helpers query the current oracle and reject a flagged receiver when fee settings are written (TLStacks721, TLStacks1155). Purchase validation then checks the payer, NFT recipient, and drop payout receiver, but omits `protocolFeeReceiver` (TLStacks721, TLStacks1155).

The settlement helpers transfer native protocol and seller fees, or ERC-20 seller fees, to the stored receiver without another sanctions query (TLStacks721, TLStacks1155). In the reproductions, a receiver was accepted while clear and later flagged. Ordinary ETH public-sale purchases still minted one token and paid the exact configured protocol fee; the contract-specific variants measured a `0.01 ether` increase at the receiver while fresh fee settings rejected the same address. The ERC-1155-specific proof also showed that a flagged drop payout receiver remains dynamically rejected.

Root cause

`TLStacks721._settleUp()` and `TLStacks1155._settleUp()` treat the configuration-time approval of `protocolFeeReceiver` as durable and transfer fees without revalidating its current oracle status.

The `TLStacks721._setProtocolFeeSettings()` and `TLStacks1155._setProtocolFeeSettings()` helpers evaluate a mutable predicate only when settings change. Owner control over the stored address does not keep the oracle's answer immutable, so a later status transition leaves the settlement paths using stale eligibility even though purchases dynamically check the other participants.

Conditions and assumptions

The condition requires an enabled oracle that initially clears the receiver and later flags it, whether through a status transition or an oracle replacement. The stored receiver must be distinct from the payer, NFT recipient, and drop payout receiver, which are rechecked. An unpaused active drop, a nonzero transferable fee, otherwise eligible participants, and no prior owner pause, fee removal, or receiver rotation are also required.

Native payments require a transfer-capable receiver; ERC-20 applicability requires a supported transfer-capable currency and a nonzero, non-waived seller fee. A frozen oracle with no relevant status transition does not independently create the stale state.

This finding is scoped to the accepted Stacks v2.8.0 package. Applicability to other versions or products requires separate evidence.

Impact

Once these conditions hold, an eligible buyer needs no privileged role to trigger direct payment of configured mint fees to a receiver that the active oracle flags. Each contract-specific proof transferred exactly `0.01 ether` in its covered purchase, and further otherwise-valid purchases can repeat the payment until the owner intervenes; the cumulative amount depends on configured fees and purchase activity.

The harm is limited to legitimate configured fee flows: the evidence does not show unauthorized fee changes, buyer overcharging, theft, privilege escalation, or permanent asset loss. The owner can stop subsequent payments by pausing or rotating the receiver. The uncommon clear-to-flagged transition, bounded fee flow, and available intervention support Low severity.

Recommendation

Check the exact stored `protocolFeeReceiver` against the current oracle immediately before every nonzero native or ERC-20 transfer to that address in each contract's shared settlement helper. Also revalidate the existing receiver whenever the owner changes the oracle. Define one explicit response for a newly flagged receiver: either revert settlement atomically or retain or credit the affected fee under accounting that prevents payment while the receiver remains flagged.

A live oracle call adds gas and external-availability dependence. Reverting can halt affected mints until owner intervention; retaining or crediting fees can preserve mint availability but requires explicit custody, accounting, release, and recovery rules.

Validation objective

Configure a receiver while clear, flag it before purchase, and verify that both contracts follow the selected safe behavior without increasing the receiver's balance. Cover `TLStacks721.purchase()`, `TLStacks721.purchaseSelected()`, `TLStacks1155.purchase()`, native flat protocol fees, native seller fees, and ERC-20 seller fees. Confirm that clear receivers remain payable, an oracle change cannot silently leave an ineligible receiver active, and zero-fee purchases are not unnecessarily blocked; for a reverting design, verify that no mint occurs, and for a holding design, verify that funds cannot be released while the current oracle still flags the receiver.

HEX-030: Selectable drops are incompatible with Editions metadata semantics

- ID: HEX-030
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLStacks721.configureDrop()`

Summary

When a `TLStacks721` selectable drop targets `ERC7160TLEditions`, a paid purchase can settle without displaying the buyer's selected URI; separately, a holder can make their token display another token's already-consumed selection. The integration therefore fails to preserve buyer choice and piece-level metadata exclusivity.

Description

Expected behavior

Selectable drops should bind each purchased index to the token minted for that purchase. The drop definition maps index `i` to `baseUri/i`, and the feature documentation (`README.md:11–21`) states that each piece can be minted only once per round. If a target cannot atomically establish and preserve that association against ordinary holders, configuration or purchase should revert before payment or selectable state persists.

Observed behavior

The `purchaseSelected()` path records each choice, consumes sale supply and buyer allowance, settles payment, and invokes `_mintSelectedTokens()` with `baseUri/index`. For Editions, `externalMint()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/multi-metadata/ERC7160TLEditions.sol:286–301`) ignores the supplied URI and mints an unpinned token whose `tokenURI()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/multi-metadata/ERC7160TLEditions.sol:431–439`) uses the collection fallback. With aligned global URIs, a 1 ether purchase of index 2 transferred payment and consumed drop supply, buyer allowance, and the index bitmap, yet the minted token remained unpinned at index 0 and returned `ipfs://selected/0` instead of `ipfs://selected/2`.

In a separate post-mint manifestation, two buyers consumed indices 7 and 8 and their tokens were first pinned to their respective selections. Another purchase of index 7 reverted because the `TLStacks721` bitmap still marked it as consumed; however, the owner of token 2 could call `pinTokenURI(2, 7)` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/multi-metadata/ERC7160TLEditions.sol:383–439`). Both tokens then returned `ipfs://drop/7` while bitmap entries 7 and 8 remained consumed. The holder neither modified token 1 nor bypassed NFT ownership.

Root cause

`TLStacks721.configureDrop()` admits selectable targets without requiring token-specific, persistent index-to-token URI semantics.

Stacks records only per-NFT, per-round index consumption in `_markIndicesMinted()` and passes the generated URI through a common mint interface. It neither retains the index-to-token assignment nor verifies the token's resulting metadata. Interface support does not establish this capability because Editions advertises the interface (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/multi-metadata/ERC7160TLEditions.sol:559–571`) even though its mint implementation ignores the URI argument.

Editions exposes one collection-wide URI array to every token, and its pinning logic (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/multi-metadata/ERC7160TLEditions.sol:383–439`) lets a token owner or delegate select any in-bounds global index for that token. The displayed metadata can therefore diverge from Stacks' consumption bitmap both at mint time and afterward.

Conditions and assumptions

An NFT owner or `ADMIN_ROLE` holder must preload distinct Editions URIs aligned with selectable indices, authorize Stacks as a minter, and configure a selectable drop. The mint-time mismatch requires a nonfallback selection; the fallback is index 0 by default but may be changed to the last global URI. Post-mint reuse requires at least two minted tokens and ownership or delegation over the token being repinned.

The condition is limited to selectable drops paired with `ERC7160TLEditions`; ordinary `ERC721TL.externalMint()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:253–262`) stores the supplied URI per token.

When the selected URI is preloaded, the holder can later pin it, so the first manifestation is an atomic selection failure rather than permanent inability to display that URI. The documentation does not separately promise immutable metadata; the second manifestation concerns an ordinary holder defeating once-per-round piece identity, not a collection administrator changing metadata.

Impact

A buyer can complete a paid sale and receive an NFT while the selected metadata is not applied even though payment and sale accounting commit. Separately, an authorized holder can make two independently owned tokens display the same URI after different selectable pieces were sold, undermining metadata choice and piece-level exclusivity.

Severity is Low because the demonstrated harm is confined to metadata display and sale-term integrity. It does not establish asset loss, duplicate ERC-721 ownership, control over another holder's token, a bitmap bypass, a quantified selected-piece value difference, or permanent inability to display the purchased URI.

Potential economic loss, refund burden, rarity or buyer-value dilution, and marketplace or reputational harm depend on configuration and valuation. The mint-time mismatch may be corrected through later holder pinning when the URI is preloaded, while reuse requires ownership or delegation over the repinned token.

Recommendation

Make `TLStacks721.configureDrop()` admit selectable mode only for targets with an explicit capability to atomically apply `baseUri/index` to the new token and preserve that index-to-token association against ordinary holder changes. Reject `ERC7160TLEditions` targets that cannot provide those semantics before sale configuration; generic `IERC721TL` or ERC-7160 interface checks are insufficient.

If Editions must support selectable drops, use a dedicated mint path or adapter that initializes the selected global index and prevents an owner or delegate from repinning a token to an index assigned elsewhere. Alternatively, separate or disallow the selectable mode while preserving Editions' normal holder-choice behavior. An implementation allowlist is simpler but narrows generic compatibility, while a post-mint URI equality check may reject legitimate dynamic renderers.

Validation objective

Verify that every selectable target either rejects incompatible configuration or immediately displays each purchased nonfallback URI and keeps it bound to the corresponding token. Payment, supply, allowance, and bitmap changes should roll back if the association cannot be established. With two distinct selections, an owner or delegate must not be able to make the second token display the first token's consumed URI; regression coverage should preserve ordinary `ERC721TL` selectable behavior, non-selectable Editions pinning, transfers, delegated ownership behavior, and round isolation.

HEX-031: Round-blind access and timing updates can alter successor-drop allocations

- ID: HEX-031
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLStacks721.updateDropPresaleMerkleRoot()`, `TLStacks721.updateDropAllowance()`, and `TLStacks721.updateDropDuration()`

Summary

Delayed administrative updates intended for an earlier drop round can overwrite a replacement round's allowlist, wallet cap, or schedule because the setters do not bind the update to an expected round. A buyer can then obtain replacement-round supply under obsolete eligibility, allowance, or timing terms while supplying the correct current round and paying the replacement price.

Description

Expected behavior

An administrative update to a drop's allowlist root, public wallet allowance, or schedule should modify only the round for which it was prepared. Once a drop is closed and a replacement round is configured, an earlier operation should revert and leave the replacement configuration unchanged.

Observed behavior

Across the six variants, an authorized administrator prepares an update while round 0 is current, and an independently ordered authorized owner then closes that drop and configures round 1. Because the unchanged update remains valid for the collection, it executes against round 1 without advancing its round counter.

The allowlist-root variants first reject an old-round member under the replacement root; the stale root update then restores that member's eligibility and permits a current-round mint. The allowance variants first reject a two-token purchase under a one-token replacement cap; the stale update raises the cap to two and the same quantity succeeds. The duration variants first reject a purchase from a replacement scheduled seven days in the future; the stale update changes it to public sale and permits an early current-round mint. These are separate manifestations of the same state-binding defect, not a compound execution chain.

Root cause

`TLStacks721` and `TLStacks1155` reuse current-drop storage across rounds, while their root, allowance, and duration setters mutate that storage without checking which round the caller intended.

The ERC-721 root, allowance, and duration setters and their ERC-1155 root, allowance, and duration counterparts accept the target identifiers and replacement values, but no `expectedRound`. The contracts store each current drop separately from its round counter in ERC-721 storage and ERC-1155 storage, so closing and reconfiguring a drop leaves the

replacement reachable through the same key. Authorization and phase checks apply to that current record but cannot establish that the queued operation belongs to its generation.

The buyer-side guards in `TLStacks721._validatePurchase()` and `TLStacks1155.purchase()` do not compensate for the missing administrative check. The stale setter leaves the replacement round number unchanged, so a buyer correctly naming that round passes the generation check after the configuration has already been altered.

Conditions and assumptions

The stale caller must remain the collection owner or an `ADMIN_ROLE` holder, remain unsanctioned, and execute while Stacks is unpaused. The replacement and stale update must be independently ordered across authorized principals or an equivalent queued path; an ordinary single EOA's nonce ordering prevents its later lifecycle transaction from overtaking its earlier update.

The replacement must be in a phase where the relevant setter accepts updates. For roots, it must be a regular presale, and the obsolete root must include a member excluded by the replacement with a valid proof. For allowances, the stale cap must exceed the replacement cap. For durations, the replacement must be future-scheduled and the stale values must open it. A buyer must act before correction, supply the current round, pass the applicable purchase and payment checks, and remain bounded by remaining supply and the applicable allowance.

Impact

The demonstrated harm is irreversible replacement-round allocation under obsolete terms: a previously excluded member can mint, one buyer can receive two tokens despite a one-token wallet cap, or a buyer can mint seven days before the configured start. The proofs cover both ERC-721 and ERC-1155 drops; restoring the intended root, cap, or schedule can stop further purchases under obsolete terms but cannot reverse a completed mint or replenish consumed supply.

The overall severity is Low. The effect is confined to the affected collection and drop, requires a still-authorized stale operation to cross a replacement boundary through independently ordered administration, and requires a buyer to act before correction.

Purchases still name the replacement round, use its configured price, and satisfy the remaining purchase checks. The evidence does not show forged authority, theft, underpayment, or a total-supply bypass. Additional allocation or economic harm depends on the obsolete root's leaf allowances, the stale cap, remaining supply, demand, and response time; no asset value or market loss was measured.

Recommendation

Bind every allowlist-root, allowance, and duration update in both contracts to the intended drop round. Add a required `expectedRound` argument and compare it with the applicable `_rounds` value immediately before reading or mutating the current drop; a mismatch should revert without changing any configuration.

This changes the setter ABIs and requires administrative clients and queued-operation systems to retrieve and submit the intended round. A versioned replacement entry point is a viable compatibility path only if the existing unbound setter is retired, restricted, or made to enforce the same round check.

Validation objective

Verify that an authorized update naming the current round still succeeds under the existing authorization, sanctions, pause, phase, and drop-type rules. An update naming round N must revert after the drop advances to round N+1, leaving the replacement root, allowance, timing, phase, round, and supply unchanged; the old proof, two-token purchase under a one-

token cap, and pre-start purchase must remain rejected.

HEX-032: Round-blind velocity and supply updates can prematurely end successor drops

- ID: HEX-032
- Status: New
- Class: Finding
- Severity: Low
- Location: TLStacks721.updateDropDecayRate(); TLStacks1155.updateDropSupply()

Summary

An authorized decay-rate or supply update prepared for one drop round can execute against a later replacement because the administrative setters do not identify the intended round. Under independently ordered administration, a stale update can end a successor ERC-721 or ERC-1155 drop after one valid mint, leaving most of that round's configured supply unavailable until administrators create another round.

Description

Expected behavior

An administrative decay-rate or supply change should apply only to the drop round for which it was prepared. If the collection or collection/token slot advances before execution, the call should revert without modifying the replacement, while an update naming the current round should continue to work within the existing authorization and value constraints.

Observed behavior

Across both token standards, an authorized operation formed during round 0 remains executable after an independently authorized owner closes that round and configures round 1. Buyer calls reject an outdated `expectedRound` (ERC-721 purchase validation; ERC-1155 purchase validation), but the administrative operations instead resolve the current drop silently.

In the decay-rate manifestations, the delayed call changes a one-day, zero-decay velocity replacement to a negative one-day rate. A valid one-token purchase then leaves 99 of 100 units but reduces `publicDuration` to zero; the phase becomes `ENDED`, and a later current-round purchase reverts. The purchase paths apply the decay to duration (ERC-721 velocity-state update; ERC-1155 velocity-state processing), and zero duration ends the drop (ERC-721 phase calculation; ERC-1155 phase calculation).

In the supply manifestations, a stale reduction to one changes a replacement's initial and remaining drop supply from 100 to one without changing its round. One valid current-round mint consumes that unit and ends the replacement, after which another purchase is rejected.

Root cause

`TLStacks721.updateDropDecayRate()`, `TLStacks721.updateDropSupply()`, `TLStacks1155.updateDropDecayRate()`, and `TLStacks1155.updateDropSupply()` validate and apply administrative mutations without binding the calldata to the caller's intended drop round.

Each contract stores the mutable current `Drop` separately from its round counter (ERC-721 round storage; ERC-1155 round storage). The ERC-721 decay setter, ERC-721 supply setter, ERC-1155 decay setter, and ERC-1155 supply setter take no `expectedRound`; they resolve and validate the current slot at execution. Because ERC-721 `closeDrop()` and ERC-1155 `closeDrop()` advance the separate counter, authorization and current-state checks can all pass against a generation the operation was not prepared to modify.

The buyer-side round check runs only when a purchase executes, after the stale administrative value has been installed. It can reject outdated buyer calldata but cannot establish the administrator's earlier intent or restore the replacement configuration.

Conditions and assumptions

The stale writer must remain the collection owner or an `ADMIN_ROLE` holder, be unsanctioned, and execute while Stacks is unpaused. Another authorized principal, or an authenticated execution system with independently ordered operations, must replace the intended round before the stale call executes. Transactions from one EOA cannot produce this ordering because nonce ordering preserves their sequence.

Decay crossover requires a velocity target and velocity replacement, an unended replacement, and a stale negative rate large enough to consume its remaining duration when a valid purchase applies it. Supply crossover requires a configured replacement that has not ended; an ERC-1155 replacement may still be `NOT_STARTED`, while ERC-721 selectable drops reject the mutation. The stale total must remain between the replacement's minted amount and initial supply. These are stale-intent failures, not authorization bypasses.

Impact

The decay variants terminate a successor velocity round after one valid mint, leaving 99 of 100 configured units unavailable in that round. The supply variants reduce a 100-unit successor allocation to one, so the first valid mint exhausts it and prevents the other 99 intended mints. For ERC-1155, the affected value is the drop's sale allocation, not the token's aggregate supply.

The completed mint succeeds, and the evidence shows no custody compromise or direct monetary loss. Sale interruption and administrative restart work are supported; missed demand or revenue remains plausible but unmeasured.

An administrator can correct a stale decay rate before the triggering purchase. Once a round has ended or been exhausted, recovery requires configuring another round, and the supply setters cannot increase the truncated round in place. This bounded, recoverable disruption and its narrow privileged-ordering prerequisites support a Low severity rating.

Recommendation

Require the ERC-721 and ERC-1155 decay-rate and supply setters to receive an `expectedRound` and compare it with the relevant `_rounds` entry before loading or mutating the current `Drop`. A mismatch should revert without changing the replacement.

Preserve the existing authorization, sanctions, pause, phase, type, and value-bound checks. For supply changes, also preserve the minted-count invariant and the prohibitions against increasing supply or reducing it below the minted amount. Changing these signatures affects the external ABI; if compatibility requires a guarded overload or versioned endpoint, update administrative integrations and retire the unbound path because leaving it callable preserves the stale mutation route.

Validation objective

Across ERC-721 and ERC-1155, prepare each setter call for round N, advance the target to round N+1 through explicit closure and, where supported, natural end and reconfiguration, then verify that the stale call reverts with the replacement's round, rate, duration, initial supply, remaining supply, and phase unchanged. Verify that current-round decay updates are applied by subsequent velocity purchases and that current-round supply reductions preserve the minted count and ERC-721 URI progression while retaining existing type, phase, increase, and below-minted reverts.

HEX-033: Stale listing authorizations can recreate canceled sales under obsolete terms

- ID: HEX-033
- Status: New
- Class: Finding
- Severity: Low
- Location: TLAuctionHouse._list()

Summary

An independently executable stale listing authorization can become valid again after a newer listing is canceled, allowing a buyer to acquire the seller's NFT under obsolete terms. The auction house assigns the recreated listing a fresh ID, so buyer-side ID checks do not reject the sale.

Description

Expected behavior

Once a token passes through a later listing lifecycle, an earlier independently ordered creation authorization should not become applicable merely because the token is again unlisted and owned by the same account. Listing creation should bind the seller's authorization to persistent per-token state that advances across intervening lifecycles.

Observed behavior

TLAuctionHouse.list() and batchList() accept only a listing configuration. TLAuctionHouse._list() requires the current listing slot to be unconfigured and `ownerOf(tokenId)` to equal `msg.sender`, but it does not require the caller to supply an expected per-token generation.

After TLAuctionHouse.delist() deletes the newer listing and returns the NFT, those predicates can become true again. An older authenticated operation can create another listing, and TLAuctionHouse._buildListing() assigns it a fresh global ID that TLAuctionHouse.buyNow() accepts as current. The demonstrated `list()` sequence preserves a pending 1 ETH authorization across an intervening 10 ETH listing and cancellation, after which the NFT is bought for 1 ETH; `batchList()` reaches the same helper but was not directly exercised.

Root cause

`TLAuctionHouse._list()` validates only current ownership and listing absence, without binding creation to a persistent per-token generation.

The listing mapping is the only per-token listing state, and deletion removes evidence that an intervening lifecycle occurred. The global `_id` is assigned only after creation succeeds, so it is an output freshness value for buyers and bidders rather than an input precondition protecting seller intent. ListingConfig supplies neither an expected persistent epoch nor an expiry.

Conditions and assumptions

The condition applies when an NFT-owning smart account or execution system allows independently authenticated calls to remain live and execute out of order. The seller must have authenticated the stale `list()` call; no listing authority is created or forged. Strict sequential nonces, expiry, or prior call cancellation prevent the sequence, so applicability is limited to integrations that leave such calls independently executable.

After a newer listing is removed, the NFT must return to the same account and auction-house approval must remain effective. The auction house must remain unpaused, the seller unsanctioned, and the stale configuration valid; changed ownership or an active listing rejects the call. A buyer must purchase before the seller cancels the recreated listing.

Once these conditions hold, the executor and buyer require no protocol privilege. An account restricted to a privileged relayer narrows applicability.

Impact

The recreated listing can escrow and irreversibly transfer the seller's NFT under terms that predate a later listing lifecycle. Buyer-side listing-ID checks do not prevent the sale because stale creation receives a fresh ID. The demonstrated scope is one NFT and one previously authenticated operation.

Severity is Low because practical seller loss requires the combined account model, restored ownership, continuing approval, valid contract state, and buyer timing described above. The 1 ETH proof price establishes only that the accepted terms predated an intervening 10 ETH listing; economic loss depends on the stale price being below the seller's current valuation. The evidence does not establish market value or use of the required account model by a deployed wallet or relayer.

Recommendation

Retain a monotonically increasing epoch for each `(nftAddress, tokenId)` independently of the deletable listing record. Require `list()` and each `batchList()` element to supply the expected current epoch, reject mismatches, and advance the epoch when a listing is created so deletion cannot make an earlier authorization valid again. Keep the epoch per token rather than using the global listing ID, which would allow unrelated listings to invalidate otherwise fresh operations.

A deadline or account-level invalidation can complement this binding but does not replace it. This design changes the public ABI and adds persistent storage and gas costs.

Validation objective

Authenticate two creation operations against the same token epoch, execute one, cancel its listing, and verify that the other reverts. Confirm that a newly authored operation using the advanced epoch succeeds. Add equivalent coverage for a stale `batchList()` element causing the documented atomic rejection while existing listing-ID checks for buyers and bidders continue to work.

HEX-034: Seller-fee multiplication can reject representable mint purchases

- ID: HEX-034
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLStacks721._settleUp()`; `TLStacks1155._settleUp()`

Summary

A configured ERC-721 or ERC-1155 mint purchase can revert when the seller-fee calculation overflows its intermediate multiplication, even though the gross sale, exact fee, and seller proceeds all fit in `uint256`. The atomic revert blocks purchases for the affected ERC-20 drop until an authorized party changes the fee or drop configuration.

Description

Expected behavior

For an otherwise valid ERC-20 mint purchase, settlement should complete whenever the gross sale, the floor-rounded seller fee, and the seller proceeds are representable. The calculation should preserve the existing round-down rule without requiring the intermediate fee numerator to fit in `uint256`.

Observed behavior

Both implementations evaluate `totalSale * feeBps / 10_000`. With a 500-basis-point fee and `totalSale = floor(uint256.max / 500) + 1`, the gross sale and mathematically correct 5% fee fit, but the intermediate product does not.

The ERC-721 regular `purchase()` and ERC-1155 public-sale `purchase()` proofs therefore reach arithmetic panic `0x11`. Each transaction rolls back before transferring funds or minting and does not consume drop supply; waiving only the seller fee lets the otherwise unchanged purchase complete. `TLStacks721.purchaseSelected()` uses the same settlement control point, so it has the same source-level exposure, although that entry point was not directly executed.

Root cause

`TLStacks721._settleUp()` and `TLStacks1155._settleUp()` calculate the seller fee with checked multiply-before-divide arithmetic, so a representable quotient can fail when the intermediate product exceeds `uint256`.

The fee cap limits the seller fee to a fraction of the sale but does not bound the multiplication's numerator or the accepted raw-unit sale total. The configured ERC-20 price is stored as a `uint256`—including in the ERC-1155 `Drop` structure—without a corresponding bound that keeps `totalSale * feeBps` representable.

Conditions and assumptions

Exposure requires an allowlisted ERC-20 with ordinary transfer semantics and sufficient raw supply, an unwaived seller fee of at least two basis points, and an active drop whose representable `totalSale` exceeds `floor(uint256.max / feeBps)`. An authorized Stacks owner sets the currency and fee, and an NFT owner or administrator sets the affected drop; once configured, any otherwise eligible buyer with enough balance and allowance can reach the revert.

The proofs use 500 basis points, quantity one, and an OpenZeppelin ERC-20 with 75 display decimals. `decimals()` affects display rather than transfer arithmetic (`dependencies/@openzeppelin-contracts-5.6.1/token/ERC20/ERC20.sol:64–78`), so this reaches the boundary without fee-on-transfer or other unsupported mechanics, but the precision is unusual. The practical claim is limited to ERC-20 drops; the equivalent native-currency balance is infeasible.

Impact

An affected ERC-20 drop remains unavailable for the implicated purchase path while the high-unit price and nonzero seller fee remain configured. Buyers can lose gas on failed attempts, but atomic rollback leaves funds and drop state unchanged; no asset loss or persistent state corruption is demonstrated.

Severity is Low because the harm is bounded to availability under an unusual configuration, and service can be restored through authorized fee or drop changes. The Stacks owner can waive or sufficiently reduce the fee, while the NFT owner or administrator can change the affected drop configuration; purchases remain blocked until such action is taken.

Recommendation

Replace the seller-fee expressions in both settlement functions with a full-precision multiply-divide operation such as `Math.mulDiv` (`dependencies/@openzeppelin-contracts-5.6.1/utils/math/Math.sol:206–279`), preserving floor rounding, fee-waiver behavior, and payout conservation. If arbitrary raw-unit totals are intentionally unsupported, an alternative is to enforce and document a safe sale-total bound before a drop becomes active. Full-precision arithmetic adds execution cost, while a bound narrows supported currencies and prices.

Validation objective

Exercise ERC-721 regular and selectable purchases and ERC-1155 public and presale conditions with zero, waived, one, two, and 500 basis points, boundary values that exist around `floor(uint256.max / feeBps)`, and multi-token gross sales. Verify that every supported configuration with a representable gross sale, exact fee, and seller-proceeds split settles with the floor-rounded full-precision result, while genuinely unrepresentable gross sales still revert atomically and ordinary-range payouts remain unchanged.

HEX-035: Highest bidders can extend auctions beyond the configured maximum duration

- ID: HEX-035
- Status: New
- Class: Finding
- Severity: Low
- Location: `TLAuctionHouse.bid()`

Summary

A current highest bidder can use ordinary late bids to recover its prior ETH bid and repeatedly move an auction's end beyond the configured maximum duration. While the bidder maintains the cycle, settlement remains unavailable and the seller's NFT stays escrowed without a cumulative deadline.

Description

Expected behavior

After the first bid prevents the seller from withdrawing the NFT, the auction should retain a finite latest completion time. An increase from the current highest bidder should either use the non-extending top-up behavior or remain subject to an absolute per-listing extension limit.

Observed behavior

`TLAuctionHouse._buildListing()` rejects only an initially configured duration greater than `MAX_AUCTION_DURATION`. The ordinary `TLAuctionHouse.bid()` path accepts the current highest bidder and adds enough time to restore five minutes whenever less than five minutes remain, without checking the cumulative duration.

The contract escrows the full replacement bid and then refunds the previous bidder. When both parties are the same account, that account recovers its prior bid and only the latest bid remains as net escrow. By contrast, `TLAuctionHouse.topUpBid()` raises the highest bid without extending the deadline, matching the documented distinction between late bids (`README.md:30`) and highest-bidder top-ups (`README.md:45`).

After a bid, seller `delist()` is unavailable. `settleAuction()` requires the timestamp to be after the mutable end, and `cancelUnsettledAuction()` starts its additional seven-day grace period from that same end.

In the demonstrated maximum-duration ETH auction, ten ordinary self-bids submitted one minute before successive deadlines increased stored duration from 2,592,000 to 2,594,400 seconds. After the original configured deadline had passed, only the 12-wei final bid remained escrowed, the NFT remained in the auction house, and both seller delisting and settlement still reverted.

Root cause

The `TLAuctionHouse.bid()` function extends the mutable `duration` without enforcing a cumulative hard end or routing the current highest bidder through non-extending top-up semantics.

The initial maximum-duration check and the non-extension rule apply only in separate entry points rather than as an auction-wide invariant. Because settlement and cancellation both derive their eligibility from the mutable duration, every uncapped late-bid extension also moves the available exit times.

Conditions and assumptions

The path requires an active reserve auction with a bid, an unpaused auction house, an eligible unsanctioned current highest bidder, and repeated transaction inclusion during successive final-five-minute windows. Each replacement must be at least 5% higher, rounded up, and the bidder must provide the full replacement bid before receiving the refund; low-cost repetition therefore depends on a low reserve or current bid.

Competition or a protocol-owner pause can curtail repetition, while gas costs, increasing bid exposure, and the risk of winning constrain the bidder. If repetition stops, the auction becomes settleable after the extended end and the bidder remains liable for the winning bid.

Impact

The concrete harm is a temporary, listing-local availability loss. While the extension cycle continues, the NFT remains escrowed, the seller cannot delist, settlement remains unavailable, and the seven-day cancellation grace period is deferred.

Severity is Low because maintaining the delay requires successive timed transactions, full increasingly large replacement bids, gas expenditure, and acceptance of winner liability. Interruption permits settlement after the extended end, and no unauthorized transfer, theft, permanent lock, or protocol-wide denial was demonstrated. Any opportunity loss or market disruption depends on the auction's economics, competition, and the duration of the delay.

Recommendation

Store an absolute per-listing end time or cumulative extension budget and enforce it in `bid()` so no late bid can move the auction beyond that bound. Align timestamp comparisons so bidding closes when settlement becomes available.

Routing an increase from the current highest bidder through non-extending top-up semantics can provide additional protection, but it should not replace a hard bound because an actor could alternate addresses. A finite limit trades some anti-sniping flexibility for seller liveness, so expose its semantics clearly if it is configurable.

Validation objective

Verify that legitimate competing late bids can extend an auction only up to the selected limit and that neither self-bids nor an alternating-address sequence can move settlement or cancellation eligibility beyond it. At each deadline boundary, confirm correct prior-bid refunds, escrow solvency, top-ups, settlement, and seller NFT recovery.

Unscored observations

OBS-001: Mint-target compatibility depends on a Transient Labs access-control interface

- ID: OBS-001
- Status: New
- Class: Observation
- Location: TLStacks721._isDropAdmin() and _isApprovedMintContract(); TLStacks1155._isDropAdmin() and _isApprovedMintContract()

Summary

Third-party ERC-721 and ERC-1155 collections can implement the advertised `externalMint` interface and authorize a Stacks strategy to mint, yet remain unable to configure a drop because configuration assumes Transient Labs-compatible `owner()` and role-query behavior. This pre-sale failure prevents affected collection operators from starting a drop through either strategy.

Description

Expected behavior

The package overview (`README.md:4–20`) states that the strategies work with any NFT using the appropriate `externalMint` signature and that an NFT owner or administrator can manage a mint. A collection meeting that mint boundary and authorizing the relevant Stacks strategy should therefore be configurable by either recognized operator, or the supported-target interface should disclose every narrower administrative requirement.

Observed behavior

Both `configureDrop` paths call an administration helper and then a fixed approved-minter preflight before storing the drop (ERC-721 checks and storage; ERC-1155 checks and storage). `_isDropAdmin()` calls `owner()` before evaluating `hasRole(ADMIN_ROLE, msg.sender)`, so an absent or ABI-incompatible ownership response stops the role branch. `_isApprovedMintContract()` separately requires `hasRole(keccak256("APPROVED_MINT_CONTRACT"), address(this))`; a missing or malformed role response reverts, while a well-formed `false` result raises `NotApprovedMintContract`.

Focused checks for both standards found that signature-only and owner-only targets were rejected, exact administrative responses permitted configuration, and every failed attempt left the drop unconfigured. Separate controls showed that the exact Stacks strategy could call `externalMint` under a collection-native `MINTER_ROLE`, yet the fixed Transient Labs role check still rejected configuration. Ownerless collections granting the exact `ADMIN_ROLE` and approved-minter role likewise accepted direct mints from the strategy but reverted before the administrator role could be evaluated.

Root cause

`TLStacks721` and `TLStacks1155` couple drop configuration to the Transient Labs ownership and role convention instead of the documented mint capability.

The owner-or-admin policy is encoded as a short-circuit expression with a typed `owner()` call on the left, so failure there prevents the alternative role check. The mint preflight then treats one concrete role hash as proof of capability, even when the target grants the strategy equivalent permission under another valid policy. Together, these assumptions make the advertised signature-based integration boundary narrower in implementation.

Conditions and assumptions

This observation applies when an operator attempts to configure a third-party ERC-721 or ERC-1155 target that implements the appropriate `externalMint` entrypoint but does not fully emulate the Transient Labs administrative interface. The target must either lack an ABI-compatible `owner()` or `hasRole(bytes32,address)` response, authorize minting under a valid identifier other than `APPROVED_MINT_CONTRACT`, or use role-only administration without a callable `owner()`. The triggering operator is a legitimate collection owner or `ADMIN_ROLE` holder where the target exposes those roles.

Canonical Transient Labs collections provide the expected selectors and role identifiers and are unaffected. Changing the fixed mint-role preflight alone will not make a target with incompatible owner handling configurable; support must address the required capabilities together.

Impact

An affected collection operator cannot create a drop through either Stacks strategy even though the target mint interface is usable. Integration may require an adapter, a collection upgrade, adoption of the exact Transient Labs role marker, or redeployment, so the direct harm is setup-time unavailability and integration rework.

The failure is immediate and atomic, before a drop is stored or buyer activity begins. The evidence establishes no asset or authorization loss, so this is an unscored compatibility observation rather than a scored security finding.

Practical scope depends on third-party integrations that use alternative administration or mint-authorization models. No integration inventory was supplied, so adoption, incidence, and incurred cost are unknown.

Recommendation

Define and document one target-capability contract for both Stacks strategies, then apply it consistently during configuration. If generic third-party support is intended, make `owner()` probing failure-tolerant so a valid administrator-role result can still be evaluated, and use a documented adapter or capability interface that can represent collection-defined mint policies. Callers satisfying neither owner nor administrator authorization, and targets that do not authorize the Stacks strategy to mint, must remain rejected.

Alternatively, narrow the compatibility statement, NatSpec, and integration tooling to require the complete Transient Labs-style `owner()` and `hasRole(bytes32,address)` interface with the exact role identifiers. Preserve the target's runtime `externalMint` authorization as the ultimate enforcement point: removing the preflight without a replacement would trade configuration-time feedback for atomic purchase-time failures, while a generalized adapter adds integration and trust surface.

Validation objective

Under the chosen compatibility boundary, verify for both token standards that each documented owner, administrator, and mint-policy path can configure and complete a mint; role-only administrators can configure, update, and close when `owner()` is absent, reverting, or malformed if that path remains supported; unsupported targets and unauthorized callers fail without storing a drop or transferring payment; and canonical Transient Labs flows remain unchanged.

OBS-002: Paid ERC-721 mint recipients may lack a usable recovery path

- ID: OBS-002
- Status: New
- Class: Observation
- Location: `TLStacks721._mintToken()` and `TLStacks721._mintSelectedTokens()`

Summary

A paid ERC-721 purchase can settle to a recipient contract that rejects safe delivery or cannot later exercise ownership, while seller payment and drop accounting remain final. With the accepted `ERC721TL` path, the NFT can become stranded at `TLStacks721` itself or another incapable contract.

Description

Expected behavior

Paid mint delivery to a contract should either obtain ERC-721 recipient acceptance or provide a reliable, appropriately authorized recovery path. If the recipient rejects delivery, the transaction should revert atomically before ownership, payment, supply, allowance, or selected-index state changes.

Observed behavior

The accepted `ERC721TL.externalMint()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:254–262`) implementation assigns ownership with plain `_mint`, so it does not invoke recipient acceptance. Both Stacks mint helpers call this entrypoint after sale validation, accounting, and payment handling. A mint revert would roll back those earlier effects; the issue arises because unchecked minting succeeds.

When regular and selectable paid purchases named `TLStacks721` itself as recipient, each purchase paid the seller, consumed drop supply and recipient allowance, and left the NFT owned by `TLStacks721`; the selectable purchase also committed its selected index and metadata URI. The payer and administrators could not transfer or burn the token, and closing the drop did not change ownership. `TLStacks721`'s `withdrawFunds()` handles only native currency and ERC-20 tokens, while the collection's `withdrawERC721()` (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/ERC721TL.sol:463–466`) cannot move a token owned by `TLStacks721`.

A regular public native-currency purchase to a contract that rejects the safe-delivery callback likewise succeeded and left the contract unable to exercise ownership. The 1 ETH seller payment and 0.01 ETH protocol fee committed, supply fell to zero, the recipient allowance was consumed, and the recipient received no recovery approval. As a control, a safe transfer to the same recipient reverted and left the control token with its original owner.

Root cause

`TLStacks721._mintToken()` and `TLStacks721._mintSelectedTokens()` use an unchecked external-mint path for paid delivery, while `TLStacks721` provides no recovery control for a recipient that cannot exercise ownership.

The `IERC721TL.externalMint()` interface (`dependencies/tl-creator-contracts-4.1.1/src/erc-721/IERC721TL.sol:52–56`) assumes that the destination can receive ERC-721 tokens, but the accepted implementation realizes that assumption with non-checking `_mint` (`dependencies/@openzeppelin-contracts-upgradeable-5.6.1/token/ERC721/ERC721Upgradeable.sol:274–294`) rather than callback-enforcing `_safeMint` (`dependencies/@openzeppelin-contracts-upgradeable-5.6.1/token/ERC721/ERC721Upgradeable.sol:296–317`). The paid-sale path therefore does not test a contract recipient's acceptance before committing delivery to it.

Conditions and assumptions

This condition applies when an active drop uses the accepted `ERC721TL` or another collection whose `externalMint` performs non-safe delivery, `TLStacks721` holds the approved-minter role, and ordinary sale validation passes. The payer or its integration chooses and funds the recipient. The self-recipient manifestation requires selecting the exact `TLStacks721` address; broader stranding requires a contract without generic execution, upgradeability, pre-existing approval, or another recovery mechanism.

A collection whose `externalMint` enforces recipient acceptance would revert the purchase atomically. Plain minting is ERC-721-compliant, so this observation concerns paid-sale delivery rather than a universal requirement that custom mint functions use safe-mint semantics. Callback rejection alone does not establish that every contract recipient lacks a later recovery path, while callback acceptance confirms delivery-time consent but does not guarantee future recoverability.

Impact

A payer who selects a genuinely incapable recipient can complete payment without obtaining usable control of the NFT. The seller retains the sale proceeds and, where configured, the protocol retains its fee, while the applicable drop supply, recipient allowance, and any selected index remain consumed. The practical loss depends on the purchase value and the recipient's actual recovery capabilities.

This is an unscored compatibility and recovery observation because the payer controls and funds the recipient selection. The evidence does not show outsider redirection of another payer's transaction, attacker profit, an unauthorized transfer, or protocol-wide harm. An intentional burn-like purchase would not represent the same harmful scenario.

Recommendation

Use a callback-enforcing approved-minter entrypoint backed by `_safeMint` for regular and selectable paid deliveries. If compatibility requires retaining the existing non-safe entrypoint for administrative or other integration workflows, separate it from the paid-sale path and require explicit, informed opt-in before unchecked delivery.

As a targeted safeguard, reject `recipient == address(this)` before accounting or payment. If intentional self-custody or delivery to incapable recipients must remain available, define a recovery path bound to the purchase beneficiary and prevent it from becoming an administrative mechanism for seizing unrelated collector assets.

A safe paid-mint path will reject contracts that can manage NFTs but do not implement `IERC721Receiver`. It also introduces a recipient callback during minting and may require custom collection integrations to adopt a new interface. These compatibility and interaction changes should be handled explicitly.

Validation objective

Verify that rejecting and self-recipient deliveries revert atomically before ownership, buyer funds, seller and protocol balances, supply, allowance, or selected-index state changes. Cover regular and selectable drops, native and ERC-20 payments, presale and public phases, EOAs, and accepting contract recipients. If a recovery design is selected instead,

verify that only the designated beneficiary can recover the purchase and that unrelated collector assets remain inaccessible.

OBS-003: Unvalidated static deadlines can make drop phases unreadable

- ID: OBS-003
- Status: New
- Class: Observation
- Location: TLStacks721.configureDrop(); TLStacks1155.configureDrop()

Summary

An authorized collection operator can store an ERC-721 or ERC-1155 drop schedule whose cumulative phase endpoint cannot be represented. Once evaluation reaches that endpoint, phase reads, purchases, and in-place repair panic until the operator closes and recreates the affected drop, advancing its round.

Description

Expected behavior

Every schedule accepted by `configureDrop` or `updateDropDuration` should remain safely evaluable throughout its lifecycle. Invalid values should revert before changing the stored drop or its round, preserving the current sale's availability and repairability.

Observed behavior

The ERC-721 configuration path and ERC-1155 configuration path accept candidate schedules without checking cumulative endpoints. Their respective ERC-721 duration-update and ERC-1155 duration-update paths likewise store replacement values after evaluating only the existing schedule.

The proof variants use a reached positive start time, zero presale duration, and a public duration selected so that `startTime + presaleDuration + publicDuration` exceeds `type(uint256).max`. When the ERC-721 evaluator or ERC-1155 evaluator reaches that expression, checked arithmetic panics with error `0x11`. The raw schedule remains readable, but phase lookup, an ordinary purchase, a safe duration correction, and replacement configuration revert.

Both purchase paths evaluate the phase before settlement or minting (ERC-721, ERC-1155). The ERC-721 and ERC-1155 `closeDrop` paths do not evaluate the phase, so an authorized operator can delete the invalid configuration and create a valid one; closure increments the drop round.

Root cause

The static timing writers in `TLStacks721` and `TLStacks1155` persist schedules without enforcing the endpoint-representability invariant required by `_getDropPhase`.

Each configuration path checks the prior or empty state and other candidate fields before assigning the proposed schedule. Each duration update establishes the old phase and then stores new timing values without evaluating them. The evaluators later derive phase boundaries through checked addition, while corrective updates and replacement configuration evaluate the already-stored phase before writing, leaving `closeDrop` as the demonstrated recovery path once an overflowing branch is reached.

Conditions and assumptions

The triggering caller must be the target NFT contract owner or an `ADMIN_ROLE` holder; a permissionless user cannot invoke either writer. The Stacks contract must be unpaused and the caller unsanctioned. Fresh configuration also requires approved-minter setup and the ordinary candidate checks, including a nonzero payout receiver and supply; a live update requires an existing drop that has not ended.

The fault becomes immediate only when a drop with remaining supply reaches the branch that calculates an unrepresentable endpoint. A future start, zero supply, or an active presale can return earlier; the active-presale controls remained correctable in place. The demonstrated durations are extraordinary values near the `uint256` limit.

Recovery requires an available NFT owner or administrator and functioning NFT authorization calls. The affected state is confined to one ERC-721 collection drop or one ERC-1155 collection-and-token-ID drop.

Impact

While the overflowing branch is reached, the affected drop is unavailable for phase reads, purchases, duration correction, and replacement configuration. Closing and rebuilding it interrupts mint availability and makes transactions prepared for the former round stale. Missed mint opportunities or revenue during recovery are possible but were not measured.

The disruption is confined to one ERC-721 collection drop or one ERC-1155 token-ID drop; other token IDs remained functional in the focused ERC-1155 proof. Failed purchases revert before funds are distributed or tokens are minted, and no asset loss, privilege gain, permanent lock, cross-drop corruption, or protocol-wide outage was demonstrated. This is an unscored observation because it requires an authorized operator to submit extraordinary near-`uint256` timing values, and the demonstrated state is recoverable through authorized closure and reconfiguration.

Recommendation

Validate every candidate schedule before storing it in both `configureDrop` and `updateDropDuration` for both token standards. A shared validator can use subtraction-based bounds to require `presaleDuration <= type(uint256).max - startTime` and `publicDuration <= type(uint256).max - startTime - presaleDuration`, reverting with a specific error before any state change.

Alternatively, calculate phase progress from elapsed time after establishing `block.timestamp >= startTime`, avoiding cumulative endpoint additions. This approach can preserve very large durations but requires careful handling of exact phase boundaries. If a maximum duration is intended to represent an open-ended sale, model that state explicitly rather than relying on an overflowing timestamp.

Validation objective

Regression tests should verify that fresh configurations and live updates cannot leave either manager with a schedule that later panics during phase evaluation; rejected writes must preserve the prior schedule and round. Cover overflow at both cumulative boundaries, immediate and future-start cases, the active-presale short-circuit, the largest valid endpoints, and equality at each phase transition. Ordinary schedules should remain purchasable and correctable, and authorized closure and reconfiguration should retain their existing behavior.

OBS-004: Pausing does not preserve active auction bidding time

- ID: OBS-004
- Status: New
- Class: Observation
- Location: TLAuctionHouse.pause(bool)

Summary

Pausing `TLAuctionHouse` blocks bids but does not suspend active auction deadlines. If a stored deadline passes before the owner unpauses, funded challengers lose the remaining competitive window, and the auction can settle immediately after resumption in favor of the incumbent.

Description

Expected behavior

When an owner pauses bidding during a live auction, resuming the auction should preserve the bidding time that participants could not access or require an explicit reopening or recovery transition before settlement. The auction feature documentation (`README.md:23–51`) describes wall-clock durations and bid-triggered extensions but does not define paused-time handling.

Observed behavior

The reproduction starts a one-day reserve auction and records a valid incumbent bid. With one hour remaining, the owner pauses the contract; a funded challenger submits the contract-calculated next bid and receives `EnforcedPause`. One second after the original deadline, the owner unpauses, `startTime + duration` remains unchanged, and the same higher bid reverts with `AuctionEnded`.

The seller cannot restore the interval: attempts through TLAuctionHouse.updateListing() and TLAuctionHouse.delist() both revert with `AuctionStarted`. TLAuctionHouse.settleAuction() then succeeds and transfers the NFT to the incumbent.

Root cause

`TLAuctionHouse.pause(bool)` toggles the global pause state without preserving elapsed bidding time in active listings.

TLAuctionHouse.bid() combines `whenNotPaused` with an absolute `startTime + duration` deadline. Only a successful bid in the final five minutes extends `duration`, but no bid can reach that extension logic while paused; the Listing records no pause timestamp or accumulated offset. Settlement therefore evaluates the unchanged deadline and becomes available after unpause once that deadline has passed.

Conditions and assumptions

Only the `TLAuctionHouse` owner can trigger the pause. This observation does not assume owner compromise or malicious intent; a legitimate emergency pause during a live auction is sufficient, and sellers or bidders cannot create the stopped state.

The auction must already have a highest bidder, the pause must begin before its stored end, and the owner must unpause strictly after that end. At exact equality, a bid remains valid and receives the normal extension. A reserve auction without a first bid has no running clock, while a scheduled auction without a bid remains updatable or delistable after unpause.

Impact

The direct harm is irreversible loss of competitive access within an affected auction: a funded challenger cannot place the otherwise valid higher bid during the pause or after resumption, and settlement can award the NFT to the incumbent without restoring the configured remaining participation time. Reduced competition may lower seller proceeds or change the winner, but those outcomes are counterfactual and were not directly proved.

The rejected bid reverts atomically, so the challenger loses no principal, and the incumbent's bid remains valid; no asset or accounting loss was demonstrated. A global pause can overlap multiple live auctions, but aggregate exposure depends on their number and value. The condition is an unscored observation because it is a trusted-owner operational footgun with bounded opportunity and fairness effects rather than proved asset loss.

Recommendation

Make each active auction's effective deadline exclude periods in which the global pause prevents bidding. Alternatively, require an explicit per-listing reopening or extension before a pause-affected auction can settle. Apply any adjusted end consistently to bidding, highest-bidder top-ups, settlement, and settlement-grace calculations so lifecycle boundaries remain aligned.

A global accumulated-pause clock can reduce per-listing storage, but it must count only overlap relevant to each auction to avoid overextending auctions that had not started. Per-listing accounting is more precise but adds state and gas costs. Preserve exact-deadline bidding and late-bid extension semantics.

Validation objective

Verify that, after unpause, each affected bid-on auction retains exactly the bidding time made inaccessible by pause and that neither settlement nor cancellation uses the unadjusted end before the restored interval and applicable grace period elapse. Cover reserve, reserve-plus-buy-now, and scheduled auctions, including repeated pauses, a scheduled start during pause, exact-deadline bids, late-bid extensions, and unbid listings.

OBS-005: Ownership renunciation can remove recovery from later oracle failure

- ID: OBS-005
- Status: New
- Class: Observation
- Location: TLAuctionHouse.setSanctionsOracle()

Summary

If `TLAuctionHouse` ownership is renounced while a nonzero sanctions oracle remains configured, a later permanent oracle failure can prevent bidders and sellers from recovering funds and NFTs held by the Auction House. Timeout cancellation records claims but does not make them withdrawable, and the ownerless contract can no longer replace or disable the failed oracle.

Description

Expected behavior

The documented recovery flow converts an auction that cannot settle into bidder funds and seller NFT claims after the grace period (README.md (`README.md:47–49`)). An administrative lifecycle transition should not eliminate every policy-compliant means of repairing a mandatory external dependency while that dependency can still gate third-party asset exits.

Recovery does not require treating an unavailable oracle as unsanctioned. The contract can preserve fail-closed compliance behavior while retaining a narrowly constrained repair mechanism.

Observed behavior

The proof starts with a healthy, independently operated oracle, two escrowed NFTs, and a 1 ETH auction bid. It then renounces Auction House ownership before placing the oracle into a permanent failure state.

Auction settlement reverts, while timeout cancellation succeeds only by recording claims (settlement and cancellation). Both `claimFunds()` and `claimNft()` still revert, and the seller cannot `delist()` the fixed-price NFT. With no owner able to replace or disable the oracle, the Auction House retains the 1 ETH and both NFTs.

In the retained-owner control, disabling the same failed oracle allows the bidder and seller to recover every demonstrated asset.

Root cause

The `TLAuctionHouse.setSanctionsOracle()` repair control is restricted to the owner, but ownership renunciation can make it permanently unreachable while a nonzero oracle remains mandatory.

`_isSanctioned()` directly calls every configured nonzero oracle and propagates a failed lookup to settlement, delisting, and claim paths. The inherited `renounceOwnership()` (`dependencies/@openzeppelin-contracts-5.6.1/access/Ownable.sol:69–78`) sets the owner to the zero address, while `Ownable2Step._transferOwnership()`

(`dependencies/@openzeppelin-contracts-5.6.1/access/Ownable2Step.sol:48–55`) clears any pending owner. The Auction House provides no independent oracle-repair or asset-release authority after that transition.

Conditions and assumptions

Applicability requires the Auction House to hold an NFT, bid, or deferred claim; retain a nonzero oracle when the trusted owner voluntarily renounces ownership; and then experience permanent oracle failure. No permissionless caller can create this complete sequence through the accepted contract, and retaining an owner or recovering the oracle prevents the terminal lock.

The condition is scoped to the affected Auction House instance and oracle-dependent exits. It does not concern a functioning oracle intentionally reporting an account as sanctioned. The evidence establishes a possible contract state, not that any current deployment has entered it.

Impact

The proof demonstrates loss of access to 1 ETH and two NFTs while custody and accounting remain intact. The same mechanism may deny access to other escrow and claims in the affected Auction House instance, but the evidence does not establish theft, accounting corruption, protocol-wide harm, current deployment exposure, or broader value at risk.

Although the resulting lock is permanent under the demonstrated assumptions, it requires an avoidable trusted-owner renunciation followed by permanent dependency failure. This is therefore an unscored observation about an operational footgun and dependency-availability risk.

Recommendation

Prevent ownership renunciation while a nonzero sanctions oracle remains configured, or atomically disable the oracle before completing renunciation. If permanent ownership is undesirable, retain a narrowly scoped, role-separated circuit breaker that can only replace or disable the oracle.

Do not rely only on checking current liabilities, because users may create new listings after renunciation while the dependency remains configured. A one-time oracle health check is also insufficient because a healthy endpoint can fail later. Preserve the compliance policy when designing recovery: automatically treating lookup errors as unsanctioned could bypass intended restrictions, whereas controlled replacement or quarantine can remain fail-closed.

Validation objective

Verify that renunciation with a nonzero oracle either reverts, removes the dependency atomically, or leaves an independently usable repair authority. Simulate later permanent failure across fixed-price delisting, auction settlement and cancellation, native-fund claims, and NFT claims; confirm an eventual policy-compliant asset exit, continued restriction when a functioning oracle returns `true`, preserved accounting and custody during timeout cancellation, and no general asset-sweep or sanctions-bypass capability.

OBS-006: Direct ERC-20 transfers to the auction house cannot be recovered as surplus

- ID: OBS-006
- Status: New
- Class: Observation
- Location: `TLAuctionHouse.claimableFunds`

Summary

A holder who transfers conventional ERC-20 tokens directly to `TLAuctionHouse` receives no claim and cannot recover the resulting surplus through the supplied interface. Normal auction accounting remains correct, but the sender loses access to the self-authorized amount unless a token-side recovery authority exists.

Description

Expected behavior

ERC-20 balances held by `TLAuctionHouse` should either secure recorded live-bid or claim liabilities or be identifiable as surplus. Any recovery mechanism should move only surplus so that active bids and claimable credits remain fully backed.

Observed behavior

A conventional `ERC20.transfer` (`dependencies/@openzeppelin-contracts-5.6.1/token/ERC20/ERC20.sol:99–103`) changes balances entirely within the token contract and does not call the recipient. A holder can therefore increase the auction house's balance without invoking `_escrowFunds` or creating claimable accounting state.

The reproduction sends 100 tokens directly to the auction house. The sender's claim remains zero, `TLAuctionHouse.claimFunds()` reverts with `NothingToClaim`, and the owner has no `withdrawFunds`-style route. A subsequent 20-token sale pays the seller correctly while the original 100-token amount remains isolated in the auction house.

Root cause

`TLAuctionHouse.claimableFunds` records only protocol-created credits and cannot attribute tokens transferred directly to the contract.

Only `TLAuctionHouse._credit()` adds such credits for failed or blocked protocol payouts; the contract never reconciles raw ERC-20 balances against aggregate live-bid and claim liabilities. Its `administrative interface` offers no surplus recovery, while escrow and settlement operate only on exact listing amounts, so ordinary auction activity neither consumes nor attributes unsolicited balances.

Conditions and assumptions

Triggering the condition requires a holder or approved spender to send a raw ERC-20 transfer to the auction house outside the documented payment paths; no auction-house function automatically routes tokens into this state. A raw transfer does not require currency allowlisting, although the reproduced case uses an allowlisted token.

The claim is limited to conventional ERC-20 behavior without an independent issuer clawback; such token-side authority could change operational recovery. Fee-on-transfer, rebasing, and independently balance-mutating tokens are explicitly unsupported by the `setCurrencyAllowed` documentation and are not required for this condition.

Impact

A mistaken sender loses access through the supplied auction-house interface to whatever conventional-token amount they authorized for direct transfer. The amount can be arbitrary, but exposure is confined to those sender-authorized assets.

This is an unscored observation because normal auction accounting remains correct and other users are unaffected. These boundaries make a liability-aware recovery path useful hardening rather than a scored security finding.

Recommendation

Maintain an aggregate liability value for each currency covering live bids and claimable credits, then expose a narrowly authorized recovery function limited to the positive difference between the contract balance and that liability. Update the liability atomically across escrow, refund, payout, credit, and claim transitions, and emit an event for every surplus recovery.

Do not add an unrestricted owner withdrawal because the same balances secure active bids and claims. A surplus-only design introduces owner or governance custody over unattributed funds, and the contract cannot reliably identify a direct sender from its current state; a timelock or documented return process may therefore be appropriate. Continue excluding tokens whose balances can change independently of transfers.

Validation objective

Regression coverage should verify that:

- A direct conventional-token transfer creates measurable surplus that an authorized recovery path can move.
- Recovery cannot reduce the balance below aggregate live-bid and claim liabilities.
- Unauthorized callers and attempts to recover when no surplus exists revert.
- Failed payouts remain fully claimable before and after surplus recovery.
- Normal bidding, settlement, refunds, and claims retain exact accounting.
- Recovery remains possible for a removed but previously used currency without endangering outstanding liabilities.

OBS-007: Receiver-checking ERC-721 collections cannot enter auction escrow

- ID: OBS-007
- Status: New
- Class: Observation
- Location: `TLAuctionHouse._transferNft()`

Summary

Owners of otherwise transferable ERC-721 tokens cannot create any auction-house listing when the token's `transferFrom` requires contract recipients to acknowledge receipt. The auction house does not provide that receiver handshake, so each attempt reverts; the seller retains the NFT and no listing state remains.

Description

Expected behavior

The auction house states that it works for any ERC-721-compliant token (`README.md:23–25`). An otherwise transferable token that requires contract recipients to acknowledge receipt should be able to enter listing escrow, while unsolicited safe transfers remain rejected.

Observed behavior

Both `TLAuctionHouse.list()` and `TLAuctionHouse.batchList()` reach `TLAuctionHouse._list()`, which builds the listing and invokes the shared NFT transfer path. For each of the five configured listing types, a receiver-checking token tentatively transfers the NFT and then invokes `onERC721Received`; because the auction house lacks that callback, the token raises `ERC721InvalidReceiver` and the transaction reverts.

The reproduction covers scheduled auction, reserve auction, reserve auction plus buy-now, buy-now, and private-sale configurations. Every mode fails with the same receiver error, while ownership remains with the seller and the listing remains unconfigured. A conventional ERC-721 lists successfully under the same setup, isolating receiver acknowledgment as the relevant difference.

Root cause

`TLAuctionHouse._transferNft()` assumes that raw `transferFrom` can escrow every compliant ERC-721 without a receiver handshake.

The function always calls `transferFrom`, but the ERC-721 transfer mechanism permits implementations to reject unsafe transfers. `TLAuctionHouse` provides neither a dedicated safe-transfer ingress path nor a guarded `onERC721Received` callback, so collections that apply receiver validation to raw transfers cannot complete the shared escrow step.

Conditions and assumptions

The seller must own and approve an NFT from an otherwise transferable collection whose raw `transferFrom` requires contract recipients to acknowledge receipt. The auction house must be unpaused and the listing configuration and currency valid; no race or unusual transaction ordering is required.

Conventional collections whose raw `transferFrom` omits receiver validation are unaffected. A guarded callback addresses only collections that would transfer when given authenticated acknowledgment; this observation does not claim that every ERC-721 transfer restriction can be supported.

This observation applies to collections using the same receiver-checking policy, but it does not establish that any particular deployed collection currently uses that policy; deployment prevalence is unknown.

Impact

Affected NFT owners cannot create scheduled auctions, reserve auctions, reserve-plus-buy-now listings, buy-now listings, or private sales through the marketplace. A batch listing also reverts when it reaches an affected token.

The practical harm is marketplace unavailability for these collections and the cost of reverted attempts. Atomic rollback preserves the seller's NFT and prevents partial listing state, so the behavior does not demonstrate theft, stranded custody, or a financial settlement failure; owners can still pursue other transfer or sale paths.

Because the receiver policy is standards-permitted but optional and the demonstrated effect is limited to atomic listing failure, this is an unscored supported-integration compatibility observation.

Recommendation

Introduce a dedicated inbound escrow flow. Before requesting a safe transfer, record short-lived expected-transfer state for the collection, token ID, seller, and operator, and accept `onERC721Received` only when every callback field matches the active listing operation.

Keep outbound delivery separate and review delisting and settlement behavior before changing the shared transfer routine. An unconditional receiver callback could accept untracked deposits, while using safe transfers for every outbound flow could let buyers or sellers reject delivery. If this compatibility class will not be supported, narrow the documented scope explicitly.

Validation objective

Verify that all five listing types succeed for an otherwise transferable receiver-checking ERC-721 while conventional listing, delisting, purchase, and settlement flows remain functional. Confirm that unsolicited safe transfers and callbacks with an incorrect collection, token, sender, or operator are rejected, batch listing remains atomic, and receiver callbacks cannot reenter listing or settlement state.

OBS-008: Uncapped native payout callbacks can starve subsequent mint execution

- ID: OBS-008
- Status: New
- Class: Observation
- Location: TransferHelper._safeTransferETH()

Summary

Native-currency purchases can run out of gas after an accepting payout receiver consumes nearly all gas forwarded to it, preventing the later mint and wasting the buyer's transaction fee. The purchase and all value transfers revert, so the buyer retains the purchase principal and no persistent asset loss was demonstrated.

Description

Expected behavior

Native payout callbacks should not consume the gas needed to complete protocol payment and minting. If strict immediate settlement is retained, a failed payout may revert the purchase, but receiver execution should remain predictably bounded. Pending purchases should also be protected from material payout-receiver changes or be able to detect them.

Observed behavior

An authorized collection administrator can replace each drop's EOA payout receiver with an accepting gas-consuming contract without advancing the drop round (TLStacks1155 receiver update, TLStacks721 receiver update). A pending purchase can therefore continue to satisfy its expected-round, currency, and maximum-price checks despite that dependency change.

With a 600,000-gas transaction budget, the regular native-currency purchase paths for both Stacks variants reach their later `externalMint` calls after the payout callback returns, then run out of gas. Supply changes, NFT balances, and buyer value transfers all roll back. EOA receiver controls succeed with the same budget, and the gas-consuming receiver succeeds with 10,000,000 gas, isolating callback gas consumption rather than explicit receiver rejection.

Root cause

`TransferHelper._safeTransferETH()` forwards native value through an uncapped low-level call, allowing a payout receiver to consume gas needed by subsequent settlement and minting.

Under EIP-150, a call requesting all available gas receives at most all but approximately one sixty-fourth of the caller's remaining gas. A receiver can consume nearly that allocation and still return successfully, leaving only the retained fraction and any unused residue. The native branches invoke the helper before later protocol-payment and external-mint work in TLStacks1155 settlement and TLStacks721 settlement; `nonReentrant` prevents reentry but does not reserve downstream gas.

Conditions and assumptions

The condition requires a native-currency purchase with a positive payout, a contract receiver that consumes most forwarded gas while returning successfully, and a transaction budget insufficient for both the callback and the remaining settlement and mint work. Zero-value transfers are skipped.

An NFT owner or NFT `ADMIN_ROLE` holder can configure a per-drop receiver; only the Stacks owner can configure the global receiver. A previously configured upgradeable receiver could later acquire gas-consuming behavior without another Stacks configuration change, although the supplied evidence identifies no such receiver. The global path applies only when the aggregate protocol payment is positive.

Impact

The supported harm is a failed mint after the buyer expends transaction gas. The buyer retains the purchase principal because the entire purchase reverts, and provisional state, prior payouts, selectable-index changes, partial loop mints, and value transfers roll back atomically. The evidence does not establish asset loss, corrupt persistent state, or permanent sale unavailability.

A per-drop receiver limits disruption to its ERC-721 collection drop or ERC-1155 collection/token pair. A gas-consuming global protocol receiver can affect qualifying native purchases across its Stacks instance when the aggregate protocol payment is positive. Authorized actors can rotate the receiver or close the drop, and a sufficiently large transaction budget may avoid the failure.

This is an unscored observation because the demonstrated harm is bounded buyer gas expenditure and temporary mint disruption, depends on privileged receiver configuration, and has an administrative recovery path.

Recommendation

Cap the gas forwarded by native mint-payout calls and reserve a tested budget for all remaining protocol-payment and mint operations. If strict immediate settlement must remain, revert when a capped payout call fails. If stronger purchase availability is preferred, a deferred credit or pull-payment design can decouple receiver behavior from purchase completion, but it introduces custody, accounting, withdrawal, and compliance considerations.

Choose the cap with legitimate smart-contract receivers and quantity-dependent mint costs in mind. Also consider pinning the expected payout receiver or advancing a buyer-visible configuration version when it changes, so pending purchases can reject an unexpected dependency change.

Validation objective

Verify that an accepting gas consumer cannot exhaust gas reserved for subsequent settlement and minting, while EOAs and supported contract receivers continue to receive exact payouts. Cover the selected strict-revert or credit behavior, per-drop and global receivers in both Stacks variants, selectable ERC-721 purchases, receiver changes, and realistic minimum and maximum mint quantities under the supported target-chain gas schedule.

OBS-009: Winning contracts may lack a usable path for auction awards

- ID: OBS-009
- Status: New
- Class: Observation
- Location: TLAuctionHouse.bid(), TLAuctionHouse._transferNft()

Summary

A contract that wins an auction can receive the NFT and finalize the seller's payment even when it cannot operate or recover the award. Because bidding fixes delivery to the bidder and settlement does not check ERC-721 recipient acceptance, the NFT can become permanently stranded after the listing is erased without a recovery claim.

Description

Expected behavior

Auction settlement should not finalize payment and clear the listing while leaving an award at a contract that neither accepted nor can redirect it. Contract bidders should be able to nominate a capable destination or enter a defined recoverable state without allowing a malicious winner to block settlement indefinitely.

Observed behavior

During TLAuctionHouse.bid(), the auction house sets both `highestBidder` and `recipient` to `msg.sender`, so a contract bidder cannot nominate a separate award address. After expiry, TLAuctionHouse.settleAuction() deletes the listing and completes the sale to the highest bidder.

Settlement calls TLAuctionHouse._transferNft(), which uses ordinary `transferFrom` and considers delivery successful when `ownerOf(tokenId)` reports the destination. A recipient-hook rejection therefore does not make delivery revert; after the transfer, TLAuctionHouse._completeSale() distributes the proceeds. The reproduction confirms that the same contract rejects safe delivery but still becomes the owner through settlement while the seller is paid, the listing is erased, and no NFT recovery claim is created.

Root cause

TLAuctionHouse binds the award recipient to the bidding caller and treats nominal ERC-721 ownership as sufficient delivery without checking whether a contract winner can accept or operate the token.

Ordinary `transferFrom` does not perform receiver acceptance, and the ERC-721 interface warns callers to confirm recipient capability (`dependencies/@openzeppelin-contracts-5.6.1/token/ERC721/IERC721.sol:75–91`). The auction house nevertheless uses that transfer result as the boundary for finalizing the sale.

The recovery path only covers auctions that remain unsettled: TLAuctionHouse.cancelUnsettledAuction() creates a seller NFT claim after the grace period, and TLAuctionHouse.claimNft() requires that claim. A successfully transferred award bypasses this recovery state.

Conditions and assumptions

The condition requires a valid auction, a successful ordinary ERC-721 transfer, and a contract that voluntarily bids and remains the winner through expiry. Permanent stranding further requires the winner to lack generic execution, upgradeability, transfer or approval methods, prior approvals, administrative seizure, and collection-specific recovery paths; a missing or reverting `onERC721Received` implementation alone is insufficient.

The bidder chooses and funds this exposure. No privileged role or race is required once these conditions exist, and any address may settle after expiry. Neither the seller nor a permissionless settler can substitute another destination for an honest bidder, and ordinary `transferFrom` behavior remains ERC-721-compliant.

Impact

An affected winning contract nominally owns the NFT but cannot use, approve, or transfer it, while the seller retains the finalized payment. Under the stated incapability assumptions, the award is permanently stranded.

The effect is bounded to one auction and results from the bidder's own integration choice; no third party can redirect the award to itself. The proof does not establish third-party loss or attacker profit, so this is an unscored recipient-compatibility observation rather than a scored security finding. Its economic magnitude depends on the NFT and bid value.

Recommendation

Let bidders bind a distinct award recipient and recovery authority when bidding, then use acceptance-checked delivery for outbound awards. If delivery fails, keep the NFT in a defined escrow or redirectable-claim state rather than simultaneously finalizing payment, erasing the listing, and leaving no recovery claim.

Specify when seller payment becomes final, who may redirect delivery, and how timeout or unwind behavior works. Prevent a deliberately rejecting winner from blocking settlement indefinitely or recovering its bid after seller payment. Replacing the shared transfer helper globally with `safeTransferFrom` alone would not resolve these liveness and griefing tradeoffs; scope the change to outbound awards and preserve intended inbound escrow and cancellation behavior.

Validation objective

Regression tests should verify that a rejecting, NFT-incapable winner cannot create a paid, cleared, and unrecoverable state; externally owned accounts and compatible contract recipients still settle; failed delivery leaves NFT ownership, bid, listing, payout, and claim state consistent; and a rejecting recipient cannot indefinitely deny settlement or recover its bid after seller proceeds become final. They should also preserve intended behavior across inbound escrow, cancellation, and supported auction and currency variants.

OBS-010: Fixed-price NFT recipients are not checked for acceptance capability

- ID: OBS-010
- Status: New
- Class: Observation
- Location: TLAuctionHouse.buyNow()

Summary

A fixed-price purchase can pay the seller and remove the listing even when the selected contract recipient rejects safe ERC-721 delivery or is the auction house itself. The NFT can consequently become inaccessible without a recovery claim.

Description

Expected behavior

A successful fixed-price sale should establish that a contract recipient accepts the NFT or retain an effective recovery controller. If delivery establishes neither property, the purchase should revert atomically without releasing payment or deleting the listing.

Observed behavior

A safe delivery from auction-house custody to a minimal recipient that rejects the receiver callback reverted and preserved both custody and the live listing. The ordinary ETH `BUY_NOW` path instead assigned the NFT to that recipient, spent the buyer's full payment, paid the seller, removed the listing, and created no recovery claim.

A second purchase selected the auction house itself as recipient. The NFT never left auction-house custody, but settlement still paid the seller, removed the listing, and left no recovery record for `claimNft()` to process, so an attempted claim reverted. This contradicts the stated custody invariant that auction-house custody corresponds to a live listing or recovery claim.

Root cause

`TLAuctionHouse.buyNow()` completes fixed-price settlement without establishing that the selected recipient can accept or control the NFT.

The recipient validation checks only the zero address and sanctions status. Settlement then reaches `_completeSale()`, whose transfer helper uses ordinary `transferFrom` to prevent recipient rejection and treats `ownerOf(tokenId) == recipient` as successful delivery. Address assignment establishes neither ERC-721 receiver acceptance nor practical control; the ERC-721 interface (`dependencies/@openzeppelin-contracts-5.6.1/token/ERC721/IERC721.sol:76–91`) places responsibility for recipient capability on callers of `transferFrom`.

The helper serves inbound escrow, auction awards, and immediate fixed-price delivery. Although preventing rejection addresses delayed-settlement concerns for auction awards, an immediate fixed-price purchase can roll back atomically, and allowing the auction house as recipient lets a self-transfer satisfy the ownership check without changing custody.

Conditions and assumptions

Execution requires a valid, open fixed-price listing; matching listing ID, currency, and maximum price; successful payment; unsanctioned parties; where applicable, no prior auction bid; and an ERC-721 whose ordinary transfer succeeds and reports the requested recipient.

For ordinary buy-now and reserve-auction-plus-buy-now sales, the payer selects the destination; for private sales, the seller does. The evidence does not show an unrelated party redirecting an honest buyer's NFT.

NFT inaccessibility at another contract recipient depends on that contract lacking generic execution, upgradeability, an existing operator, or collection-specific recovery. A custom ERC-721 may reject a self-transfer or provide privileged rescue, and receiver acceptance alone does not ensure later operability. The transfer behavior is ERC-721-compliant; the concern is application-level settlement safety.

Impact

A participant can complete a purchase whose NFT is unusable while the payment and seller payout remain final. In the auction-house-recipient case, the NFT remains in contract custody without a live listing, recovery claim, arbitrary execution facility, or collection-level recovery path in the demonstrated configuration.

For other contract recipients, permanent loss depends on their actual recovery capabilities. The harm is bounded to the affected NFT and purchase; the evidence does not demonstrate theft, involuntary redirection, protocol insolvency, or a profitable attacker path.

The self-selected, transaction-bounded exposure makes this an unscored recipient-safety observation rather than a scored security finding, while still warranting integration hardening and recipient guidance.

Recommendation

Use receiver-aware transfer for immediate fixed-price delivery, or establish an explicit payer- or recipient-controlled recovery mechanism when non-safe delivery must remain. Reject the auction house itself as a final recipient unless settlement records a usable claim.

Separating this branch from the shared helper preserves the option to retain non-safe semantics for escrow ingress and delayed auction settlement. Receiver-aware delivery would prevent direct transfers to contracts that intentionally omit `IERC721Receiver`, and its callback remains an external interaction; preserve atomic rollback and the existing reentrancy protections.

Validation objective

Regression tests should verify that rejecting receivers, non-receiver contracts, and the auction-house address either cause settlement to revert with the listing and payment unchanged or receive an effective recovery claim. EOAs and valid ERC-721 receivers should continue to settle successfully across all three fixed-price listing types and both ETH and ERC-20 payment. After every successful sale, auction-house NFT custody should correspond to a live listing or usable claim, including for custom tokens that reject self-transfer or implement unusual transfer behavior.

OBS-011: Expired listings can be renewed in removed currencies

- ID: OBS-011
- Status: New
- Class: Observation
- Location: TLAuctionHouse.updateListing()

Summary

An original seller can reopen an expired, unbid scheduled auction after its ERC-20 currency has been removed from the allowlist. The renewed listing can accept a new bid and settle in that currency even though a fresh listing would be rejected, bypassing the protocol owner's current admission policy.

Description

Expected behavior

Removing an ERC-20 from the allowlist should block new sale admission without invalidating unchanged existing sales. This grandfathering boundary is documented in CurrencyAllowlist.sol and covered by an existing-sale regression.

The update documentation states that a new configuration follows the same requirements as `list`, including current currency approval in TLAuctionHouse.list() and updateListing(). A material update that reopens an expired auction should therefore follow the same admission rule.

Observed behavior

After the protocol owner removes the ERC-20, the expired auction rejects its first bid with `AuctionEnded`, and a fresh listing in that currency reverts with `CurrencyNotAllowed`. The original seller can nevertheless update the expired auction with a future opening, new reserve price, new duration, and new listing ID.

Once the renewed auction opens, a bidder can fund it with the still-removed ERC-20. The auction then settles normally, transferring the NFT to the bidder and paying the seller.

Root cause

TLAuctionHouse.updateListing() does not revalidate the retained currency against the current allowlist before rebuilding the listing.

The function checks seller authority, sanctions status, prior bids, listing-type equality, and currency equality before invoking `_buildListing`. Unlike TLAuctionHouse._list(), it never applies CurrencyAllowlist._isCurrencyAllowed(). TLAuctionHouse._buildListing() then replaces the configurable sale terms and assigns a new listing ID.

Conditions and assumptions

The path requires a scheduled auction created while the ERC-20 was allowed, followed by expiry without a bid while the NFT remains escrowed and the currency is removed. The original, unsanctioned seller must update the listing while the auction house is unpaused; the NFT, seller, listing type, and currency must remain unchanged. Any prior bid prevents the update, and a global pause blocks both renewal and bidding.

Completing the renewed lifecycle also requires a funded bidder who explicitly accepts the new listing ID, currency, and terms. The bidder is not silently moved onto the replacement terms.

Impact

The removed-currency admission control can be bypassed for an already-escrowed NFT. An expired sale that could no longer accept bids can receive a new executable lifecycle and newly escrow bidder funds in the removed token.

The direct scope is limited to the original seller's existing unbid listing, retaining the same NFT, listing type, and currency. If removal followed a token incident, renewal could prolong exposure or complicate incident response, but that consequence is inferred.

No unsafe token behavior, involuntary bidder action, or realized financial loss was demonstrated. The result is therefore an unscored, actionable configuration-boundary observation.

Recommendation

Apply the current currency-allowance validation before `updateListing` rebuilds sale terms and assigns a new listing ID. This would make admission-equivalent reconfiguration follow the same rule as a fresh listing while keeping execution of unchanged grandfathered terms separate.

A universal check would also prevent sellers from changing prices or schedules while a currency remains removed. If limited grandfathered updates are required, define a narrower policy that still prevents reopening an expired auction or otherwise creating a new executable lifecycle; payout-only changes can remain available through the separate payout-receiver update path.

Validation objective

Create a scheduled auction in an allowed ERC-20, let it expire without a bid, remove the currency, and verify that material renewal is rejected without modifying the existing listing. Also verify that unchanged grandfathered listings follow the intended policy, updates using allowed currencies still work, native-currency listings remain unaffected, and restoring the ERC-20 allowance re-enables valid reconfiguration.

OBS-012: Unbounded selectable supply can make aggregate bitmap reads unavailable

- ID: OBS-012
- Status: New
- Class: Observation
- Location: TLStacks721.configureDrop()

Summary

Consumers that request a selectable drop's complete minted-index bitmap can lose access to that view after an authorized collection controller configures a sufficiently large supply. `TLStacks721` accepts the supply without a ceiling, while its aggregate getter allocates and reads the entire supply-derived bitmap.

Description

Expected behavior

Every accepted selectable-drop configuration should leave supported read interfaces usable within a defined, bounded work and response budget, or expose a bounded pagination mechanism. A valid configured supply should not make the aggregate minted-index view impossible to materialize.

Observed behavior

For a selectable supply of 3,000,000, configuration consumed 204,908 gas, while the aggregate read returned 11,719 words, encoded 375,072 bytes, and consumed 31,011,253 gas. The same read failed without return data under an explicit 10,000,000-gas allowance, while `isIndexMinted` remained usable.

The contract also accepted `initialSupply = 2^72 - 255`, which makes the requested array contain `2^64` words. Under Solidity 0.8.30, allocation reverted with `Panic(0x41)`, and natural drop expiry neither cleared the configured supply nor restored the aggregate getter.

Root cause

`TLStacks721.configureDrop()` accepts a selectable drop without bounding the supply that controls aggregate bitmap size.

The function checks that `initialSupply` equals the remaining `supply` but imposes no numerical ceiling. `TLStacks721.getMintedIndices()` later computes `ceil(initialSupply / 256)`, allocates an array of that length, reads every bitmap word, and returns the complete array; its work, memory allocation, and response size therefore scale with the unrestricted stored value even when no indices have been minted.

Conditions and assumptions

For a legitimate collection, configuration requires the collection owner or an `ADMIN_ROLE` holder; Stacks must be approved as a mint contract, the existing drop must be fresh or ended, and configuration must occur while Stacks is unpaused. The caller and payout receiver must pass sanctions checks, and the payout and currency settings must be valid.

A permissionless caller can obtain equivalent authority only over a compatible collection it controls, so the demonstrated effect remains self-scoped in that case. Cap-independent failure requires `initialSupply` of at least `2^72 - 255`; lower values may still exceed a particular consumer's bounded query budget.

Impact

At sufficiently large accepted supplies, consumers that rely on the aggregate getter can lose the affected collection's complete bitmap view and must reconstruct it from `IndicesMinted` events or issue bounded `isIndexMinted` reads. A cooperative collection owner or administrator can use `TLStacks721.closeDrop()` to restore an empty result for the current round.

This is an unscored observation because the effect is confined to an optional, tenant-local view, requires authorized configuration and an exceptional supply for cap-independent failure, and has bounded alternatives and cooperative recovery. No asset loss, mint failure, cross-collection impact, or shared-service disruption was demonstrated.

Recommendation

Replace the all-at-once bitmap export with a word-range or cursor-based getter and enforce a maximum page size. If the aggregate interface must remain, reject selectable supplies above a documented ceiling selected so every accepted configuration stays within the supported query budget.

Pagination requires consumers to reconstruct the bitmap from multiple pages, while a hard supply ceiling constrains very large collections. Either approach should bound the amount of state a single supported read can allocate and traverse.

Validation objective

Verify exact bitmap reconstruction across page and 256-bit word boundaries, maximum-page rejection, and reads for empty, active, ended, closed, and reconfigured rounds. If a supply ceiling is adopted, test the largest accepted value and rejection at one greater; in either design, confirm that every accepted state remains readable without allocation panic and that scalar reads and selectable purchases retain their existing semantics.

OBS-013: Presale documentation omits the required recipient pre-hash

- ID: OBS-013
- Status: New
- Class: Observation
- Location: TLStacks721.updateDropPresaleMerkleRoot(); TLStacks1155.updateDropPresaleMerkleRoot()

Summary

Presale proofs generated from the documented `(address,uint256)` leaf are rejected by both mint contracts, preventing eligible ERC-721 and ERC-1155 purchases until an authorized operator replaces the root and redistributes proofs. The verifiers require the recipient address to be hashed before it is encoded with the allowance, a step omitted from the root-configuration documentation.

Description

Expected behavior

The public presale leaf specification should identify the exact ABI types and every hashing step used by the on-chain verifiers. A proof built independently from that specification should authorize the intended recipient and allowance across regular ERC-721 and ERC-1155 presales and selectable ERC-721 presales, while altered recipients or allowances remain invalid.

Observed behavior

The root-parameter documentation in TLStacks721 and TLStacks1155 says that each leaf encodes the recipient and permitted mint count. Under the documented direct tuple, the double-hashed leaf is `keccak256(bytes.concat(keccak256(abi.encode(recipient, allowance))))`.

The ERC-721 validator and ERC-1155 verifier instead compute `recipientHash = keccak256(abi.encode(recipient))` and then hash `abi.encode(recipientHash, allowance)`. Changing the first ABI field from an `address` to its `bytes32` hash changes the resulting leaves, roots, and proofs.

The reproductions show the direct tuple being rejected with `NotOnAllowlist` and the same entitlements succeeding after using the recipient-prehashed schema. The ERC-721 manifestation covers paid regular and selectable purchases, the ERC-1155 manifestation isolates a root-only correction, and a two-leaf cross-contract variant confirms the same incompatibility in both mint contracts. Rejection occurs before supply or mint accounting, settlement, and minting in the respective ERC-721 and ERC-1155 paths.

Root cause

The `TLStacks721` and `TLStacks1155` presale integration boundary documents an `address` leaf field while both verifiers substitute a separately hashed `bytes32` recipient.

No canonical public leaf-hash helper, maintained generator, or fixed vector exposes the additional transformation. The package's ERC-721 and ERC-1155 fuzz tests reproduce the implementation formula, so they establish internal consistency without independently checking interoperability with the documented direct tuple.

Conditions and assumptions

The condition applies when an NFT owner or NFT `ADMIN_ROLE` holder configures an active presale for a regular ERC-721 or ERC-1155 drop, or a selectable ERC-721 drop, from the literal documented tuple and a buyer submits the corresponding proof. The Stacks contract must be approved to mint the NFT. Velocity drops cannot have a presale, and public-sale minting does not use this proof.

Integrations already using the recipient-prehashed formula are unaffected. Practical exposure depends on whether an operator's tooling relies on the documented direct tuple and whether the root and proofs can be corrected and redistributed before time-sensitive presale access expires.

Impact

An incompatible root denies intended presale purchases that use the corresponding direct-tuple proofs. Included failed transactions can cost buyers gas, while diagnosing the mismatch, rebuilding the tree, redistributing proofs, and replacing the root can delay sale proceeds and cause buyers to miss time-sensitive presale access.

The demonstrated transactions revert atomically before payment, supply or allowance consumption, and minting. They do not retain purchase funds or grant unauthorized access, and an authorized operator can replace the root before an eligible drop ends. This conditional, recoverable integration failure supports treatment as an unscored observation rather than a scored security finding.

Recommendation

Retain the recipient-prehashed format to preserve compatibility with any roots and proofs generated for the current verifier, and make that format explicit across the public API and supported tooling. Document the exact ABI types, recipient pre-hash, inner and outer hashing, pair ordering, and tree conventions; provide a canonical `hashPresaleLeaf(address,uint256)` helper or maintained builder with fixed singleton, even-leaf, and odd-leaf vectors; and check a sample proof before activating a root.

If a future version instead adopts direct `(address,uint256)` leaves, introduce an explicit schema version and migration boundary because changing the verifier would invalidate roots and proofs generated under the current format. Derive regression expectations independently from the published specification rather than copying the verifier expression.

Validation objective

Verify that the documentation, supported generator, any public helper, and both on-chain verifiers produce identical fixed vectors. Correct recipient-and-allowance proofs should mint through regular ERC-721 and ERC-1155 presales and selectable ERC-721 presales; altered values should fail. Cover singleton, even-leaf, and odd-leaf trees, native-currency and ERC-20 purchases, pre-activation sample-proof checks, and either compatibility or an explicit migration result for roots generated under the current schema.

OBS-014: External callbacks can make listing events report stale receivers

- ID: OBS-014
- Status: New
- Class: Observation
- Location: `TLAuctionHouse.bid()`, `TLAuctionHouse._list()`, and `TLAuctionHouse.topUpBid()`

Summary

A seller-authorized payout-receiver update made during an NFT transfer, native-currency refund, or ERC-20 escrow callback can be followed by a full-listing event that reports the previous receiver. Authoritative storage, escrow, and settlement remain correct, but an event-sourced integration that treats the last full `Listing` payload as authoritative could regress to stale state.

Description

Expected behavior

After callback-capable external interactions complete, a full `Listing` event should represent the authoritative listing state. If an authorized nested update emits `ListingUpdated`, no later event in the same transaction should replace it with an older snapshot.

Observed behavior

Three callback paths allow the recorded seller to call `TLAuctionHouse.updatePayoutReceiver()` during an outer operation. In the native-auction path, a contract seller that previously bid on its own listing receives an outbid refund; its callback emits `ListingUpdated` with the new receiver before `bid()` emits `AuctionBid` with the old receiver. Final storage, the new highest bid, the refund, NFT delivery, and settlement to the new receiver remain correct.

During initial NFT escrow, custom ERC-721 transfer logic can notify the contract seller after `_list()` stores the listing. The callback emits `ListingUpdated` with the new receiver, but `_list()` then emits `ListingConfigured` from the original object. Both `list()` and `batchList()` use `_list()`; the proof invokes `list()` directly, while storage, custody, and the later payment use the new receiver.

During an ERC-20 top-up, a sender callback can make the same authorized update while `TransferHelper._safeTransferFromERC20()` pulls the additional escrow. `topUpBid()` then emits `AuctionBidToppedUp` with the increased bid but the old receiver; storage records the new receiver and the full top-up is escrowed.

Root cause

Across `TLAuctionHouse.bid()`, `TLAuctionHouse._list()`, and `TLAuctionHouse.topUpBid()`, the contract emits a cached `Listing` after an external interaction that can validly change `payoutReceiver`.

`bid()` and `topUpBid()` retain pre-callback memory copies through native-currency or ERC-20 transfers, while `_list()` retains the struct created before the NFT transfer. The seller-only receiver update remains callable during these interactions and updates storage, so the nested event is fresh while the later outer event is stale. The outer reentrancy guards therefore do not preserve event-snapshot consistency across this setter boundary.

Conditions and assumptions

All variants require an unpaused operation, the stored seller to be a callback-reachable contract, applicable sanctions and recipient checks to pass, and the callback to succeed. A revert cannot leave the update or event sequence.

The native-refund path requires the seller to self-bid, be the previous highest bidder, and be validly outbid. The callback must fit the 100,000-gas native payout allowance; failure or exhaustion rolls back nested changes, and the payout logic credits the refund instead.

The listing path requires custom ERC-721 transfer logic that notifies the seller; ordinary OpenZeppelin `transferFrom()` (`dependencies/@openzeppelin-contracts-5.6.1/token/ERC721/ERC721.sol:115–136`) has no receiver hook, although the auction house claims compatibility with ERC-721-compliant tokens (`README.md:23–24`). The top-up path requires an owner-vetted, allowlisted callback-capable ERC-20, an active auction, sufficient balance and allowance, and one contract acting as seller and highest bidder. Native top-ups and ordinary ERC-1363 `transferFrom()` calls do not invoke the modeled hook.

Impact

The concrete effect is an internally inconsistent event stream: the last full `Listing` event in a transaction can report an obsolete payout receiver even though an earlier `ListingUpdated` and authoritative storage are correct. A chronological indexer or application that replaces its model from each full payload could therefore display or retain stale receiver data.

No relying consumer or downstream harm was demonstrated. The proofs preserve on-chain authorization, listing state, refund and escrow accounting, NFT custody or delivery, payment, and settlement; they do not establish asset loss, payout diversion, or an accounting error.

Because the supported harm is an integration and event-integrity hazard rather than a demonstrated security loss, this is an unscored observation.

Recommendation

Build these full-`Listing` event payloads from authoritative post-interaction storage. Reload the listing after callback-capable native refunds, NFT escrow, and ERC-20 transfers before emitting `AuctionBid`, `ListingConfigured`, or `AuctionBidToppedUp`; alternatively, redefine the events to carry only fields owned by the outer operation.

For listing, emitting `ListingConfigured` before the external NFT transfer would also keep a nested `ListingUpdated` chronologically last, while a failed transfer would still revert the logs. A shared reentrancy policy for `updatePayoutReceiver()` could instead reject callback-time changes, but it would restrict existing seller behavior and callback composability. Preserve the current seller, recipient, refund, escrow, and settlement checks whichever policy is selected.

Validation objective

Exercise a native outbid refund, custom ERC-721 transfer logic through single and batch listing, and an ERC-20 sender callback during a top-up. For each path, verify that the callback-time update is explicitly rejected or that the trailing full-listing event matches `getListing()` and cannot supersede a newer `ListingUpdated` with stale data. Also verify correct

refund or credit behavior, escrow and bid state, NFT custody and delivery, payment and settlement to the authoritative receiver, full rollback where the outer transfer reverts, and unchanged ordinary token behavior.

OBS-015: Drop lifecycle events omit round identity

- ID: OBS-015
- Status: New
- Class: Observation
- Location: `ITLStacks721Events` lifecycle events; `ITLStacks1155Events` configuration, update, and closure events; `ITLStacks1155Events.Purchase`

Summary

Off-chain consumers cannot determine a drop's lifecycle round from its configuration, update, purchase, or closure event alone because the event schemas omit the round. Otherwise identical actions in successive rounds can produce byte-identical event fields and may be conflated by a partial-history consumer, although on-chain round enforcement remains intact and no affected consumer or resulting harm was demonstrated.

Description

Expected behavior

For self-contained lifecycle telemetry, each `DropConfigured`, `DropUpdated`, `Purchase`, and `DropClosed` record should bind its payload to the round it describes. Configuration and update records should report the current stored round, purchase records the validated `expectedRound`, and closure records the pre-increment round that was closed.

Observed behavior

An authorized actor can configure the same terms for a resource in successive rounds, and real allowance changes can produce the same resulting drop terms in each round. The ERC-721 configuration and update entrypoints, and the corresponding ERC-1155 configuration and update entrypoints, emit byte-identical event fields when all serialized values match, even though storage identifies different rounds.

Purchases exhibit the same gap. The ERC-721 `purchase` and ERC-1155 `purchase` paths accept and enforce `expectedRound`, yet otherwise identical purchases in rounds 0 and 1 can emit the same `Purchase` fields.

Closure records are also roundless: `TLStacks721.closeDrop()` and `TLStacks1155.closeDrop()` advance round state but omit the closed round from `DropClosed`. Authorized closures of rounds 0 and 1 can therefore have identical emitters, topics, and data while storage advances to round 2. Blockchain coordinates still distinguish each occurrence; the ambiguity concerns the event's encoded semantic fields.

Root cause

The `ITLStacks721Events` lifecycle schemas, the `ITLStacks1155Events` configuration, update, and closure schemas, and its `Purchase` schema discard the round discriminator at the logging boundary.

`TLStacks721` stores round state separately by NFT address, while `TLStacks1155` stores it by NFT address and token ID in their respective ERC-721 and ERC-1155 mappings. Configuration and update emissions serialize the resource key and drop terms without that state; purchase emissions do not forward the already-validated round; closure emissions do not capture

the round before incrementing it. Identical serialized inputs therefore yield identical event content across distinct lifecycle generations.

Conditions and assumptions

The condition applies to the custom TLStacks721 and TLStacks1155 lifecycle events when the same Stacks emitter and NFT resource are reused across successive rounds. An exact collision requires every logged field to match, including the token ID for ERC-1155. Configuration, update, and closure require the NFT owner or an `ADMIN_ROLE` holder, along with the applicable sanctions, mint-approval, pause-state, drop-validity, and lifecycle checks; purchases require no administrative role once a drop is active.

A consequential misattribution additionally assumes an off-chain consumer treats isolated, incomplete, or out-of-order event content as a complete sale identity while ignoring ordered lifecycle history, transaction calldata or traces, and canonical log coordinates. Complete ordered history can reconstruct the round, and the current-round getters expose current state but do not label historical records. No relying consumer or documented requirement for self-contained events was supplied.

Impact

A partial-history indexer, webhook, analytics pipeline, accounting system, or fulfillment service could assign a configuration, update, purchase, or closure to the wrong round, conflate records, or discard a later record as a duplicate. This consequence is inferred: no affected consumer or resulting malfunction was demonstrated.

The practical scope is limited to event observability. Round-keyed storage and `expectedRound` validation continue to govern on-chain accounting and purchases; complete ordered history can reconstruct round attribution, purchase calldata exposes `expectedRound`, and canonical log coordinates distinguish individual occurrences. The condition is therefore an unscored observation rather than a scored security finding; no on-chain control failure or security or functional harm was established.

Recommendation

Introduce versioned, round-bearing forms of each affected lifecycle event. Emit the current stored round for configuration and updates, the validated `expectedRound` for purchases, and the captured pre-increment round for closures; index the value only where filtering requirements justify the additional topic.

Changing an existing event signature changes its signature topic and requires coordinated ABI, indexer, and client migration. If compatibility is material, emit a new event version or temporarily dual-emit the legacy and round-bearing forms, accounting for additional log cost, duplicate-consumption risk, and any required historical backfill.

Validation objective

For both contracts, repeat matching configuration, update, purchase, and closure actions across successive rounds and verify that each new record reports the applicable round and no longer collides across rounds. Cover every update entrypoint and both ERC-721 purchase paths, and confirm that first-round numbering, one-step round advancement, authorization, stale-round rejection, round-keyed accounting, settlement, and minting behavior remain unchanged.

OBS-016: Deprecated sanctions sources can silently stop receiving updates

- ID: OBS-016
- Status: New
- Class: Observation
- Location: SanctionsCompliance._isSanctioned

Summary

A deployment that remains configured to the deprecated Chainalysis sanctions oracle can let an address designated after the oracle's final update pass sanctions screening and create an auction listing. The adapter accepts a Boolean with no freshness signal, so a stale negative is indistinguishable from a current clean result.

Description

Expected behavior

When sanctions screening is enabled, protected operations should reject currently sanctioned parties or fail closed when the configured source's freshness cannot be established. A stale negative should not be treated as a current clean determination.

Observed behavior

Chainalysis's oracle documentation identifies the tested Ethereum oracle, states that it is no longer maintained, and records March 18, 2026 as its last update. OFAC's May 20, 2026 notice designated `0xaC4cC4B68ea24BbFAAC8fD127B67Ed445AccCE22` after that cutoff. A block-header-bound state proof returned `false` for this address and `true` for a pre-freeze positive control.

Using the authenticated oracle runtime and state, the proof configured the real TLAuctionHouse.list path. The post-freeze address passed both seller and payout-recipient checks and transferred its NFT into auction escrow through the internal listing path, while the same gate rejected the positive control.

Root cause

`SanctionsCompliance._isSanctioned()` forwards the configured oracle's Boolean response without enforcing source freshness or lifecycle validity.

The IChainalysisSanctionsOracle.isSanctioned interface provides only a Boolean result. The adapter has no authenticated timestamp, epoch, list root, heartbeat, or maximum permitted age with which to distinguish a maintained response from stale state.

The contracts call the source live rather than caching sanctions determinations. If that source stops updating, the call continues to succeed and a missing post-freeze entry appears identical to a current negative.

Conditions and assumptions

The condition requires an instance configured to the deprecated Ethereum oracle, a target designated after the March 18, 2026 final update and still absent from the oracle's state, and no owner migration before the protected action. The caller must also meet the selected function's ordinary prerequisites; in the demonstrated path, the seller owned and approved the NFT and the auction house was unpaused.

Oracle selection and replacement are trusted owner operations. The supplied package does not establish any current deployment's configuration, monitoring, or response procedure, so present exposure and duration are unknown. A current or future instance is within scope only while it uses this deprecated source; the owner can replace it through TLAuctionHouse.setSanctionsOracle.

Impact

The directly supported harm is a sanctions-policy bypass: a post-freeze designated seller can create a listing and place its NFT into escrow despite the enabled gate. Because the seller owns and approves the NFT, the evidence establishes neither theft nor unauthorized asset control.

Other paths sharing `_isSanctioned` will also trust stale negatives when their ordinary prerequisites hold. Representative payer, recipient, and payout checks appear in TLStacks721._validatePurchase and TLStacks1155.purchase, but these broader effects are source-traced inferences rather than individually executed outcomes.

This is an unscored observation because the risk depends on a trusted owner retaining the deprecated dependency, while the evidence establishes neither an affected deployed configuration nor a financial consequence. It remains actionable dependency-lifecycle and monitoring guidance.

Recommendation

Replace any deprecated oracle configuration and enforce freshness at the sanctions-adapter boundary. A maintained source or adapter should expose an authenticated update time, epoch, or list root and enforce a maximum age, causing protected operations to fail closed or pause when freshness cannot be established.

Keep oracle replacement and emergency recovery callable without consulting the stale or unavailable source. Because fail-closed behavior can reduce availability during an outage, define the maximum age and recovery procedure to balance that cost against accepting unverifiable negatives.

Validation objective

Verify that a fresh negative permits an otherwise valid action, a fresh positive is rejected, and a stale or freshness-unknown negative cannot pass. Confirm that the owner can replace an unavailable source without invoking it and that representative auction, ERC-721 mint, and ERC-1155 mint paths enforce the same freshness property.

OBS-017: Decay-rate documentation can lead administrators to disable velocity sales

- ID: OBS-017
- Status: New
- Class: Observation
- Location: TLStacks721.updateDropDecayRate(), TLStacks1155.updateDropDecayRate()

Summary

An administrator who follows the copied `decayRate` parameter description can store a Merkle-root-sized value that makes subsequent velocity purchases revert on checked duration arithmetic. This can interrupt mint availability and associated sales for the affected drop or ERC-1155 token ID until an authorized administrator corrects the rate.

Description

Expected behavior

The `decayRate` documentation should describe the parameter's signed, per-token duration semantics. Values accepted through the configuration interface should not make otherwise valid purchases panic during duration adjustment.

Observed behavior

The parameter descriptions in TLStacks721.updateDropDecayRate() and TLStacks1155.updateDropDecayRate() instead identify `decayRate` as a presale Merkle root, while both setters accept any supplied `int256` without checking whether later multiplication or cumulative duration addition is safe.

A valid one-leaf root can fall below `int256.max` while exceeding `uint256.max / 3`. After an authorized administrator stored such a root as a positive rate, two one-token purchases succeeded in each implementation and the third reverted with a checked-arithmetic panic. Resetting the rate to zero allowed the next purchase to succeed.

Root cause

The `TLStacks721.updateDropDecayRate()` and `TLStacks1155.updateDropDecayRate()` control points pair incorrect `decayRate` documentation with no bound on the signed value stored for later duration arithmetic.

The parameter text was copied from the adjacent presale-root setter, and the accepted rate is not coupled to the current duration, remaining supply, or permitted mint quantity. Positive rates are added to `publicDuration` in the ERC-721 duration update and ERC-1155 duration update under checked arithmetic, so a value accepted during configuration can later panic during an ordinary purchase.

Conditions and assumptions

The path requires a configured, not-ended velocity drop in its public phase. The mint contract must be unpaused, and an unsanctioned NFT owner or `ADMIN_ROLE` holder must explicitly reinterpret a low-half `bytes32` Merkle root as `int256` and set it as the rate before purchases occur.

Reliance is less likely because the function name, notice, parameter name, signed ABI type, and high-level velocity-drop description (`README.md:18–21`) contradict the copied parameter text. Velocity drops cannot have presales, while the actual ERC-721 and ERC-1155 root setters take `bytes32` and reject velocity drops.

No deployment or integration evidence establishes current exposure or actual reliance on the generated documentation. The condition applies where an administrator or integration uses that devdoc to configure the affected contracts.

Impact

In the demonstrated configuration, the third one-token purchase reverts, and the unchanged overflow condition can prevent further mints and associated sales for that drop or ERC-1155 token ID.

The failed purchase is atomic, the effect is tenant-local, and an authorized administrator can restore minting by changing the rate. No buyer fund loss or broader protocol harm is demonstrated. Because the trigger depends on exceptional privileged reliance and the configuration is recoverable, this is an unscored safe-configuration observation.

Recommendation

Correct both `decayRate` parameter descriptions to state the value's units and sign behavior, including that negative values shorten and positive values extend the public duration per minted token. Enforce a safe arithmetic invariant when configuring or updating a velocity drop: reject positive rates whose cumulative adjustment over the remaining supply can overflow `publicDuration`, and guard per-call multiplication for the permitted mint quantity.

If extreme marathon durations are intentional, explicitly capped or saturating duration arithmetic is an alternative. Any bound may limit those durations, so select and document the supported maximum deliberately.

Validation objective

Confirm that generated devdoc no longer describes `decayRate` as a Merkle root; root-sized and arithmetic-boundary inputs either fail at configuration with an explicit domain error or remain safe across all permitted purchases; and ordinary positive, negative, and zero rates preserve their intended duration behavior.

OBS-018: Recipient-retention checks can reject valid hook-enabled ERC-20 payouts

- ID: OBS-018
- Status: New
- Class: Observation
- Location: TransferHelper._safeTransferFromERC20()

Summary

Purchases using a hook-enabled ERC-20 can revert after a payout recipient validly forwards funds during the token callback. The shared transfer check mistakes the recipient's reduced end balance for underpayment, making affected ERC-721 and ERC-1155 drops unavailable and costing purchasers gas while the transaction otherwise rolls back.

Description

Expected behavior

After an allowlisted ERC-20 credits the requested amount, a payout should not fail solely because the recipient moves that credit during a post-credit callback. The payment check should still reject actual under-crediting, including fee-on-transfer behavior.

Observed behavior

A forwarding seller-payout recipient causes regular `TLStacks721` and `TLStacks1155` purchases to revert with `InsufficentERC20Transfer`. A forwarding global protocol-fee recipient causes the same failure in an ERC-721 purchase with a positive seller fee. ERC-721 settlement and ERC-1155 settlement route both payout types through the shared transfer check.

In each directly exercised failure, the payer's balance and allowance, recipient and forwarding-destination balances, NFT balance, and drop supply revert to their prior values. The otherwise-identical purchases succeed when the recipient retains the credit until `transferFrom` returns.

Root cause

`TransferHelper._safeTransferFromERC20()` treats the recipient's net balance increase after `transferFrom` as proof of the amount received.

The helper records the initial balance, makes the external token call, and requires the final balance increase to equal or exceed the requested amount. If the token credits amount `A` and invokes a recipient hook that forwards any positive amount `F` before returning, the measured increase is `A - F`; the helper therefore reports underpayment even though the recipient received the complete credit. This check enforces retention through the end of the token call rather than gross receipt.

Conditions and assumptions

The package describes the mint contracts as accepting any allowlisted ERC-20 (`README.md:11–21`), while the observed payout path specifically requires a hook-enabled token allowlisted through the owner-controlled ERC-721 or ERC-1155 currency setting. An active, positive-price drop must send proceeds to a recipient that forwards a nonzero amount inside its post-credit callback, and the payer must be distinct from that recipient with sufficient balance and allowance.

The protocol-fee path instead requires a positive, non-waived ERC-20 seller fee and a forwarding fee recipient configured through TLStacks721.setProtocolFeeSettings(). Plain ERC-20 tokens, recipients that retain funds until `transferFrom` returns, delayed forwarding, ETH payments, and the TLAuctionHouse ordinary escrow pull do not trigger the condition.

Impact

Qualifying purchases remain unavailable until the relevant configuration changes or the recipient stops forwarding during the callback. Each failed attempt consumes purchaser gas. A forwarding per-drop payout recipient affects that drop; a forwarding global fee recipient can affect every non-waived, positive-fee drop paid in the compatible hook-enabled currency on that contract instance.

The revert is atomic: no payer debit, allowance consumption, forwarded balance, NFT mint, or drop-supply change persists. This bounded, configuration-dependent availability effect is administratively reversible and has no demonstrated security loss, so it is reported as an unscored supported-integration compatibility observation.

Recommendation

Pull the full ERC-20 payment into a protocol-controlled staging balance and verify the exact inbound increase before distributing seller proceeds and protocol fees. Distribution should not use a recipient’s post-callback balance as evidence of gross receipt.

This design adds token transfers and gas and must account for callbacks, reentrancy, blocklisting, and outgoing-transfer failures. If callback-enabled currencies are intentionally unsupported instead, enforce and document that currency and recipient compatibility boundary rather than relying on the implicit retention check.

Validation objective

Regression tests should show that full-credit post-callback forwarding succeeds for seller and protocol-fee recipients while retaining recipients continue to work and actual under-crediting remains rejected. Cover regular and selectable ERC-721 purchases and ERC-1155 purchases, including correct fee splits, mint accounting, atomic rollback, zero and waived fees, and sender-equals-recipient cases.

OBS-019: Ownership renunciation while paused can permanently disable sales

- ID: OBS-019
- Status: New
- Class: Observation
- Location: TLStacks721.pause(); TLStacks1155.pause(); TLAuctionHouse.pause()

Summary

Pausing a sales contract and then renouncing ownership permanently disables sales at that contract address because no account remains authorized to unpause it. Drop administrators retain closure paths and AuctionHouse escrow can exit through its delayed recovery flow, but restoring sales may require a replacement deployment and reconstruction of contract-local state.

Description

Expected behavior

A paused sales contract should retain an authorized route to resume operations or prevent the irreversible removal of every unpause authority. A nominated successor should remain available when it is the only recovery path.

Observed behavior

`TLStacks721`, `TLStacks1155`, and `TLAuctionHouse` each combine `Ownable2Step` with `Pausable` (ERC-721 declaration, ERC-1155 declaration, and AuctionHouse declaration). An owner can call `pause(true)` and then inherited `renounceOwnership()` (`dependencies/@openzeppelin-contracts-5.6.1/access/Ownable.sol:69–78`). Renunciation dispatches to `Ownable2Step._transferOwnership()` (`dependencies/@openzeppelin-contracts-5.6.1/access/Ownable2Step.sol:48–55`), which clears the pending owner before setting the active owner to the zero address.

The pause flag remains set until `_unpause()` (`dependencies/@openzeppelin-contracts-5.6.1/utils/Pausable.sol:101–110`), and its only production call sites are the owner-only `pause(false)` branches. In the demonstrated sequence, successful ERC-721 and ERC-1155 purchases and a funded auction precede this transition; afterward, the former owner cannot unpause, the nominated successor cannot accept ownership, and purchases and settlement revert. The ownerless pause blocks ERC-721 purchases, including selectable purchases, ERC-1155 purchases, and AuctionHouse settlement, along with other pause-guarded sales operations.

Root cause

`TLStacks721.pause()`, `TLStacks1155.pause()`, and `TLAuctionHouse.pause()` make unpausing owner-only without preventing inherited ownership renunciation from clearing the last unpause authority.

The ownership and pause lifecycles are composed without an invariant tying them together. `renounceOwnership()` is neither overridden nor conditioned on pause state, and these contracts provide no secondary recovery control; clearing both active and pending owners therefore makes the existing pause irreversible at that address.

Conditions and assumptions

Only the trusted contract owner can trigger the condition by calling `pause(true)` and then `renounceOwnership()`; each affected contract instance must undergo the sequence separately. No unprivileged trigger or ownership takeover was identified. A pending owner remains a recovery option only if it accepts ownership before renunciation.

The in-scope contracts contain no proxy, secondary unpause authority, or state importer. Recovery through external storage intervention or undocumented deployment-level mechanisms falls outside the implementation assessed here.

Impact

The ownerless pause permanently removes core sales functionality at the affected contract address. For the multi-tenant mint contracts (`README.md:11–20`), configured sales stop and new configuration is blocked, although drop administrators can still close ERC-721 drops and ERC-1155 drops. Underlying NFTs are not inherently disabled if a surviving NFT owner or administrator can authorize a replacement minter, but replacement does not natively preserve sale state and may require bespoke reconstruction.

For `TLAuctionHouse`, new listings, bids, purchases, and normal settlement stop. Sellers can immediately delist unbid NFTs; for a bid-on auction, the seller or highest bidder can cancel only after the auction ends and the seven-day grace period expires (auction exits), after which funds and NFTs remain claimable while paused.

This is an unscored operational observation because only the trusted owner can invoke the avoidable sequence and ordinary auction assets retain recovery. The owner already has authority to pause indefinitely; the additional harm is permanent removal of the recovery route at that address plus replacement and state-reconstruction costs. ERC-721 and ERC-1155 drop payments settle atomically, and no permanent loss of ordinary purchaser funds or ordinary auction assets was demonstrated.

Recommendation

Enforce the lifecycle invariant that a paused deployment cannot become ownerless. Override `renounceOwnership()` in all three sales contracts and reject renunciation while paused.

Alternatives include disabling renunciation, atomically unpausing before renunciation, or retaining a separately governed recovery authority. Disabling renunciation preserves permanent administrative control, a recovery authority expands governance power, and atomic unpausing can briefly reopen user operations. Preserve the existing pause-independent drop-closure and auction-exit paths.

Validation objective

For each contract, verify that `pause(true)` followed by attempted renunciation either reverts or leaves an authorized, callable unpause path; a pending ownership handoff remains recoverable; and unauthorized callers cannot unpause. Confirm that drop closure, immediate unbid delisting, delayed auction cancellation, and claims continue to work while paused.

OBS-020: Terminal authority changes can eliminate every remaining mint-issuance path

- ID: OBS-020
- Status: New
- Class: Observation
- Location: TLStacks721.setSanctionsOracle(); TLStacks1155.setSanctionsOracle()

Summary

A collection can permanently lose access to its remaining ERC-721 or ERC-1155 drop issuance when creator ownership is transferred to a non-calling recipient, TLStacks ownership is renounced, and the sanctions oracle later fails permanently. With no alternate recovery authority, purchases revert and no demonstrated in-package route can repair the oracle, close the drop, mint directly, or migrate issuance.

Description

Expected behavior

Before an outstanding drop loses terminal administrative authority, the system should preserve at least one callable route to issue or retire remaining supply, migrate minting, or repair an external dependency. A creator ownership handoff should not silently eliminate the final creator-side administrative route before the recipient demonstrates control.

Observed behavior

The creator contracts can be initialized without administrators and use an immediate one-step ownership transfer (`dependencies/@openzeppelin-contracts-upgradeable-5.6.1/access/OwnableUpgradeable.sol:98–117`). After TLStacks becomes the sole approved minter and a drop is configured, the creator owner can transfer ownership to a contract that cannot originate or forward calls; the owner-or-admin gate (`dependencies/tl-creator-contracts-4.1.1/src/lib/OwnableAccessControlUpgradeable.sol:76–80`) then prevents the former owner from minting directly or approving a replacement minter.

TLStacks can subsequently renounce ownership (`dependencies/@openzeppelin-contracts-5.6.1/access/Ownable.sol:69–78`), yet a purchase still succeeds while the sanctions oracle is healthy because its creator-side minter role survives. If the oracle later reverts, the sanctions lookup propagates that error before minting, while the former TLStacks owner can no longer call `setSanctionsOracle()`.

`closeDrop()` remains independent of the oracle, but ERC-721 drop administration and ERC-1155 drop administration require creator owner or administrator authority. The inaccessible creator owner and lack of administrators therefore also block closure, direct minting, and approval of an alternate sales contract. For each creator variant, the proof completed one healthy purchase, then left two planned units unchanged after the failed purchase and confirmed that oracle repair, closure, direct minting, alternate-minter approval, unapproved external minting, and ownership-sink forwarding all failed.

Root cause

`TLStacks721.setSanctionsOracle()` and `TLStacks1155.setSanctionsOracle()` leave oracle recovery exclusively owner-controlled even though TLStacks ownership can be renounced after creator-side recovery authority has become unusable.

No cross-component invariant ensures that at least one of the two recovery boundaries remains usable. The creator transfer only rejects the zero address and completes without recipient acceptance; without an administrator, creator-side issuance and migration controls depend on the new owner being able to call them. TLStacks may renounce ownership while a nonzero oracle remains configured, but oracle errors fail closed. A callable creator administrator could migrate from a failed TLStacks instance, and a callable TLStacks owner could replace a failed oracle; discarding both makes a later permanent oracle failure terminal.

Conditions and assumptions

This condition requires a creator with no surviving administrator, TLStacks as its sole approved minter, and a configured drop with remaining planned supply. Creator ownership must be transferred to a non-calling or otherwise unusable recipient, and TLStacks must renounce ownership while a nonzero sanctions oracle is still configured and healthy.

The oracle must later revert permanently, with no alternate minter, proxy administrator, upgrade authority, oracle-repair route, or other off-package recovery mechanism. Both authority-discarding actions require the respective trusted owners; no permissionless route to either action was demonstrated.

Impact

Under these conditions, configured future issuance becomes indefinitely unavailable: purchases fail, while closure, direct minting, alternate-minter approval, and oracle repair are inaccessible. The proof left two planned units unchanged in both the ERC-721 and ERC-1155 cases, so the collection can lose the associated future sale opportunity.

Failed purchases revert atomically, so buyer funds are not trapped, and existing tokens, holder balances, and transfers are not directly impaired. The remaining supply represents a configured issuance plan rather than escrowed property or a hard creator-level supply entitlement; any sale-opportunity loss depends on that plan and its pricing. Broader collection-administration effects are outside this observation.

Because the condition requires two avoidable trusted authority transitions, a later permanent dependency failure, and no external recovery—and does not affect existing tokens or buyer funds—it is an unscored operational and dependency-availability observation.

Recommendation

Preserve at least one callable, narrowly scoped recovery or migration authority until configured drops are retired and the external oracle is no longer needed. Options include acceptance-based creator ownership transfers, requiring or validating an administrator or alternate minter before a terminal handoff, and either preventing TLStacks ownership renunciation while a nonzero oracle remains configured or retaining a limited oracle-repair authority. As an operational safeguard, close outstanding drops and verify creator roles before terminal authority changes.

A persistent recovery role weakens the finality of ownership renunciation, while fail-open oracle handling may conflict with sanctions requirements. The selected design should preserve the intended compliance behavior without making one external failure unrecoverable.

Validation objective

For both ERC-721 and ERC-1155 drops, verify that a non-calling creator recipient cannot silently eliminate the final creator-side authority, an active drop cannot lose both migration and oracle-repair paths, and a later oracle revert leaves at least one authorized closure, repair, direct-mint, or alternate-minter route. Failed purchases should continue to roll back atomically, and a successful oracle response should continue to enforce the intended sanctions policy.

OBS-021: The minimum signed decay rate can disable velocity purchases

- ID: OBS-021
- Status: New
- Class: Observation
- Location: TLStacks721._updateDropState(); TLStacks1155._updateDropState()

Summary

Configuring a velocity drop with `type(int256).min` causes otherwise-valid positive purchases to revert, disabling minting for that drop until an authorized administrator changes the rate or closes the drop. Buyers can lose transaction gas because both token managers accept a value their duration updates cannot safely negate.

Description

Expected behavior

Both mint managers should reject decay rates that their purchase paths cannot safely consume. For a negative rate, the duration adjustment should derive its magnitude without an unrepresentable signed operation.

Observed behavior

The ERC-721 configuration and ERC-1155 configuration paths store `type(int256).min`; their ERC-721 and ERC-1155 live setters impose no magnitude restriction.

With an active velocity drop configured this way, an otherwise-valid positive purchase through the ERC-721 or ERC-1155 path reaches `Panic(0x11)` before settlement or minting. The transaction rolls back, leaving supply, public duration, and the buyer's token balance unchanged. A buyer cannot alter the rate, while the NFT administrator can replace it with `-1` without advancing the round and then complete the purchase.

The ERC-721 edge regression separately covers the panic and recovery with nonzero call value.

Root cause

The TLStacks721._updateDropState() and TLStacks1155._updateDropState() functions unsafely negate a full-domain signed decay rate before converting its magnitude to `uint256`.

The negative-rate branch computes `uint256(-1 * drop.decayRate) * numberToMint`. The signed range is asymmetric: `-2^255` is representable, but positive `2^255` is not, so multiplying `type(int256).min` by `-1` overflows under checked arithmetic. The configuration and update paths therefore accept a wider domain than the duration-adjustment sink can process.

Conditions and assumptions

The condition applies only to an active velocity drop whose NFT contract owner or an `ADMIN_ROLE` holder stored the exact value `type(int256).min`, and to a positive purchase that otherwise passes the phase, sanctions, round, currency, price, supply, and allowance checks. Buyers cannot create or repair the condition.

The effect is confined to the addressed ERC-721 drop or ERC-1155 token ID. Phase calculation does not consume the rate, so while the drop remains active, an unpaused manager and an unsanctioned authorized administrator can update it in place; the administrator can also use the phase-independent ERC-721 or ERC-1155 close path.

Impact

An affected drop cannot process otherwise-valid positive purchases until an authorized administrator repairs the rate or closes the drop. This removes mint availability for one collection drop or token ID and can cause buyers' reverted attempts to consume transaction gas.

The revert occurs before settlement and minting and rolls back all earlier state changes; the evidence demonstrates no retained payment, partial mint, or permanent asset loss. This result establishes the code-path behavior, not the presence of an affected deployed configuration or any realized loss. The privileged exact-value precondition, single-drop scope, atomic failure, and available same-round correction make this an unscored hardening observation.

Recommendation

Reject `type(int256).min` consistently in both managers' initial-configuration and live rate-update paths, and defensively enforce the same domain at the duration-adjustment sink so future write paths cannot bypass the restriction.

If the full signed range must remain supported, derive the negative magnitude without first negating it as an `int256`, and explicitly define how the resulting unsigned `2^255` adjustment should affect duration. Rejecting the signed minimum narrows acceptance by one ABI-valid value; supporting it requires deliberate semantics for that magnitude.

Validation objective

Test initial configuration and live updates for both the ERC-721 and ERC-1155 managers. Verify that the signed minimum is rejected before storage or processed without a panic, that the adjacent representable negative value retains its intended behavior, that failures produce no partial effects, and that authorization and round accounting remain unchanged.

OBS-022: Extreme velocity rates can reject otherwise permitted purchase batches

- ID: OBS-022
- Status: New
- Class: Observation
- Location: TLStacks721._updateDropState(); TLStacks1155._updateDropState()

Summary

An extreme negative velocity-decay rate can make an otherwise supply- and allowance-compliant batch purchase revert when the contract multiplies the rate magnitude by the quantity. The atomic failure prevents minting and can cost the buyer transaction gas, although smaller batches remain available.

Description

Expected behavior

Velocity drops should adjust their remaining public duration for every quantity accepted by the supply and per-wallet allowance checks. A negative adjustment greater than the remaining duration should reach the existing zero-duration saturation behavior, or the contract should reject unsupported parameter combinations predictably before a buyer submits a purchase.

Observed behavior

The ERC-721 configuration path and ERC-1155 configuration path accept `-type(int256).max` as a representable decay rate. With remaining supply and per-wallet allowance both set to three, the ERC-721 purchase checks and ERC-1155 purchase checks admit a three-token quantity, but the subsequent rate-magnitude multiplication exceeds `type(uint256).max` and raises `Panic(0x11)` before the negative-duration saturation branch.

The failed purchases are atomic: NFT balances remain zero and available supply remains three. Otherwise-identical one-token purchases fit the multiplication, mint one token through each manager, and set the public duration to zero.

Root cause

`TLStacks721._updateDropState()` and `TLStacks1155._updateDropState()` multiply an unconstrained decay-rate magnitude by the permitted purchase quantity before applying negative-duration saturation.

The initial configuration and the ERC-721 rate-update control and ERC-1155 rate-update control do not bound the rate against the maximum purchasable batch. Purchase validation considers supply and per-wallet allowance, so each input can be valid while their product is not representable; checked arithmetic therefore reverts before the code can compare the adjustment with the remaining duration.

Conditions and assumptions

An NFT contract owner or `ADMIN_ROLE` holder must configure an active velocity drop with an extreme, representable decay-rate magnitude. Remaining supply and per-wallet allowance must then permit a quantity for which `abs(decayRate) * numberToMint` exceeds `type(uint256).max`; any buyer can submit that quantity without protocol privileges.

The demonstrated boundary uses `decayRate = -type(int256).max` and quantity three. This astronomical rate is an exceptional configuration rather than an established normal-use value, which materially limits applicability and likelihood.

Impact

The demonstrated effect is rejection of a three-token purchase that satisfies the configured supply and wallet allowance. The revert is atomic: no tokens are minted, no supply accounting is committed, and purchase funds are not retained, although the submitted transaction can consume gas and integrations can encounter localized sale friction.

The effect is confined to the affected ERC-721 collection or ERC-1155 collection/token pair. Smaller batches remain possible, and administrators retain correction and closure paths; the evidence does not establish asset loss, persistent state corruption, or complete unavailability. Together with the exceptional privileged configuration, this bounded effect supports an unscored observation.

Recommendation

Enforce a closed arithmetic domain for every accepted velocity configuration and permitted batch. Configuration and rate-update paths should reject combinations whose maximum permitted batch can overflow, and purchase paths should defensively check `numberToMint > type(uint256).max / rateMagnitude` before multiplication.

For negative rates, the contracts can compare the adjustment with the remaining duration using division and apply zero-duration saturation without first forming an overflowing product. For positive rates, define and enforce an explicit rejection or bounded-update policy. Rejecting extreme configurations narrows the currently accepted numeric domain, while saturation requires documenting the chosen semantics.

Validation objective

Test both contracts at the largest fitting rate-times-quantity product and immediately beyond it. A supply- and allowance-compliant negative adjustment greater than the remaining duration should either saturate successfully or cause the unsupported configuration or purchase to fail with an intentional custom error rather than an arithmetic panic. Ordinary smaller purchases should retain their existing behavior.

OBS-023: Positive velocity adjustments can overflow the stored duration

- ID: OBS-023
- Status: New
- Class: Observation
- Location: TLStacks721._updateDropState(); TLStacks1155._updateDropState()

Summary

A velocity drop configured with an extreme positive duration or rate can reject an otherwise-valid purchase when its duration adjustment exceeds the remaining integer headroom. The buyer receives no token and may lose transaction gas, while purchases for that ERC-721 drop or ERC-1155 token ID remain unavailable until an authorized administrator changes the configuration.

Description

Expected behavior

Positive “marathon” velocity rates are intended to extend an open edition's duration after each mint, as described in README.md (`README.md:18–20`). Accepted configurations should preserve enough integer headroom for permitted purchases or apply an explicit overflow policy, so an eligible purchase does not encounter an arithmetic panic.

Observed behavior

Both mint managers calculate `durationAdjust = uint256(decayRate) * numberToMint` and add the result to `publicDuration`. The multiplication can fit even when the subsequent checked addition cannot.

Two phase-valid boundary states expose the condition in each manager. A maximum stored duration and rate of one cause the first one-token purchase to panic, while a one-day duration and maximum positive `int256` rate allow the first one-token purchase but cause the second to panic. Each failing transaction rolls back, leaving the duration, supply, and buyer's NFT balance unchanged; setting the rate to zero then allows a purchase to succeed.

Root cause

TLStacks721._updateDropState() and TLStacks1155._updateDropState() add positive velocity adjustments to `publicDuration` without checking its remaining `uint256` headroom.

Configuration and parameter-update surfaces accept the full duration and positive-rate domains without establishing an invariant that later permitted purchases remain representable. The negative-rate branch explicitly saturates at zero when an adjustment exceeds the duration, but the positive branch has no corresponding upper bound or saturation rule.

The ERC-721 purchase path and ERC-1155 purchase path perform the state update before settlement and minting, so the checked overflow reverts the entire transaction.

Conditions and assumptions

An NFT owner or `ADMIN_ROLE` holder must first configure a duration or positive rate near the integer limit. The triggering buyer requires no administrative role but must satisfy the ordinary active-phase, sanctions, supply, allowance, round, currency, and price checks, and the mint manager must be unpaused.

These values are astronomical. The evidence establishes behavior within the accepted parameter domain, not that such parameters are used in production. For the maximum-rate case, the claim is limited to one-token purchases where multiplication fits; purchases of three or more encounter a separate multiplication boundary outside this entry.

An authorized party can repair the configuration by changing the rate or duration when the normal pause and sanctions gates permit, or close the affected drop. The demonstrated recovery sets the rate to zero.

Impact

The demonstrated harm is bounded purchase unavailability for one ERC-721 drop or one ERC-1155 token ID. A buyer submitting the failing purchase receives no token and may lose transaction gas.

The transaction reverts atomically: it retains no payment, consumes no supply, and performs no partial mint. No unauthorized asset movement, cross-drop effect, or permanent loss was demonstrated.

Because the condition requires extreme privileged parameters, remains localized, and has direct administrative recovery, it is an unscored operational hardening observation. Unavailability can continue while the configuration remains unchanged, including when an authorized administrator is unavailable.

Recommendation

Enforce a positive-velocity headroom invariant when configuring a drop and whenever its duration, rate, or supply changes. For example, validate an overflow-safe bound equivalent to `rate <= (maximumDuration - publicDuration) / remainingSupply`, using the chosen maximum duration and all remaining permitted supply.

Alternatively, make the state update saturate at a documented maximum duration. Saturation preserves purchase liveness but changes sale-lifetime semantics, whereas rejecting extreme configurations preserves exact arithmetic semantics while restricting the currently accepted parameter domain. In either design, replace the raw positive addition with an explicit overflow-aware policy and a domain-specific error.

Validation objective

For both mint managers, verify that maximum duration plus a positive adjustment and the maximum-rate first-success/second-purchase sequence follow the selected policy without an arithmetic panic. Confirm that ordinary positive adjustments still extend duration and that rejected configuration changes do not alter supply, balances, payment state, or drop accounting.

OBS-024: Auction self-destinations can leave assets in protocol custody

- ID: OBS-024
- Status: New
- Class: Observation
- Location: `TLAuctionHouse._isValidRecipient()`

Summary

Sales, payouts, protocol-fee distributions, and deferred claims can finalize while an NFT, fungible balance, or inaccessible credit remains in protocol custody because the auction house accepts its own escrow address as a terminal recipient.

Description

Expected behavior

A terminal destination accepted by the auction house should receive the asset outside escrow. If delivery cannot produce that custody transition, the operation should revert before deleting its listing or beneficiary entitlement, or preserve recoverable accounting for the beneficiary.

Observed behavior

`_isValidRecipient` rejects only zero and sanctioned addresses, so the auction house passes its own destination checks.

For NFT delivery, `buyNow` deletes the listing and calls `_completeSale`. A standard ERC-721 transfer from the house to itself succeeds, and `_transferNft` accepts the unchanged `ownerOf(tokenId) == to` state as delivery. The seller is paid, but the NFT remains house-owned without a listing or claim. `claimNft` similarly deletes an existing entitlement before the successful no-op transfer.

For standard ERC-20s, a house-to-house transfer subtracts and re-adds the same amount. A direct payout therefore reports success without paying the seller or recording a credit. `claimFunds` clears the claimant's credit before the same no-op transfer, leaving the house balance unchanged.

Native self-payouts follow a different path. Because the auction house has no payable receiver, `_payout` converts the failed transfer into `claimableFunds[address(house)][address(0)]`. Settlement still completes, but the house exposes no route to originate the claimant-authorized withdrawal. In contrast, a direct native claim to the house reverts atomically and preserves the original claimant's credit.

The package's custody properties require fungible balances to equal live bids plus credits and every house-owned NFT to remain listed or claimable in `TLAuctionHouse.invariant.t.sol:48-74`. The reproduced self-destinations violate those properties, while the invariant handler's destination set excludes the house.

Root cause

`TLAuctionHouse._isValidRecipient()` does not reject the auction house itself as a terminal recipient.

Downstream transfer logic infers completion from nominal transfer success or ownership equality without requiring custody to leave the house. Standard ERC-20 and ERC-721 self-transfers do not revert, while `_payout` converts a failed native transfer into credit for the invalid destination. Transfers that revert roll back state safely.

Conditions and assumptions

An authorized actor must select the exact house address: a buyer for public buy-now delivery, a seller for payout or private-sale delivery, a claimant for deferred recovery, or the owner for protocol fees. No demonstrated path lets an unrelated actor alter another beneficiary's destination.

The NFT and claim-erasure cases depend on standard ERC-721 and ERC-20 self-transfer behavior. A token that rejects self-transfer causes those operations to revert and preserve prior state; ERC-20 payouts remain affected because `_payout` instead records an inaccessible house-owned credit. Ordinary auction settlement does not self-deliver because the house cannot bid through the exposed interface, and a native claim redirected to the house reverts and preserves the original credit.

Applicability is limited to the custody and recovery paths exposed by the accepted `TLAuctionHouse` interface.

Impact

A buyer can pay for an NFT that remains outside every recovery state exposed by the auction house. A seller or the protocol can lose an entire configured payout or fee, and an entitled claimant can erase an entire ERC-20 or NFT recovery record.

These losses are bounded to the self-directed transaction or entitlement. The condition does not demonstrate theft, attacker profit, corruption of another user's claim, or protocol-wide insolvency. The accepted `TLAuctionHouse` interface exposes no recovery path for retained assets; recovery outside that interface is not established.

This is an unscored observation because the participant controlling the relevant destination must select the auction house itself, and no outsider can redirect value. The condition therefore warrants configuration and recipient-safety guidance rather than a scored finding.

Recommendation

Reject `address(this)` in `_isValidRecipient` so the same invariant applies to listing payout receivers, updated payout receivers, protocol-fee receivers, sale recipients, and claim destinations. As defense in depth, `_transferNft` can reject `from == to`, and payout handling can prevent finalization or self-owned credit creation when the destination is the auction house.

Any recovery facility for retained assets should preserve recorded beneficiary ownership and distinguish surplus from live escrow liabilities; a broad owner sweep could introduce a separate custody risk. Rejecting the house also removes the possibility of intentional donations to it, but the current contract provides no useful or recoverable semantics for such donations.

Validation objective

Regression coverage should verify that every self-destination is rejected before payment collection, listing deletion, claim clearing, or receiver-setting mutation, while ordinary external recipients continue to work. Tests should cover native and ERC-20 seller payouts, protocol fees, buy-now and private-sale NFT delivery, ERC-20 and NFT claims, self-transfer-rejecting assets, and the existing native-claim rollback behavior. Stateful destination generation should include the house address and preserve the documented solvency and NFT-custody properties.

OBS-025: Self-address payout receivers can place mint proceeds in protocol custody

- ID: OBS-025
- Status: New
- Class: Observation
- Location: `TLStacks721._checkPayoutReceiver()`; `TLStacks1155._checkPayoutReceiver()`

Summary

TLStacks721 and TLStacks1155 accept their own addresses as payout receivers. Purchases with positive native-currency seller proceeds then revert atomically, while standard ERC-20 purchases complete and place net seller proceeds under the protocol owner's withdrawal authority; if ownership has already been renounced, the supplied contracts provide no authorized path to withdraw retained proceeds.

Description

Expected behavior

The Stacks mint system separates collection-level drop administration from protocol ownership in its multi-tenant design (`README.md:11–21`). A completed paid mint should route net seller proceeds to the selected payee without leaving an unattributed balance in the shared minter; the ERC-721 no-custody invariant and ERC-1155 no-custody invariant encode that expectation. Any intentional protocol custody should retain collection-level attribution and a collection-authorized recovery path.

This condition does not assume that every external contract accepts native currency; settlement may remain fail-closed when an external recipient rejects payment.

Observed behavior

An NFT collection owner or `ADMIN_ROLE` holder can set the payout receiver during initial configuration or a live update. The ERC-721 configuration paths and ERC-1155 configuration paths pass the address to validators that reject only zero or sanctioned receivers, so each contract accepts its own nonsanctioned address.

With positive native-currency seller proceeds, settlement sends an empty-calldata call to the configured receiver through the native transfer helper. A self-call cannot reach a payable receiver on either minter, so the helper reverts and the entire purchase rolls back: the buyer keeps the payment, drop supply remains unchanged, and no NFT is minted.

With a standard ERC-20, the ERC-20 transfer helper transfers net seller proceeds from the buyer to the configured receiver and accepts the resulting balance increase. When the receiver is the minter, ERC-721 settlement or ERC-1155 settlement completes, the buyer receives the NFT, and the proceeds remain in that minter. The collection actor cannot recover this balance; the ERC-721 withdrawal and ERC-1155 withdrawal functions are protocol-owner-only and pay the owner.

If protocol ownership has already been renounced (`dependencies/@openzeppelin-contracts-5.6.1/access/Ownable.sol:69–78`), collection administration remains independent and can still configure and operate a drop. ERC-20 purchases then continue to complete, but no nonzero caller can satisfy the owner-only withdrawal path; the

accepted proofs show former protocol owners and collection actors failing to withdraw, and the ERC-1155 variant also shows ownership restoration failing.

Root cause

`TLStacks721._checkPayoutReceiver()` and `TLStacks1155._checkPayoutReceiver()` accept each minter's own address as an ordinary payout destination even though native settlement cannot pay that address and ERC-20 settlement moves collection proceeds under protocol-owner control.

The validators model receiver validity only as nonzero and nonsanctioned. Settlement then assumes that a successful transfer completes seller payment, but an ERC-20 transfer to the minter creates a globally owner-controlled balance without per-collection or per-drop accounting, while the corresponding native-currency transfer deterministically reverts. Ownership renunciation strengthens the retention consequence; it is not a separate mechanism.

Conditions and assumptions

The condition requires an approved Stacks mint role on the collection and a collection owner or `ADMIN_ROLE` holder to select the exact minter address during configuration or a payout update. The drop must be active and unpaused, and the purchase must produce positive net seller proceeds; relevant parties must pass sanctions checks. ERC-20 custody further requires a conventional allowlisted token plus buyer funds and allowance.

Custody matters when collection and protocol control are distinct and the configuration is unintended. An active, cooperative protocol owner can withdraw and return retained tokens. Package-level inaccessibility additionally requires the currency to have been allowlisted before protocol ownership was renounced and assumes no token-level clawback, migration, or other external recovery.

Only net seller proceeds are affected because seller fees are routed separately. Zero-price sales expose no value. Native-currency self-payout does not create custody: it reverts atomically and remains correctable by an authorized collection administrator.

Impact

Native-denominated self-payout prevents purchases with positive seller proceeds on the affected drop until an authorized collection actor changes the receiver. The revert is atomic, so it does not consume buyer funds or supply and does not mint an NFT.

For standard ERC-20 drops, buyers receive their NFTs but net seller proceeds accrue outside collection control. An active protocol owner can withdraw and return them, whereas an ownerless minter exposes no package-level recovery caller; repeated affected purchases can increase the retained balance. The impact remains limited to net seller proceeds from drops configured with the exact self-address, and no unrelated-attacker redirection or buyer overcharge is demonstrated.

This is an unscored observation because an authorized collection owner or administrator must make the avoidable and conspicuous self-address configuration. Correctly configured drops and unrelated collections are not shown to be affected, but explicit validation would remove a material operational footgun across both mint strategies.

Recommendation

Reject `payoutReceiver == address(this)` in the shared payout-receiver validator of each Stacks implementation so the check applies to both initial configuration and live updates. Preserve the existing zero-address and sanctions checks and atomic failure when an external recipient rejects native currency.

If protocol custody is intended, expose it as an explicit mode with per-collection or per-drop accounting, a collection-authorized claim path, and defined behavior when protocol ownership is renounced. This alternative adds storage, accounting, and trust-management complexity; a dedicated external receiver keeps those responsibilities outside the shared minter. A validation change addresses future configuration but does not recover retained balances, which require a separate migration or token-specific recovery assessment.

Validation objective

Verify that initial configuration and payout updates reject each minter’s own address for native- and ERC-20-denominated drops while ordinary compatible, nonsanctioned external receivers continue to settle. Cover regular and selectable ERC-721 purchases and regular ERC-1155 purchases; successful ERC-20 sales should leave no unaccounted minter balance, and failed native payouts should remain fully atomic. If explicit custody is retained, verify beneficiary attribution and collection-authorized recovery after protocol ownership renunciation.

OBS-026: Global seller-fee changes can reprice active mint drops

- ID: OBS-026
- Status: New
- Class: Observation
- Location: `TLStacks721._settleUp()`; `TLStacks1155._settleUp()`

Summary

Changing a global seller-fee rate can reduce collection proceeds from active ERC-721 and ERC-1155 mint drops because settlement uses the current rate rather than terms bound to the drop round. Preconstructed purchases remain valid, allowing up to 5% of each affected sale to be redirected to the protocol fee receiver.

Description

Expected behavior

Collections should be able to rely on the seller economics approved for an active drop round. A purchase should retain that rate, respect a collection-selected fee ceiling or minimum net proceeds, or revert until the collection accepts revised terms.

Observed behavior

`TLStacks721.purchase()` and `TLStacks1155.purchase()` validate the expected round, currency, and maximum gross token price, but neither binds the seller-fee rate or minimum collection proceeds. The owner can update each manager's relevant global rate through `TLStacks721.setProtocolFeeSettings()` or `TLStacks1155.setProtocolFeeSettings()` without advancing an active drop's round.

For ERC-721, the proof configures an active regular drop at a zero seller fee and prepares a 10 ETH purchase before the owner raises the ETH rate to 500 BPS. The unchanged purchase succeeds, delivering the NFT while paying 9.5 ETH to the collection payout receiver and 0.5 ETH to the protocol fee receiver.

For ERC-1155, the proof repeats the condition for both ETH and a standard ERC-20. Each unchanged 10-unit purchase succeeds at the same round after the corresponding fee rises from zero to 500 BPS, delivering the token while splitting payment 9.5 units to the collection and 0.5 units to the fee receiver.

Root cause

In `TLStacks721` and `TLStacks1155`, settlement reads mutable global seller-fee state at purchase time without binding an applicable rate, fee ceiling, or minimum net proceeds to the active drop round.

The ERC-721 `Drop` record and ERC-1155 `Drop` record store sale terms but no seller-fee commitment. Because owner updates replace the global rates while purchase validation leaves seller economics unchecked, ERC-721 settlement and ERC-1155 settlement deduct the latest rate from collection proceeds.

Conditions and assumptions

The condition requires an active, positive-price, non-waived drop, continued mint authorization, a nonzero owner-selected fee applied after drop configuration but before purchase, and a sale amount large enough that the deduction does not round to zero. The purchase must execute before the collection closes the drop or revokes mint authorization, and economic harm requires a fee receiver distinct from the collection payout receiver.

Only the respective Stacks owner can change the rate. Changes are capped at 5% and event-visible; collections can close a drop or revoke mint authorization, but no enforced notice or activation delay guarantees a response window.

This observation concerns an ordinary trusted fee update that unexpectedly alters an existing collection commitment; it does not establish an unprivileged trigger. If collections explicitly accept any seller fee up to the cap throughout a round, the repricing remains within that accepted trust boundary.

Impact

Each affected sale can redirect up to 5% of gross proceeds from the collection payout receiver to the protocol fee receiver. The reproduced 10-unit sales redirected 0.5 units: the ERC-721 path used ETH, while the ERC-1155 paths covered ETH and a standard ERC-20. In each case, the buyer paid the unchanged gross price and received the asset.

The effect may accumulate across remaining supply or multiple active drops, but aggregate exposure is inferred rather than directly demonstrated. Zero-price and fee-waived rounds incur no seller-fee deduction, and sufficiently small payments may round the deduction to zero.

This is an unscored operational and configuration-design observation because the trigger is an authorized action by a trusted owner, not an untrusted capability. The 5% cap, event visibility, and collection closure or authorization-revocation options bound the risk, although the contracts guarantee no response window.

Recommendation

Bind collection-authorized seller economics to each drop round. Snapshot the relevant ETH or ERC-20 fee when the round is configured, or let the collection specify a maximum fee or minimum net proceeds that purchase settlement enforces.

If global fee increases must remain available, apply them only to newly configured or explicitly opted-in rounds, or delay activation long enough for collections to close affected drops. Preserve round-scoped fee waivers. Snapshotting adds per-round storage, collection constraints add configuration complexity, and delayed activation postpones fee-policy changes.

Validation objective

Configure a regular ERC-721 ETH drop and ERC-1155 ETH and standard ERC-20 drops at a zero seller fee, prepare purchases, and then raise the relevant global rate to 500 BPS. Each existing round should either settle at its collection-approved economics or reject the purchase until the collection accepts the new terms; a new or opted-in round should use the updated rate. Also verify waived rounds, rounding boundaries, and close-and-reconfigure behavior.

OBS-027: Late selected-index conflicts amplify reverted purchase gas

- ID: OBS-027
- Status: New
- Class: Observation
- Location: `TLStacks721._markIndicesMinted()`

Summary

A buyer who obtains earlier inclusion for an index near the end of another buyer's pending selectable batch can force the later purchase to perform many bitmap writes before reverting. Those writes roll back, but the buyer submitting the failed transaction can pay substantially more gas than if the conflict appeared first.

Description

Expected behavior

A selectable batch containing an already-minted or repeated index should revert atomically. First-inclusion allocation can remain first-winner, but the contract should identify an already-invalid batch before performing avoidable storage writes for preceding entries.

Observed behavior

`TLStacks721.purchaseSelected()` validates the phase, allowance, currency, and price, then calls `_markIndicesMinted()` before updating drop state, settling payment, or minting. In the reproduced public-sale sequence, one buyer first purchases the index placed last in another buyer's 64-index array. That conflict-last call consumes 1,501,724 units of nested-call execution gas, versus 83,488 for the same entries with the conflict first, before both calls revert with `IndexAlreadyMinted`.

All 63 preceding bitmap writes in the conflict-last call roll back. The earlier buyer's claimed bit remains set, while the victim receives no NFT and drop supply remains unchanged by either failed call.

Root cause

`TLStacks721._markIndicesMinted()` interleaves conflict validation with bitmap mutation, so caller-controlled index order determines how many storage writes execute before a late conflict reverts.

The function walks the array in order, checking each index's bounds and bitmap state and immediately setting its bit before inspecting the next entry. The vendored `BitMaps` library (`dependencies/@openzeppelin-contracts-5.6.1/utils/structs/BitMaps.sol:9-50`) stores each 256-index bucket in a separate word, so a sparse batch can perform many zero-to-nonzero storage writes before detecting the conflict. Reversion restores state but does not eliminate the execution cost.

Selectable indices are explicitly public, unreserved, and subject to first inclusion (`TLStacks721.sol:467-477`). The advisory condition is therefore not first-winner allocation or conflict reversion; it is the avoidable position-dependent work created by mutating storage before the whole batch has been checked.

Conditions and assumptions

The demonstrated path requires an active selectable ERC-721 public sale, visible pending calldata or equivalent order-flow access, and earlier inclusion. The target index must initially be clear and near the end of the victim's array; the attacker must pass sanctions checks and have recipient allowance and funds for the configured sale price and any buyer-facing ETH protocol fee. Enough post-attack supply and victim allowance must remain for the full victim batch to pass `_validatePurchase()`. An earlier validation failure does not reach the bitmap-write amplification, while private order flow or protected ordering reduces targeting feasibility.

The evidence covers the public-sale path and does not establish presale reachability. The measured magnitude uses 64 indices spaced by 256 with an allowance of 64 and supply of 16,384, causing preceding entries to touch distinct bitmap words; smaller or denser batches perform materially less additional work.

Impact

A successfully ordered conflict can make a buyer's selected-index transaction revert only after consuming substantially more execution gas than an equivalent conflict-first failure. For an independently submitted on-chain transaction, the inferred direct harm is the chain-specific fee paid by the sender for failed execution; the reproduction measured 1,501,724 units in its maximized local case but did not measure a sender debit or native-token amount. This remains an unscored observation because the demonstrated harm is bounded to one transaction whose batch size the sender selected.

The cost is further bounded by configured allowance, remaining supply, calldata size, the transaction gas limit, and the block gas limit. All attempted victim writes roll back; allowance and supply updates, payment settlement, and minting are not reached, and attached native value is returned on revert. The attacker acquires the targeted index, consumes supply and ordinary allowance, receives the NFT, and bears the configured sale price and any buyer-facing ETH protocol fee. The evidence establishes no persistent denial, protocol-wide degradation, asset loss, retained victim payment, or state corruption.

Recommendation

Separate conflict validation from bitmap mutation. Build per-bucket masks in memory, reject out-of-range indices, duplicates within calldata, and intersections with already-minted bits, and only then OR each validated mask into its storage word. A simpler two-pass existing-state check must independently preserve duplicate rejection, because checking only storage before a later write pass would miss repeated clear indices. This change adds reads, memory processing, and successful-path gas; a conservative batch cap can also bound worst-case work, but does not by itself remove position-dependent pre-revert writes.

Validation objective

Add regression coverage showing that placing an existing index first or last cannot produce SSTORE-scale growth from preceding entries. Verify that out-of-range and duplicate inputs still revert, no bitmap bit changes on any failed batch, and valid selectable purchases preserve supply, allowance, settlement, URI, and mint behavior.

Disclaimer

This report documents a time-bounded security assessment of the materials and versions identified in the scope of work. The assessment was limited to the stated scope, assumptions, methods, and information available during the engagement. Unless expressly stated otherwise, it does not cover subsequent code changes, deployment configurations, operational practices, integrations, or third-party systems and dependencies.

A security assessment can reduce risk but cannot identify every vulnerability or guarantee that a system is secure, correct, or fit for a particular purpose. The absence of a reported finding should not be interpreted as evidence that no vulnerabilities exist. The client remains responsible for secure development, testing, deployment, monitoring, dependency management, and ongoing review.

Recommendations are intended to address the specific risks described in this report within the reviewed context. They are not exhaustive implementation specifications, and alternative remediations may be appropriate. The client remains responsible for assessing and testing changes in the context of the complete system. Implementing a recommendation does not by itself establish that a finding has been resolved.

Statuses beginning with “Team” and project-team statements recorded in Notes are project-team representations provided for context; they do not constitute independent verification by this report. `New` and `Open` describe an entry’s Report-stage disposition, not its remediation state. A finding is verified as resolved only when this report expressly records that conclusion. Verification is limited to the documented scope and versions of the remediation review and does not establish that the change introduces no additional risk.

This report is not an endorsement of the project, its team, or any associated product or asset. It does not constitute investment, financial, or legal advice and should not be used as the sole basis for making security, operational, or investment decisions.